

Auto

CSIT 321 - Project

Arbitrage

FINAL THESIS

Advaith Sujith - 7837525

Anushka Roy - 7753226

Asraa Shaikh - 7859557

Mohammed Rayhaan - 7722485

Rhythm Bikram Shahi - 7877304

Certification

Student Number: 7877304, 7837525, 7859557, 7753226, 7722485
Student Phone/Mobile No. 0547876756
Student E-mail: rbs663@uowmail.edu.au
Lecturer Name: Dr. George
Due Date: 11/19/2024
Date Submitted: 11/19/2024

PLAGIARISM:

The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism.
The University of Wollongong in Dubai also endorses a policy of non-discriminatory language practice and presentation.

PLEASE NOTE: STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED

DECLARATION:

I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism. We/I declare that the project proposal has not been used in any UOWD courses before and that this project idea is a total new idea of the project team

Signature of Student: rhythm

Optional Marks:

Comments:

Lecturer Project Proposal Receipt (To be filled in by student and retained by Lecturer upon return of assignment)

Subject: CSIT321

Student / Team Name: AutoArbitrage

Due Date: 02/06/2024

Project Title: AutoArbitrage

Student Number: 7877304, 7837525, 7859557, 7753226, 7722485

Date Submitted: 02/06/2024

Signature of Student: rhythm

Student Project Proposal Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Student/Team Name: AutoArbitrage

Due Date: 02/06/2024

Project Title: AutoArbitrage

Student Number: 7877304, 7837525, 7859557, 7753226, 7722485

Date Submitted: 02/06/2024

Signature of Lecturer:

Executive Summary

This document provides a comprehensive blueprint for the development and implementation of AutoArbitrage, an innovative application designed to capitalize on price discrepancies within the dynamic futures market. By leveraging advanced algorithmic trading techniques and real-time data analysis, AutoArbitrage aims to automate the process of identifying and executing profitable arbitrage opportunities between the Binance and Bybit cryptocurrency exchanges.

A thorough feasibility analysis is conducted to assess the technical, economic, and operational viability of the project. This analysis includes a SWOT analysis to identify strengths, weaknesses, opportunities, and threats, as well as a risk assessment to mitigate potential risks.

This document outlines the detailed functional and nonfunctional requirements of the AutoArbitrage system. These requirements encompass user interface design, data processing capabilities, security protocols, and performance metrics. The high-level and low-level design of the system is presented, detailing the software architecture, database design, and algorithmic strategies employed for arbitrage detection and execution.

The security of user data, funds, and the overall system integrity is a paramount concern. The document addresses security measures such as encryption techniques, access controls, and intrusion detection systems to safeguard against potential threats.

The implementation plan provides a detailed roadmap for the development, testing, and deployment of the AutoArbitrage application. It includes a timeline, resource allocation, and quality assurance procedures to ensure the timely and successful delivery of the project.

By automating the complex process of cryptocurrency arbitrage, AutoArbitrage offers a powerful tool for both novice and experienced traders to maximize their returns and minimize risks. This document serves as a foundation for this innovative solution's successful development and deployment.

Acknowledgment

Team AutoArbitrage would like to express its appreciation to Dr. Milos, our mentor, who inspired us and gave up his precious time during this project. This project could not have been carried out without his unwavering support and direction. Additionally, we would like to express our gratitude to Drs. George Tsaramiris, and Farhad, our capstone project coordinators, for their invaluable feedback and constructive criticism, which ultimately led to our best work. Lastly, we all owe a debt of gratitude to the FEIS faculty professors at the University of Wollongong in Dubai for sharing their expertise with us.

Contents

Programming Languages:	10
APIs: Connecting to Binance and Bybit:	11
WebSockets: Maintaining Persistent Connections:	11
Databases: AWS for Data Storage:	11
UI Framework: Avalonia for Cross-Platform Compatibility:	11
Strengths	13
Weaknesses	13
Opportunities	14
Threats	14
BitsGap	15
Cryptohopper	15
ArbitrageScanner	16
Pionex	16
Research and Learning	17
Technology Exploration and Prototyping	17
Development and Implementation	17
Testing and Validation	18
Documentation and Reporting	18
Presentation and Future Recommendations	18
Login and Sign-up	20
Trade Page	20
News Page	21
Profile Page	21
Transaction Page	21
Wallet page	21
Settings Page	21
Database	21
Security	22

Risk Level	22
Learning Trading	22
Learning C# and Avalonia	23
Trading Account for Testing	23
Security Issues	23
Possible Solutions	24
Avalonia UI	25
XAML:	25
C#:.....	25
MVVM:	26
Potential System Evolution	26
Security Constraints	26
AI Constraints	27
Accessibility Constraints.....	27
4.1 Use Case Diagrams	27
Trading Flow of AutoArbitrage	27
System Management and Configuration	28
Binance and Bybit Exchanges integration	29
AutoArbitrage use case.....	30
4.2 Sequence Diagrams	30
Login.....	30
Registration.....	31
Identifying and Signaling an Arbitrage opportunity.....	32
View Transactions	33
News	33
Profile	34
4.3 Common functions.....	35
Login.....	35
Select Cryptocurrencies	35
Real-time data Acquisition.....	36
Arbitrage Opportunity Detection	36
Start/Stop trading.....	37

View Transactions	38
View Trading News	39
Wallet	39
Performance Requirements	40
Low Latency	40
High Availability and Reliability.....	41
Scalability	41
Measuring Performance.....	41
Security Requirements.....	41
Authentication and Authorization	41
Data Protection	41
Additional Security Issues.....	42
Operational Requirements	42
Legal Requirements	43
1.1 Outline.....	43
1.2 System Overview.....	43
1.3 Functional requirements	44
User Interface (UI)	44
Data Processing	45
Exchange Connectivity	45
Trading Automation	45
1.4 Assumptions and dependencies	45
1.5 Architectural strategies	46
Presentation Layer (Avalonia - Cross - Platform WPF XAML).....	47
Application Layer	47
Notification Service.....	47
Data Storage.....	47
1.6 ER Diagram	53
1.7 Class Diagram.....	54
1.8 User Interfaces.....	55
2.1 Outline.....	63
2.2 Wireframes	63

I. Introduction

1. Innovation Field

AutoArbitrage is a prime example of a new generation of financial technology that uses algorithmic trading to seize opportunities that were previously unattainable by human traders. The application uses WebSockets, a technology that allows persistent, two-way communication between servers and clients, to continuously monitor real-time price data from two of the top cryptocurrency exchanges, Binance and Bybit. AutoArbitrage's primary function, which is to detect and take advantage of price differences between the exchanges, depends on this real-time data feed.

Technology is transforming financial markets, as evidenced by AutoArbitrage's lightning-fast operations, which analyze data and spot opportunities in milliseconds. Given the volume of data and the speed at which prices change in the futures market, human traders cannot manually attain this level of efficiency.

Although AutoArbitrage's main purpose is to find and take advantage of price differences for futures market arbitrage, the application also includes AI-powered sentiment analysis to improve the trading tactics of the application. This creative application of AI aims to assess market sentiment and guide trading decisions by leveraging the abundance of data found in news sources and social media feeds.

2. Problem Statement

There are some significant obstacles in the way of ArbiSoft Incorporated's software solution development. First off,

- Price differences can arise and vanish quickly due to the extreme volatility and unpredictability of cryptocurrency markets, necessitating real-time data processing.
- It is essential to guarantee the integrity and correctness of this data because any mistakes could result in poor trading choices and monetary losses.
- Integrating various web socket protocols and APIs from different cryptocurrency exchanges, each with its technical requirements, is another major challenge. These systems must synchronize and communicate with each other seamlessly.
- Scalability is also an issue because adding more exchanges or more market activity can result in a much larger volume of data.

- Given the sensitive nature of financial data and the frequency of cyber-threats in cryptocurrencies, security is a significant concern. Strong security measures are required to guard against data breaches and illegal access.
- Lastly, the practical usability of the software depends on creating an intuitive user interface that offers concise, actionable insights without overwhelming users. Delivering a dependable, effective, and secure solution requires addressing these issues.

3. Background Study

Investors face a double-edged sword due to the fundamental volatility of cryptocurrency markets, which can result in both large losses and chances for arbitrage profit (Makarov, 2020).

With an emphasis on taking advantage of market inefficiencies, ArbiSoft Incorporated is an expert in cryptocurrency analysis and trading. They focus on two primary areas: futures market strategies and cross-exchange spot arbitrage. While futures market strategies entail locking in agreements to buy or sell assets at a predetermined price later, hedging against volatility, and securing profits, spot arbitrage exploits price differences across platforms.

ArbiSoft uses technologies like REST calls and web sockets to collect and process real-time price feeds to be successful. By analyzing this data, arbitrage opportunities are found, giving users timely signals to take advantage of market inefficiencies. Their objective is to offer resources that enable users to seize profitable opportunities in the dynamic cryptocurrency market.

ArbiSoft has tasked us with creating an app to recognize and seize these opportunities to support their foray into the futures market. By integrating the app with their current technologies, clients will be able to replicate their success with spot arbitrage and navigate the futures market with efficiency and confidence.

4. Our Solution

We used cutting-edge data processing methods and effective algorithms that guarantee real-time data capture and analysis to overcome the difficulties in creating the software solution for ArbiSoft Incorporated. This allows for quick responses to changes in the market. Strict validation and error-checking procedures are incorporated to guarantee data integrity and accuracy and lower the possibility of making poor trading decisions.

To enable smooth communication and synchronization, a versatile and modular architecture is created to integrate multiple web socket protocols and APIs from various exchanges.

To guarantee that there is no delay, the application is multi-threaded. Techniques like load balancing and distributed computing are used to achieve scalability, guaranteeing effective management of growing data volumes.

Sensitive financial data is shielded from cyber threats and unwanted access by the implementation of security measures like access control, authentication, and encryption.

System resilience is improved through routine security audits and updates. To ensure the software is useful and efficient, a user-friendly interface offers concise, actionable insights without overwhelming users. This application provides ArbiSoft Incorporated with a dependable, effective, and secure solution for resolving these issues.

5. Target Niche

By balancing automated trading features with user control, AutoArbitrage hopes to serve a wide range of cryptocurrency investors. Because of this method, it can be used by both beginners who are just starting out in the cryptocurrency arbitrage space and experienced traders who want more control over their trading tactics.

AutoArbitrage provides an easy-to-use interface for beginners. By making arbitrage trading less complicated, these features reduce the barrier to entry. This allows beginners to begin trading without requiring a thorough understanding of technical analysis or market dynamics. They can easily navigate the platform and grasp the basic ideas of arbitrage strategies with the help of the application's clear onboarding guides.

On the other hand, AutoArbitrage's customizable features are advantageous for experienced users. They can adjust their trading tactics on the platform to suit their tastes and risk tolerance. Experienced users who desire more control and the ability to adjust their trading strategy in light of their knowledge and market insights will find this flexibility appealing.

6. Technologies

The system's goals of real-time data processing, identifying arbitrage opportunities, and promoting user interaction depend heavily on these technologies:

Programming Languages:

- **C#:** AutoArbitrage's core application development is built on this programming language, and the framework we used throughout the project is Avalonia UI. Building the desktop

application, managing WebSocket connections, processing API requests, and putting the application's business logic into practice are all made possible by its strong features and compatibility with the .NET framework.

- **Python:** The pre-trained machine learning model, derived from the FinBERT model on Hugging Face, was used to analyze the sentiment of news articles displayed to users in our news tab.
- **SQL:** Generally regarded as a query language, SQL can also be viewed as a programming language, especially when it comes to stored procedures and functions that are unique to a database. AutoArbitrage manages data in its selected database solution using SQL.

APIs: Connecting to Binance and Bybit:

AutoArbitrage relies heavily on APIs (Application Programming Interfaces) to link to cryptocurrency exchanges such as Binance and Bybit. The system can execute trades, obtain real-time price data, and obtain other crucial information from the exchanges thanks to these APIs.

WebSockets: Maintaining Persistent Connections:

- **WebSockets:** To keep enduring connections to cryptocurrency exchanges, this communication protocol is necessary. By enabling the real-time data flow, WebSockets guarantees that AutoArbitrage receives real-time price updates without requiring continuous polling, which can be ineffective and cause delays. This stream of real-time data is essential for spotting and seizing short-lived arbitrage opportunities.

Databases: AWS for Data Storage:

- **AWS (Amazon Web Services):** The system can be set up to make use of a database service provided by the AWS ecosystem rather than a SQL database directly.

UI Framework: Avalonia for Cross-Platform Compatibility:

- **Avalonia:** AutoArbitrage's user interface is developed using this cross-platform UI framework. Avalonia makes the application available to a larger user base by enabling its deployment on both Windows and Linux systems.

7. Summary

The purpose of this document is to give a thorough introduction to AutoArbitrage, a program created to detect and profit from price fluctuations in cryptocurrency futures markets. By integrating with two significant cryptocurrency exchanges, Binance and Bybit, and using real-time price data to notify users of potentially lucrative trading opportunities, AutoArbitrage seeks to clarify how it streamlines and automates the arbitrage trading process. The document lists AutoArbitrage's primary functions,

such as its capacity to identify price disparities, alert users to possible trades, and carry out trades following user-specified parameters.

This document also goes on to describe AutoArbitrage's technical architecture, including the programming languages (C#, Python, and SQL), the use of WebSockets and APIs to connect to exchanges and retrieve real-time data, the use of cloud-based database solutions like AWS for data management and storage, and the selection of Avalonia for cross-platform user interface development. Additionally, it draws attention to the difficulties that arbitrage trading presents, including market volatility, the intricacies of API integration, and the requirement for strong security measures, and it offers solutions to reduce these risks.

Furthermore, by explaining the idea of futures contracts, this document highlights how crucial it is to comprehend futures trading in order to use AutoArbitrage effectively, setting it apart from spot trading. Along with outlining upcoming enhancements and expansions for AutoArbitrage, it also seeks to show how the application can streamline and automate arbitrage trading, making it accessible to both beginner and experienced cryptocurrency investors.

II. Feasibility

1. Project Modification :-

Our initial task was to create AutoArbitrage that would interface with Binance and OKX, two particular cryptocurrency exchanges. Finding and taking advantage of arbitrage opportunities in these exchanges' futures markets was the aim. However as the project progressed, a crucial element surfaced that required a change to the original plan.

We learned that OKX does not allow trading in the Futures market in the United Arab Emirates (UAE). This restriction posed a major challenge because the project was being developed in the UAE. The ability to access futures markets on both targeted exchanges is essential to AutoArbitrage's core functionality. The feasibility of the project was jeopardized in the absence of access to futures trading on OKX within the United Arab Emirates.

We chose Bybit as the second exchange for AutoArbitrage to get around this problem. A trustworthy cryptocurrency exchange that provides a full-featured futures trading platform is Bybit. Importantly, we can continue developing and testing the application without running into regulatory obstacles because Bybit is available within the United Arab Emirates.

2. Scope Statement :-

The scope of this project is to develop AutoArbitrage, an application specifically designed to identify and capitalize on cryptocurrency arbitrage opportunities in the futures market. This application will primarily connect to two leading cryptocurrency exchanges: Binance and Bybit.

AutoArbitrage makes use of WebSockets and APIs to collect real-time price information from Bybit and Binance. This application examines this data and finds disparities in futures contract prices for different cryptocurrencies on these exchanges. The main focus of AutoArbitrage's functionality will be futures arbitrage, which is a technique of making money off of price discrepancies in contracts that commit parties to buying or selling an asset at a specific price on a future date.

AutoArbitrage will be delivered as a desktop application, available for both Windows and Linux operating systems. Users will need to download and install the application on their personal computers to access its features and functionalities. This approach allows for a dedicated trading environment for both Linux and Windows users and provides users with direct control over the application's installation and operation.

3. SWOT Analysis :-

Strengths

- Focus on Future's Arbitrage: AutoArbitrage is an expert at spotting and seizing futures market arbitrage opportunities, because it enables us to tailor the application's features and algorithms to the unique characteristics of futures trading, potentially giving it an edge over more general-purpose cryptocurrency trading platforms.
- Automated High-Speed Execution: AutoArbitrage uses fast APIs and Websockets to execute trades and understand data at speeds incomparable to that of a human.
- Real-Time Data Processing: The application is designed to utilize WebSockets and REST APIs to gather real-time price data from Binance and Bybit APIs. This real-time data processing capability is crucial for effectively identifying and acting upon fleeting arbitrage opportunities in the fast-paced cryptocurrency market.
- Experienced Team Members: Members of the AutoArbitrage team possess a wide range of skills including proficiency in C#, Python, and SQL. They are also constantly driven, ready to learn, and prepared to take on problems head-on and find solutions without giving up.

Weaknesses

- Limited by Performance of Exchanges: AutoArbitrage can only function as well as Binance and Bybit functions. If there are any technical difficulties for either Binance or Bybit or the speed of one exchange is slower than the other, AutoArbitrage will be affected.

- Risk of Slippage: High-speed trading requires extremely high-speed execution; delays by even milliseconds can cause slippage¹.
- Team new to Trading: Our limited understanding of the Crypto Futures Market has been our first challenge since we have never traded prior. Getting familiar with Futures Trading consumed time and caused delays.
- Exchange Fees: The fees that need to be paid between both exchanges will always reduce the profits acquired by a small amount.
- Server Location: The Location of the Binance and Bybit servers plays a role in the speed at which the trades can be executed, And since the Servers are in Tokyo and Singapore respectively, the latency to the United Arab Emirates may cause slippage.

Opportunities

- Growing Cryptocurrency Market: The cryptocurrency market has experienced substantial growth in recent years and continues to expand. This growth creates a larger pool of potential users for cryptocurrency trading tools like AutoArbitrage.
- New Arbitrage in Futures Market: Automated arbitrage traditionally has been only prevalent in spot markets. AutoArbitrage now brings these strategies into the futures market as well.
- Increasing Demand for Arbitrage Tools: As the cryptocurrency market matures, the demand for sophisticated tools that can automate and optimize trading strategies is likely to increase. AutoArbitrage's specialized focus on futures arbitrage could position it well to capture a segment of this growing market.
- Potential for Expansion: AutoArbitrage could be expanded to incorporate additional features, integrate with more cryptocurrency exchanges, or even be adapted for other types of financial markets in the future.
- Experience for the Team: Members of the team will get hands-on experience with Avalonia and Trading and will increase their knowledge.

Threats

- Regulatory Changes: The regulatory landscape for cryptocurrency trading is constantly evolving. Changes in regulations could impact the legality or feasibility of certain trading strategies or even restrict access to cryptocurrency exchanges in certain jurisdictions. AutoArbitrage needs to be adaptable to these changes to remain viable.
- Market Volatility: Although market Volatility plays a huge role in affecting and increasing the amount of profit that can be made, it equally also plays a role in the amount of loss that can be made.
- Security Risks: AutoArbitrage stores the API keys of the users; Access to these keys by bad actors allows them to execute unauthorized trades and send money to themselves.

¹ Slippage: In the context of AutoArbitrage, Slippage refers to the situation where a price of a trade has changed from the expected initial price due to slow execution speed.

- Intense Competition: The cryptocurrency trading software is highly competitive, with numerous established platforms and new entrants vying for users. AutoArbitrage differentiates itself through unique features, performance, or pricing to succeed.

4. Competitive Analysis :-

The Table below represents a Competitor Analysis of AutoArbitrage compared to four other similar tools in the cryptocurrency arbitrage space:

Competitor	AutoArbitrage	Bitsgap	CryptoHopper	Arbitrage Scanner	Pionex
Rest API Integration	✓	✓			
Futures Market Support	✓	✓			✓
Identifying Price Discrepancies	✓	✓	✓	✓	
Connection Between Exchanges	✓	✓	✓	✓	✓

Table II-1: **Competitive Analysis**

BitsGap

An automated trading bot is a feature of the cryptocurrency trading platform BitsGap. These bots use a variety of strategies, such as Dollar-Cost Averaging (DCA) and profit-focused Grid Bots, to execute trades around the clock by connecting to user exchange accounts via API keys. By looking for price differences, it enables users to adjust their trading tactics and arbitrage opportunities. Nelson (2024).

Key difference: AutoArbitrage focuses solely on futures market arbitrage with real-time data, targeting active traders. Bitsgap provides a wider range of automated bots for both spot and futures, requiring more user input and catering to a broader audience.

Cryptohopper

A cloud-based trading bot that provides automated trading based on trading indicators and signals is one of the many features of the flexible platform Cryptohopper. It has an easy-to-use interface,

extensive market analysis tools for numerous cryptocurrencies, and the ability for users to subscribe to external signals (Nielson, 2024).

Key Difference: AutoArbitrage specializes in futures market arbitrage, while Cryptohopper is more focused on spot market trading.

ArbitrageScanner

ArbitrageScanner is a platform and bot that outperforms conventional scanners, supporting coverage of more than 75 CEXs, 25+ DEXs, and 20 blockchains. When matching bundles are discovered, the bot will notify users based on their customized search parameters, which include preferred exchanges, coins, and profit thresholds (Nielson, 2024).

Key Difference: AutoArbitrage concentrates on Futures market arbitrage for active traders, while ArbitrageScanner focuses on spot market arbitrage with features geared toward beginners.

Pionex

One of the first cryptocurrency exchanges to provide free built-in trading bots was Pionex. By enabling the purchase, sale, and trading of cryptocurrency assets in their "physical" form on spot markets or by using future contracts to speculate on price movements in the futures market, it serves a wide spectrum of investors. By offering chances to earn passive income through Ethereum staking and other interest-bearing services, Pionex broadens its offerings beyond its primary trading activities (Nielson, 2024).

Key Difference: AutoArbitrage emphasizes real-time data and active trading within the futures market, while Pionex focuses on automated bots for beginners and passive investors in both spot and basic futures markets.

In a nutshell, AutoArbitrage sets itself apart with its real-time data processing, user-friendly interface for active traders, and specialized focus on futures market arbitrage. By giving users useful insights while maintaining decision-making control, it seeks to close the gap between fully automated solutions and manual arbitrage trading. The choice is based on the experience, trading style, and market preference of the individual trader, as each competitor offers a distinct set of features and target audiences.

5. Work Breakdown Structure

Below is a Work Breakdown Structure of the project based on deliverables:-

Research and Learning

- Cryptocurrency Market Dynamics:- Conducting thorough research on cryptocurrency markets, focusing on futures markets:
 - Volatility and price fluctuations.
 - Futures Market Concepts and Strategies.
 - Market Trends, Behaviors, and Liquidity Dynamics.
- Arbitrage Strategies: Studying various arbitrage strategies, focusing on their applicability to the cryptocurrency futures market.
- Technical Proficiency:-
 - Gaining an understanding of WebSockets and REST APIs for real-time data retrieval and exchange communication.
 - Learning and practicing programming languages such as C# and Python for efficient implementation and interaction with exchange servers.
 - Exploring machine learning concepts and techniques.
- Competitor Analysis: Analyzing existing competitor solutions and their limitations, focusing on their strengths and weaknesses. Identifying areas for differentiation and improvement for AutoArbitrage.

Technology Exploration and Prototyping

- Hands-on Experience: Experimenting with the sample code and tutorials to gain practical experience with WebSockets, REST APIs, and chosen programming languages.
- Basic Functionality Prototypes: Developing small-scale prototypes to demonstrate core functionalities:-
 - Retrieving real-time price feeds from Binance and Bybit
 - Identifying price discrepancies.
 - Displaying trading information and basic user interface elements.
- Testing and Iterations: Conducting rigorous testing and iterative development to ensure functionality, data accuracy, and identify potential issues.

Development and Implementation

- Core Modules: Developing and implementing the main components of AutoArbitrage, including:-
 - Handling real-time data retrieval from exchange APIs using WebSockets and REST calls.
 - Analyzing price data to identify potential arbitrage opportunities based on user-defined parameters and algorithms
 - Generating alerts and notifications for users, presenting relevant information and potential trade opportunities.
 - Creating a user-friendly interface for displaying trading information, settings, and interacting with the application.
- Database Integration: Integrating a database for storing user data, transactions, and potentially machine learning model data.

- Security Implementation: Implementing robust security measures to protect sensitive user data and ensure secure communication.

Testing and Validation

- Unit Testing: Conducting thorough unit testing of individual modules and functions to ensure proper functionality.
- Integration Testing: Testing the interaction between different modules to ensure seamless integration and data flow.
- User Acceptance Testing: Gathering feedback from potential users to assess usability and identify areas for improvement.
- Performance Testing: Evaluating system performance under different load conditions to ensure responsiveness and stability.

Documentation and Reporting

- Technical Documentation: Documenting the code, system architecture, and technical implementation details for future reference and maintenance.
- User Manual: Creating a comprehensive user guide explaining how to use AutoArbitrage effectively.
- Thesis Writing: Writing our thesis, integrating research findings, development process, testing results, and analysis into a cohesive document

Presentation and Future Recommendations

- Preparing a final presentation summarizing the deliverables:-
 - Outcomes and achievements.
 - Key learning and challenges.
 - Future recommendations for improvement.
- Considering potential future enhancements:
 - Automated trading functionality.
 - Integration with more cryptocurrency exchanges.
 - Cross-platform compatibility (MacOS, Android, iOS).

6. Performance Measurement Baseline

Our team leader broke down every project task into its component parts and assigned each team member to it. After completing their assigned tasks, each team member submitted them to the leader for review and assistance. The team leader would compile all of the work, keep an eye on each member's progress, and remind everyone of the tasks' due dates.

It was expected of every team member to complete their work on time and meet deadlines. To ensure everyone had access to the most recent versions, all project-related files were uploaded to a GitHub repository. In order to monitor our progress, our project coordinator was also added to our repository.

To keep the company representative and mentor of our project informed of our progress, our team held meetings with him from time to time. To maintain constant oversight and discuss the next tasks, our team also met on campus regularly. We also kept a weekly diary that details our weekly contributions to the project.

7. Milestones and Associated Dates

This Gantt chart outlines the project timeline for developing the new application. The project spans from early May to early December 2024, detailing each key phase and task involved in the development process.

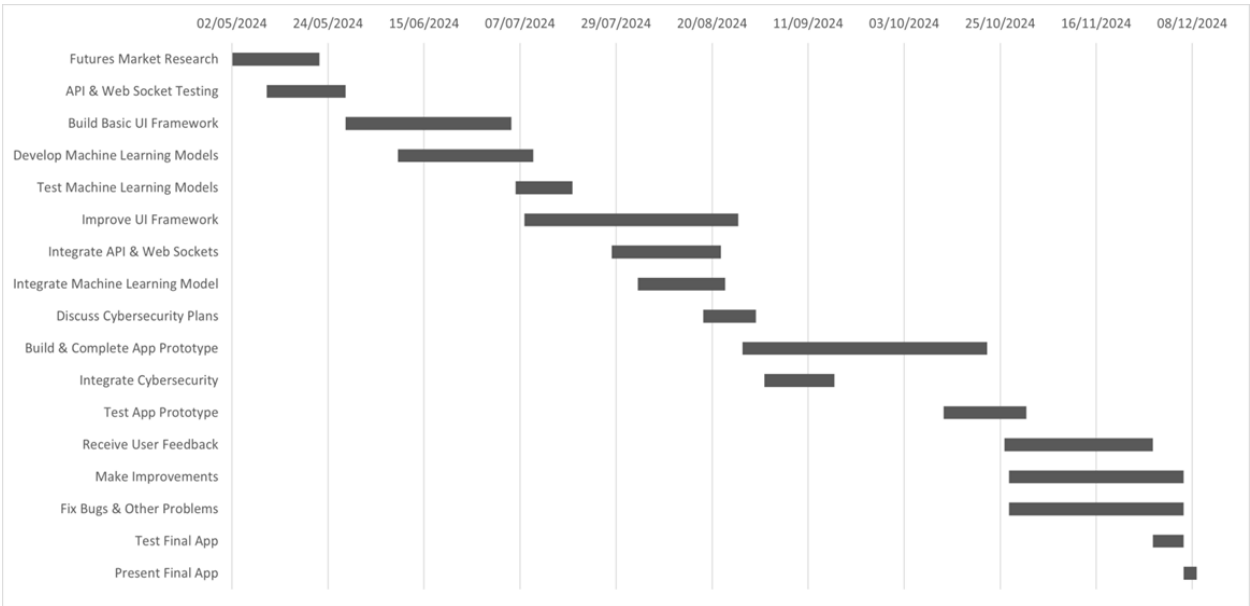


Figure II-1: Gantt Chart

8. Staffing

For this project, a full-time team of five developers—two app developers, one machine learning developer, one cybersecurity specialist, and one UI/UX developer—was needed. We met with our mentor to get professional advice and guidance for our project. We dedicated the first semester (three months) to planning and research to develop clear requirements. During this period, we focused on building the base of our app and gathering as many resources as possible. The first semester was also devoted to learning procedures and comprehending our technology. The second

semester (three months) was set aside expressly for the implementation and testing phases, which included bug fixes and a final system check to guarantee seamless operation.

The team members of this project are:-

- Rhythm Shahi - UI/UX Developer
- Advait Sushruth - ML Developer
- Asraa Shaikh - App Developer
- Anushka Roy - App developer
- Mohammed Rayhaan - Cybersecurity Expert

Throughout the project, Dr. George Tsaramirsis, the coordinator, was available for the weekly meetings. It was always expected of the project coordinator to be accessible and take part in all meetings pertaining to the project until its completion, or later if necessary.

9. Division of Work

	Rhythm	Advait	Asraa	Rayhaan	Anushka
Login and Sign-up					
UI Pages			Asraa		Anushka
Password Validity Checker	Rhythm		Asraa		
Email Validity Checker			Asraa	Rayhaan	
Existing Email Checker	Rhythm				
Trade Page					
UI Page	Rhythm		Asraa		Anushka
Binance Order book	Rhythm	Advait			
Bybit Order book	Rhythm			Rayhaan	
Perpetuals	Rhythm	Advait			
Pricing Units		Advait	Asraa		Anushka
Funding Rates	Rhythm			Rayhaan	
Trading Feature	Rhythm	Advait			

News Page					
UI Page		Advaith	Asraa		Anushka
News Integration	Rhythm	Advaith			
Sentiment Analysis	Rhythm	Advaith			
News Update		Advaith			
Profile Page					
UI Page				Rayhaan	Anushka
Edit and Save Info	Rhythm		Asraa		Anushka
Transaction Page					
UI Page				Rayhaan	
Update Transaction		Advaith		Rayhaan	
Wallet page					
UI Page				Rayhaan	
Account API	Rhythm				
Settings Page					
UI Page		Advaith			Anushka
Tabs Functionality	Rhythm		Asraa		
Database					
AWS Functionality	Rhythm	Advaith			
Login Functionality	Rhythm		Asraa		Anushka
Sign-up Functionality					
News Data Storage		Advaith			
News Data Retrieval		Advaith	Asraa		
Transactional Data	Rhythm	Advaith	Asraa		

User Data Storage	Rhythm				Anushka
User Data Retrieval				Rayhaan	
Security					
Database Security				Rayhaan	
API Key Security				Rayhaan	
User Password Hashing				Rayhaan	
Anti-SQL Injections				Rayhaan	

Table II-2: *Division of Work*

10. Risk Management Plan

Risk Level	
High	The project needs to be stopped if the preparation for this risk is not done properly.
Medium	The project can be continued but not efficiently if the preparation for this risk is not done properly.
Low	The project can be continued without recognizable issues, or the risk is not that big of an issue as the solution is simple.

Table II-3: *Risk level*

Learning Trading

The team had no experience in trading whatsoever, which meant that they had to learn about it before starting the project.

Learning Trading			
	Negligible	Moderate	Severe

Very Likely	Low	Medium	High
Possible	Low	Low	Medium
Very Unlikely	Low	Low	Low

Table II-4: *Learning Trading Matrix*

Learning C# and Avalonia

This was the first project the team was working on a project that required C# and Avalonia, Which would delay the project a little more as they had to learn both of them.

Learning C# and Avalonia			
	Negligible	Moderate	Severe
Very Likely	Low	Medium	High
Possible	Low	Low	Medium
Very Unlikely	Low	Low	Low

Table II-5: *Learning C# and Avalonia Matrix*

Trading Account for Testing

Testing on real trading accounts would be extremely risky as in case something went wrong, real money would be lost.

Trading Account for Testing			
	Negligible	Moderate	Severe
Very Likely	Low	Medium	Low
Possible	Medium	High	Medium
Very Unlikely	Low	Medium	Low

Table II-6: *Trading Account for Testing Matrix*

Security Issues

Users store the API keys to two of their cryptocurrency wallets, meaning AutoArbitrage needs extremely high security to ensure no bad actors can steal the data.

Security Issues

	Negligible	Moderate	Severe
Very Likely	Low	Medium	High
Possible	Low	Low	Medium
Very Unlikely	Low	Low	Low

Table II-7: **Security Issues Matrix**

Possible Solutions

Risk No.	Risks	Possible Solutions
1	Learning Trading	The Team can learn about trading through online courses and YouTube videos, and even with the help of their mentor.
2	Learning C# and Avalonia	The Team can learn about them through online courses and YouTube videos, and even with the help of their mentor.
3	Trading Account for testing	The team can simply use testnet accounts by Bybit and Binance for testing.
4	Security issues	The team needs to integrate serious and multiple security measures to protect sensitive information.

Table II-8: **Possible Solutions**

III. Requirements

1. Assumptions

- The stakeholders are expected to possess a strong understanding of cryptocurrency markets, particularly arbitrage trading opportunities on Binance and Bybit.
- Users are assumed to have sufficient funds in their Binance and Bybit accounts to execute trades, cover potential losses, and comply with platform requirements.
- Secure API keys for both Binance and Bybit accounts are essential to access the features of AutoArbitrage.
- Furthermore, users will need access to a Windows or Linux desktop system to operate the application.

2. Frameworks

Avalonia UI

Avalonia UI was chosen as the primary framework for AutoArbitrage due to its .NET cross-platform compatibility (Anderson,2021). This allowed us to develop a single codebase that can be deployed on both Windows and Linux, streamlining the development process and ensuring a consistent user experience across different operating systems.

Avalonia itself is delivered via the NuGet package manager, which plays a crucial role in simplifying the setup, maintenance, and management of the framework within our development environment. Through NuGet, we were able to easily install and update the necessary Avalonia libraries, ensuring that we always worked with the latest stable version which helped reduce the complexities of manual installation and configuration. With NuGet handling the package management, we could focus more on development, knowing that the framework and its components would be kept up-to-date and compatible with other .NET libraries and tools.

The following are the components that were used under the Avalonia UI framework:

XAML:

XAML is a declarative markup language in Avalonia UI and a framework for the .NETFramework. It is used for defining the visual structure and layout of desktop applications (Abuhakmeh,2021). It enabled us to create visible and reusable UI components through a declarative approach, by allowing us to define the layout and appearance of the interface separately from the application's runtime logic.

XAML's hierarchical structure plays a significant role in simplifying the process of designing complex interfaces. Organizing UI elements in a tree-like structure, made it ideal in visualizing and updating the entire interface. The hierarchical model also facilitates more efficient management of resources like styles, templates, and data bindings, which can be shared across multiple components.

Moreover, the preview functionality significantly sped up the iterative design process, allowing for quick adjustments to layout, color schemes, and element positioning. With Avalonia, we can preview these changes in both Windows and Linux environments, ensuring that the design is consistent across platforms as the application evolves.

C#:

C# was chosen as the backend programming language for AutoArbitrage due to its versatility and seamless integration with Avalonia UI. Due to its powerful features for handling complex logic, through asynchronous programming, it can handle real-time data feeds and process multiple trading tasks simultaneously.

With C# object-oriented programming features such as data encapsulation, inheritance, and polymorphism, C# enables AutoArbitrage to maintain a modular and extensible architecture.

Works well with the extensive .NET ecosystem, offering access to a vast array of libraries and frameworks that significantly enhance the application's performance. These resources enable the efficient handling of network communication, data processing, and mathematical computations, enhancing both the scalability and speed of AutoArbitrage as it processes large volumes of data and executes numerous transactions.

MVVM:

Avalonia's support for MVVM (Model-View-ViewModel) pattern also plays an integral part in our application's architecture, providing a structured approach through its data binding system. This system facilitates seamless data flow by synchronizing the View with the ViewModel, ensuring that real-time trading data is reflected instantly in the interface without requiring manual updates or additional code to handle UI changes.

This real-time data flow is crucial for a crypto trading application like AutoArbitrage, to make informed trading decisions. Through MVVM, the ViewModel acts as an intermediary, allowing the View to remain decoupled from the underlying logic, thus simplifying the UI and making it more flexible to change without impacting core functionality.

MVVM also improves testability, as the ViewModel can be independently unit-tested without needing to rely on the actual UI components. This leads to more efficient development, as we can focus on business logic separately, without worrying about side effects on the UI. MVVM supports easier debugging and enhances code reusability, allowing features like real-time updates, order executions, and dynamic trading metrics to be handled effortlessly

3. Design Constraints

Potential System Evolution

- **Exchange Integration and API Flexibility:** AutoArbitrage could be developed further by integrating additional cryptocurrency exchanges, providing users with expanded trading options and increased opportunities for arbitrage. This will require a flexible approach to support various APIs and exchange-specific protocols seamlessly.
- **System Expansion:** While AutoArbitrage is currently built for Windows and Linux systems, there is potential to extend operability to macOS and mobile platforms like iOS and Android, broadening accessibility.

Security Constraints

- **API Key Management:** AutoArbitrage relies on secure API keys for accessing and executing trades on exchanges. To protect data, the system will securely store and manage these API

keys, using encryption and secure access controls. This is crucial in future expansions to ensure the keys are handled securely during integrations with new exchanges.

AI Constraints

- **AI integration Limitations for Trading Recommendations:** There was an initial consideration to implement AI for providing trading suggestions, but we decided against it due to the inherent risks of exposing financial data to AI algorithms. Relying on AI for real-time trading advice can introduce significant security and ethical concerns, as it requires granting AI systems access to sensitive market and user data. Additionally, the unpredictable nature of AI-driven decisions in volatile markets could pose financial risks to users, which led us to prioritize more secure methods of data analysis.

Accessibility Constraints

- **Accessibility Limitations:** Accessibility considerations were taken into account to ensure the application could be used by a diverse range of users. However, due to time and resource constraints, certain advanced accessibility features, such as support for users with severe impairments, were not fully implemented in this version. Future versions will aim to enhance accessibility by incorporating additional assistive technologies and ensuring compatibility with standards like WCAG (Web Content Accessibility Guidelines).

4. Functional Requirements

4.1 Use Case Diagrams

Trading Flow of AutoArbitrage

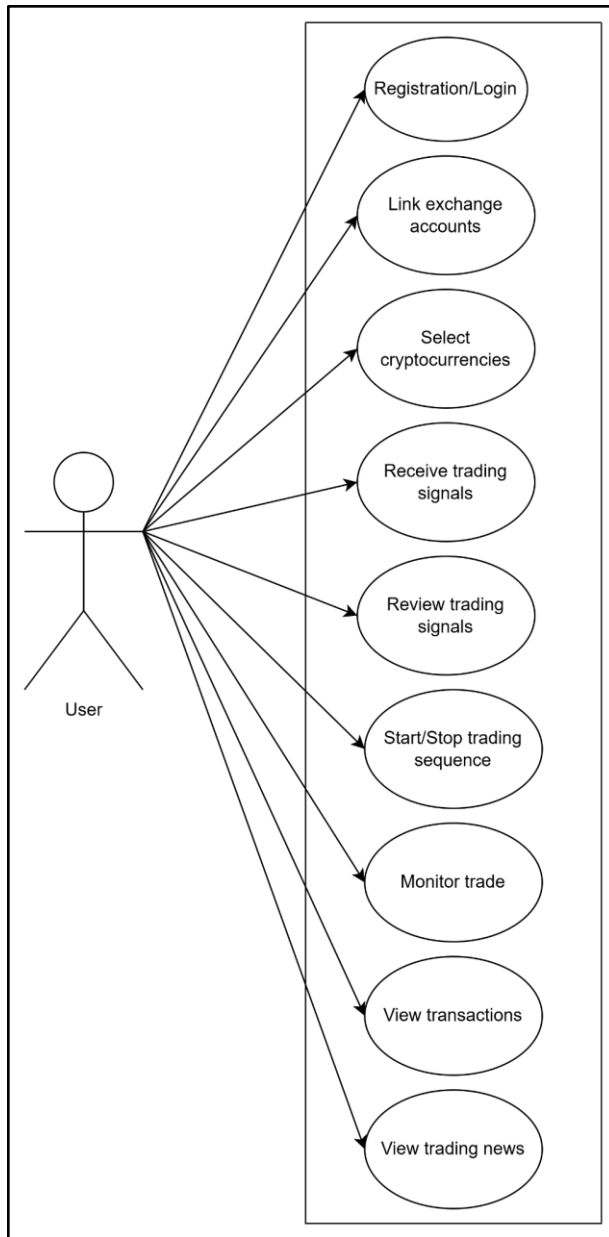


Figure III-1: *Trading Flow Use Case Diagram*

System Management and Configuration

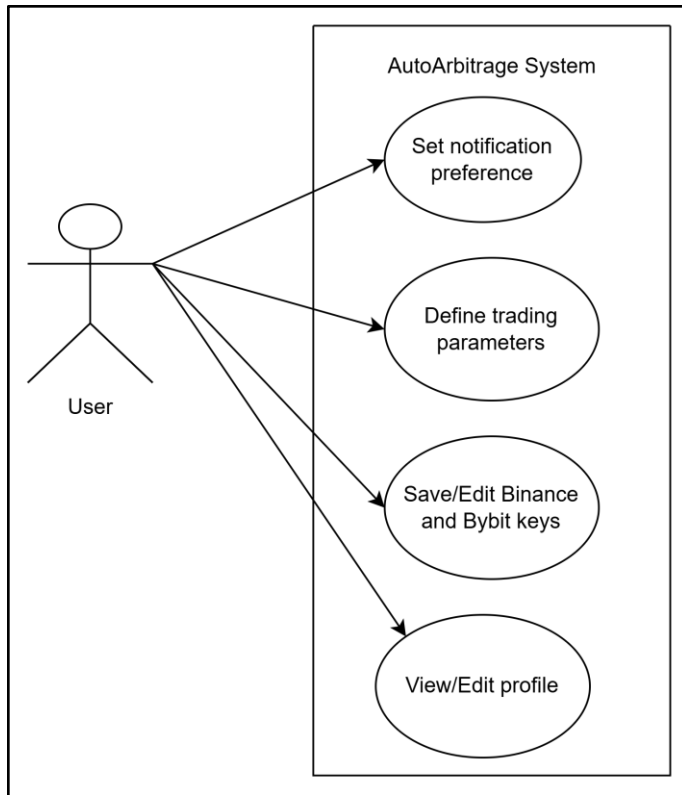


Figure III-2: **System Management and Configuration Use Case Diagram**

Binance and Bybit Exchanges integration

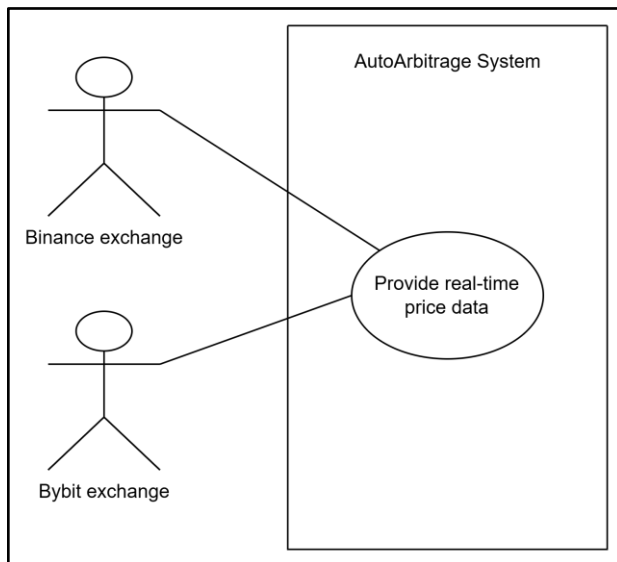


Figure III-3: **Binance and Bybit Exchanges Use Case Diagram**

AutoArbitrage use case

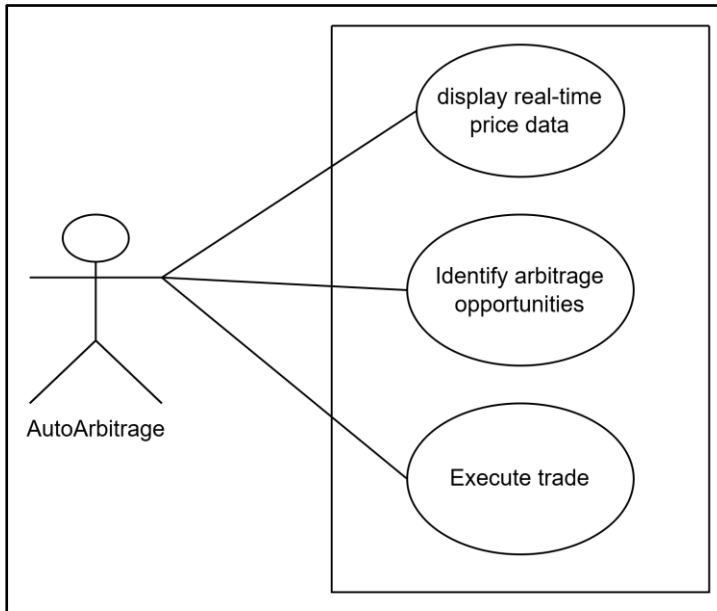


Figure III-4: **AutoArbitrage Use Case Diagram**

4.2 Sequence Diagrams

Login

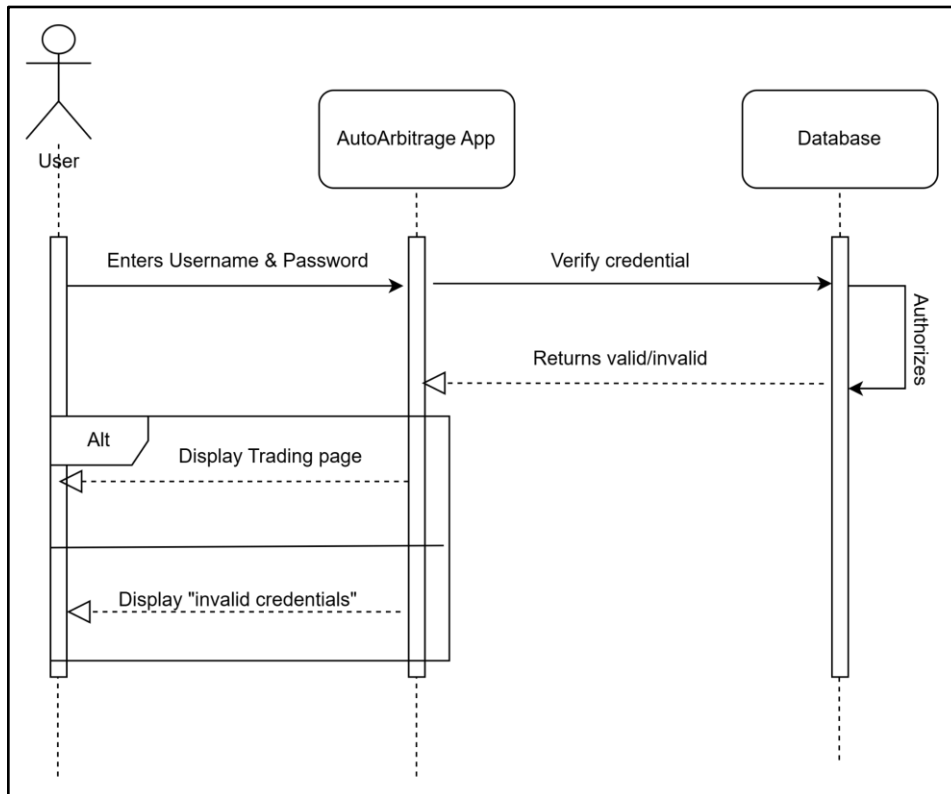


Figure III-5: **Login Sequence Diagram**

Registration

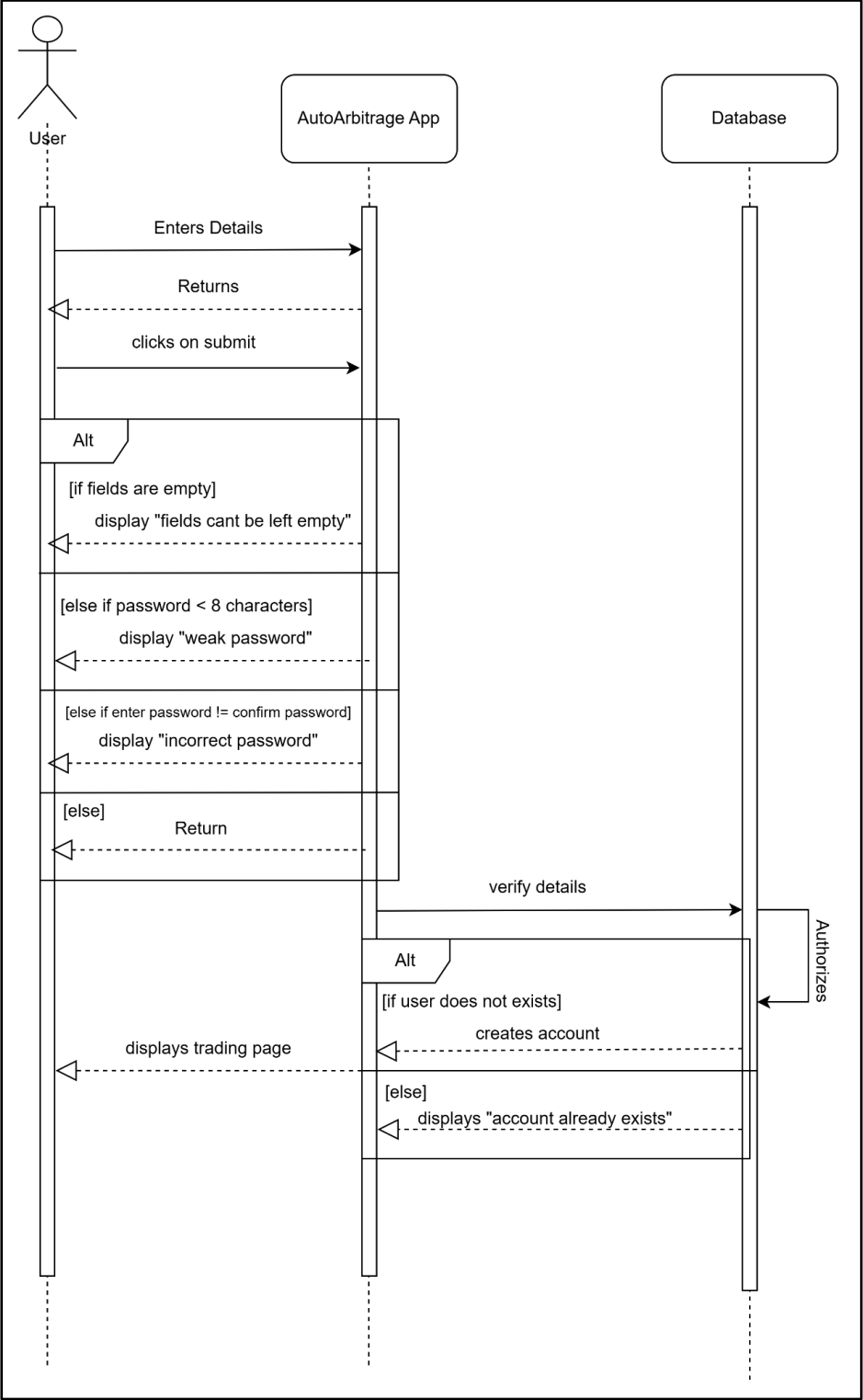


Figure III-6: **Registration Sequence Diagram**

Identifying and Signaling an Arbitrage opportunity

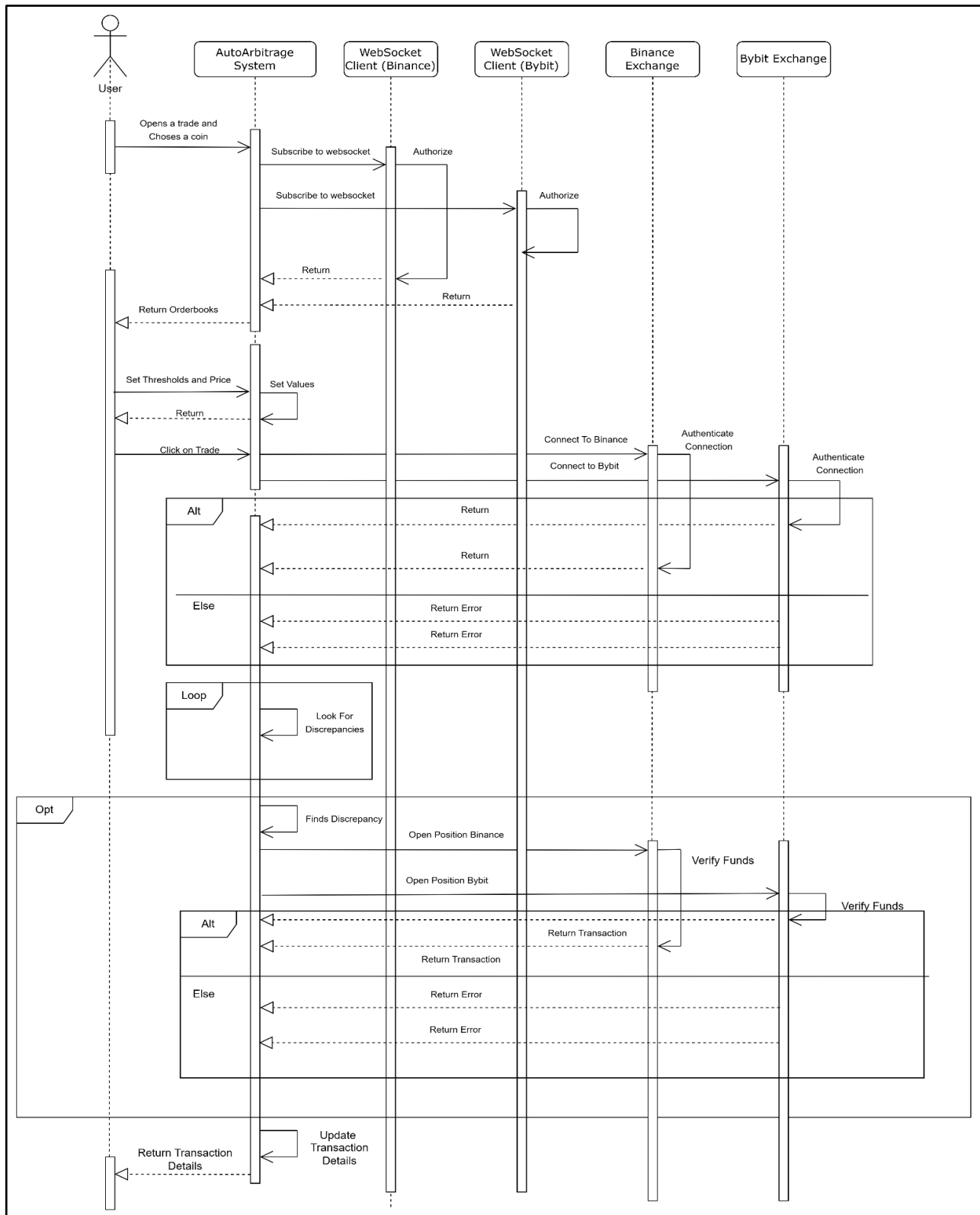


Figure III-7: Identifying and Signaling opportunities Sequence Diagram

View Transactions

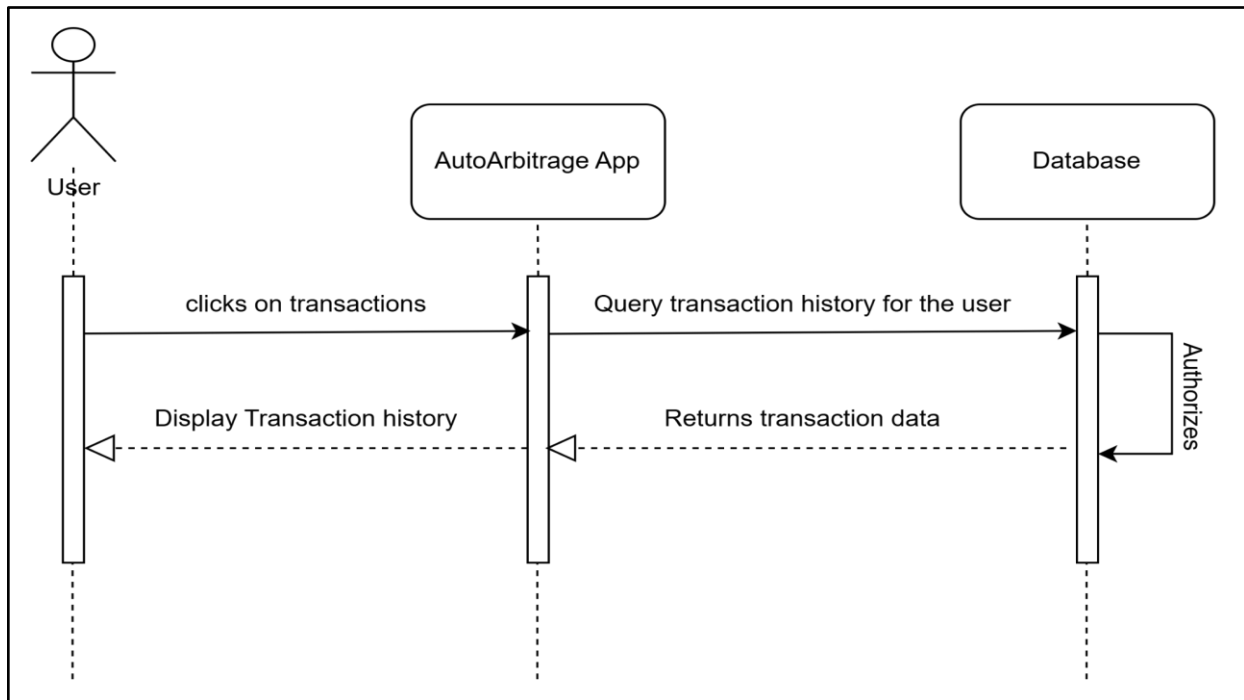


Figure III-8: **View Transactions Sequence Diagram**

News

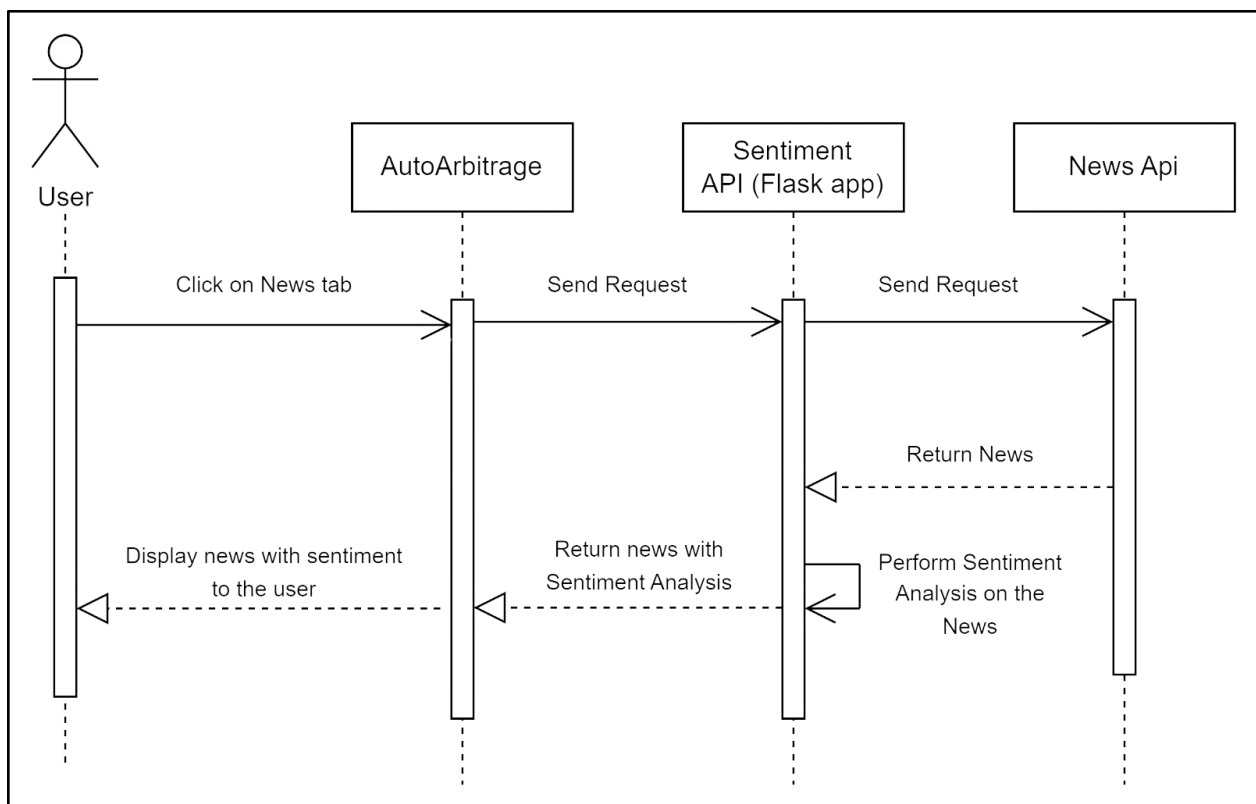


Figure III-9: **News Sequence Diagram**

Profile

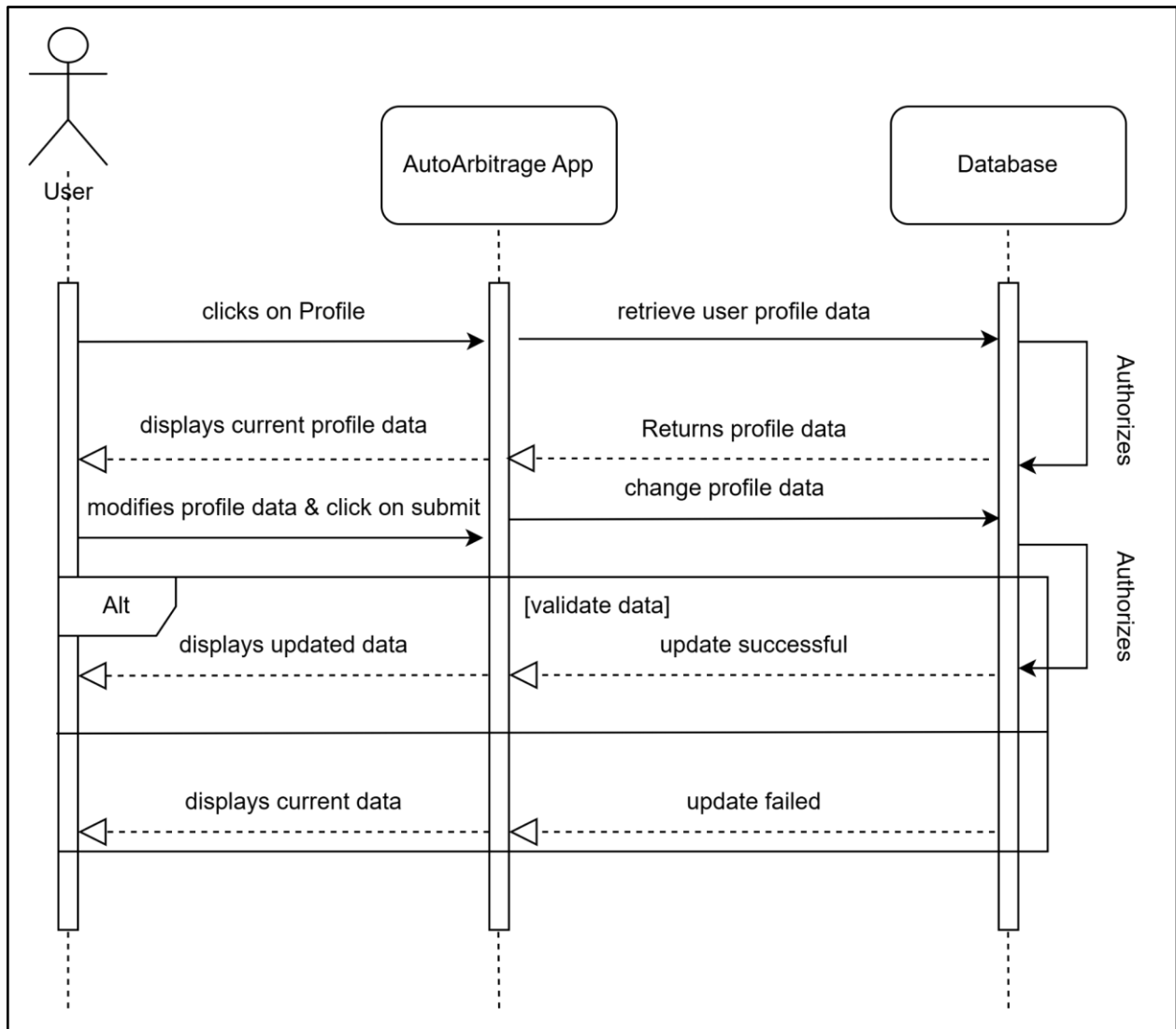


Figure III-9: **Profile Sequence Diagram**

4.3 Common functions

Login

Function	Login
Description	Allows users to login to the AutoArbitrage Application.
Input	Username, password
Source	User
Output	User logs in to the application and is redirected to the main trading page.
Destination	AutoArbitrage System
Action	System verifies user credentials against stored data. If successful, grants access to the user's account.
Requires	Valid username and password.
Pre - Condition	Users must have an existing registered account.
Post - Condition	User is logged in and can access the app's features.
Side-effects	None

Table III-1: ***Login Functional Requirement Table***

Select Cryptocurrencies

Function	Select Cryptocurrencies
Description	Allows the user to choose which cryptocurrencies to monitor for arbitrage opportunities.
Input	List of available cryptocurrencies, User selection
Source	User input
Output	List of selected cryptocurrencies
Destination	AutoArbitrage System
Action	User clicks on preferred cryptocurrency
Requires	Access to the supported cryptocurrencies, User interaction with the interface
Pre - Condition	Users must be logged in.

Post - Condition	The system is now actively tracking the selected cryptocurrencies for potential arbitrage opportunities.
Side-effects	None

Figure III-2: **Select Cryptocurrencies Functional Requirement Table**

Real-time data Acquisition

Function	Real-time data Acquisition
Description	This function establishes and maintains persistent connections to cryptocurrency exchanges (Binance and Bybit) to continuously capture real-time price data for various cryptocurrencies in their futures markets. This data is essential for identifying price discrepancies that signal potential arbitrage opportunities.
Input	None
Source	Binance Futures API, Bybit Futures API, WebSockets
Output	Real-time price feeds for selected cryptocurrency
Destination	AutoArbitrage System
Action	The function utilizes WebSocket connections to subscribe to real-time data streams from Binance and Bybit. The process is continuous to ensure up-to-date information.
Requires	Active and stable internet connection, Secure API connections (Binance Futures API, Bybit Futures API), WebSockets library and implementation in C#, Secure user authentication
Pre - Condition	System must be running and connected to the internet.
Post - Condition	The system continuously receives and updates real-time market data from both Binance and Bybit. This data is then made available for further processing to identify arbitrage opportunities
Side-effects	Network traffic due to continuous data streaming. Minimal impact on exchange server load.

Table III-3: **Real-Time Data Acquisition Functional Requirement Table**

Arbitrage Opportunity Detection

Function	Arbitrage opportunity detection.
----------	----------------------------------

Description	This function continuously analyzes real-time price feeds for selected cryptocurrencies in futures markets from Binance and Bybit. It calculates the price differences for each cryptocurrency across the two exchanges, taking into account trading fees and other relevant factors. The function flags and reports opportunities where the price difference exceeds a user-defined threshold, suggesting a potential for profitable arbitrage.
Input	Users click on preferred cryptocurrency.
Source	Binance Futures API, Bybit Futures API, WebSockets
Output	Open the positions.
Destination	AutoArbitrage System
Action	The function receives a continuous stream of real-time price data from the "Real-Time Data Acquisition" function (). It compares the prices of each selected cryptocurrency across Binance and Bybit, factoring in exchange fees to calculate potential profit margins. Opportunities with a price difference exceeding the user-defined threshold are flagged and reported, indicating a potential for profitable arbitrage.
Requires	System must be receiving real-time price data from exchanges. A minimum price difference threshold must be set by the user.
Pre - Condition	Active and stable internet connection, Access to current fee structures for Binance and Bybit futures trading, Efficient algorithms for calculating price differences and profitability.
Post - Condition	The system continuously monitors and identifies potential arbitrage opportunities based on real-time data and user-defined parameters.
Side-effects	Minimal computational load as analysis involves relatively straightforward calculations.

Table III-4: **Arbitrage opportunity detection Functional Requirement Table**

Start/Stop trading

Function	Start/Stop Trading
Description	Enables the user to activate or deactivate the automated trading system. When activated, the system will automatically execute trades based on identified arbitrage opportunities and pre-defined user settings. When deactivated, it stops trading.

Input	User clicks on start/stop.
Source	User input.
Output	start/stop looking for discrepancies.
Destination	AutoArbitrage System
Action	The system checks the user command. If "Start", it activates the trading engine and begins monitoring for opportunities and executing trades based on defined parameters. If "Stop", it deactivates the trading engine, halting any further automatic trade execution.
Requires	Active connection to exchanges, Valid API keys, Sufficient funds in exchange accounts, Defined trading parameter
Pre - Condition	Users must be logged in. Trading parameters might need to be configured beforehand.
Post - Condition	N/A
Side-effects	N/A

Table III-5: **Start/Stop Functional Requirement Table**

View Transactions

Function	View Transactions
Description	Allows the user to review a history of their completed trades. This could include details like trade time, cryptocurrency traded, entry and exit prices on each exchange, trade volume, fees, and profit or loss per trade.
Input	User clicks on transactions.
Source	User input.
Output	List of past transactions with detailed information for each trade
Destination	AutoArbitrage System
Action	The system retrieves the user's transaction history from the system logs.
Requires	Access to stored transaction data, Secure user authentication.
Pre - Condition	Users must be logged in.
Post - Condition	The user has a clear view of their past trading history. They may gain insights into their trading patterns and overall profitability.

Side-effects	None
--------------	------

Table III-6: **View Transactions Functional Requirement Table**

View Trading News

Function	View Trading News
Description	This function would allow users to access a curated feed of news articles, market analyses, and potentially social media sentiment data relevant to cryptocurrency futures trading. This information could help users stay informed about market trends, breaking news, and potential trading opportunities.
Input	User clicks on News.
Source	User input.
Output	A feed of relevant news articles, headlines, summaries, links to full articles with sentiment analysis.
Destination	AutoArbitrage System
Action	Moves mouse to xyz
Requires	Access to reliable news data sources (Futures News API), Secure User Authentication
Pre - Condition	Users must be logged in.
Post - Condition	The user is presented with a stream of current and relevant news related to the cryptocurrency futures market. This could improve their awareness of market conditions, potential risks, and emerging trading opportunities
Side-effects	None

Table III-7: **View Trading News Functional Requirement Table**

Wallet

Function	Wallet
Description	Displays a comprehensive overview of the user's cryptocurrency holdings, including available funds for trading and total equity.
Input	Users cryptocurrency holdings
Source	Cryptocurrency exchange API

Output	Visual representation of the user's portfolio, including available funds for trading, total equity, breakdown of holdings by cryptocurrency.
Destination	AutoArbitrage System
Action	The system retrieves the user's cryptocurrency holdings from the exchange API and presents them in a user-friendly format.
Requires	User authentication, Access to cryptocurrency exchange API
Pre - Condition	Users must be logged in.
Post - Condition	User can view their current cryptocurrency holdings and their overall portfolio value.
Side-effects	None

Table III-7: *Wallet Functional Requirement Table*

5. Non-Functional Requirements

Performance Requirements

Low Latency

- Real-time Data Processing: In order to successfully take advantage of arbitrage opportunities in the ever-changing cryptocurrency markets (Binance, Bybit) continuously stress the necessity of real-time data processing. All Arbitrage components, from data collection to trade execution, were tuned for speed and low latency.
- WebSocket Advantages: WebSockets are used to connect in real time with cryptocurrency exchanges (Binance, Bybit). AutoArbitrage maintains a persistent connection, WebSockets eliminates the overhead of repeated requests that come with conventional HTTP requests, enabling data to stream back and forth continuously.
- Efficient trading algorithm: Efficient trading algorithms for data parsing, price comparison, and profitability calculations are essential to keep processing times low. AutoArbitrage's calculations are performed faster, and the system is more responsive to fluctuations.
- Latency Test Cases: We also have Latency test cases that are designed to measure the system's response time for actions like placing market orders and receiving market data updates. These tests aim to ensure that the system operates within acceptable time limits for critical operations.

High Availability and Reliability

- System Uptime: AutoArbitrage is designed to remain operational with minimal downtime, since AutoArbitrage deals with financial transactions it requires high availability.
- Redundancy and Failover: To reduce downtime, AutoArbitrage is designed such that it could include redundancy and failover systems. This might involve using multiple servers or data sources so that if one fails, the system can smoothly switch to a backup.
- Exchange Uptime Dependency: AutoArbitrage is reliant on the dependability and uptime of the cryptocurrency exchanges being utilized (Binance, Bybit).

Scalability

- Handling Increased Data Volumes: AutoArbitrage is built to manage increasing data volumes as the number of users, cryptocurrencies tracked, or exchanges integrated rises. In order to keep any one server from becoming overloaded, this may entail strategies like load balancing, which divides incoming traffic among several servers.
- Database Performance: AutoArbitrage stores data efficiently as its role in scalability. AutoArbitrage's database is able to store and retrieve large amounts of data without becoming a bottleneck.

Measuring Performance

AutoArbitrage's Performance metrics is tracked through:-

- Latency: The Average time AutoArbitrage takes to process incoming data, identifying arbitrage opportunities, and executing trades.
- Throughput: The number of trades or transactions AutoArbitrage handles per unit of time.
- Uptime/Downtime: Monitors how available AutoArbitrage is over a specific period
- Resource Utilization: Monitors how AutoArbitrage's resources like CPU, memory, and network bandwidth are being used.

Security Requirements

Authentication and Authorization

AutoArbitrage uses Robust mechanism to verify user identity:-

- Secure Login Mechanisms: AutoArbitrage makes use of a strong password mechanism where the password length needs to be at least 8 characters.

Data Protection

How AutoArbitrage Safeguards user data, including API keys, credentials, and transactions history :-

- Encryption: To Safeguard data during transmission and storage, AutoArbitrage uses encryption algorithms. This stops sensitive data from being accessed by unauthorized parties.
- Data Integrity: AutoArbitrage ensures data integrity by using hashing techniques which creates a distinct “fingerprint” of the data to identify any unauthorized changes.
- Secure Data Storage: To avoid unwanted access, AutoArbitrage keeps sensitive user data in a safe database with suitable access control.

Additional Security Issues

- MySQL Injection Vulnerability: AutoArbitrage successfully confirms test cases by blocking SQL injection attempts
- AWS Security Protocol Integration: AutoArbitrage utilizes AWS (Amazon Web Services) security protocols to improve access control and data protection, this implies that the overall security posture of AutoArbitrage is significantly influenced by its deployment environment.

Operational Requirements

- Stable Internet Connection: High speed internet connection is essential for the application to function properly. Given that AutoArbitrage relies on real-time data from exchanges like Binance and Bybit any disruption in connectivity would delay operations and prevent trade execution.
- API Key Authentication: Users must provide valid API keys from Binance and Bybit accounts to securely access and interact with the exchanges. This is a mandatory requirement for the application to authenticate and establish secure connections to retrieve market data and execute trades.
- Continuous Exchange Availability: Our application heavily relies on the availability of the Binance and Bybit APIs to fetch real-time market data and execute trades. Any downtime or issues with the APIs from these exchanges will result in the system being unable to carry out arbitrage actions, making the uptime of these APIs a critical operational requirement.
- News API Integration: AutoArbitrage relies on News API to gather real-time positive and negative news related to cryptocurrency markets. This information is essential for assessing market sentiment and making informed trading decisions. Timely and accurate news updates are crucial for identifying potential arbitrage opportunities and adjusting trading strategies accordingly.

Legal Requirements

- Intellectual Property Rights: AutoArbitrage respects third-party intellectual property rights by ensuring that all libraries,API's and software components are used under proper licenses and supervision.
- Compliance to Exchange Policies: Operations in AutoArbitrage comply within the terms and conditions of each integrated exchange .This involves abiding with each exchange's specific API usage limits ,data access permissions and restrictions on arbitrage trading ,guaranteeing all operations are within authorized control.
- Risk Awareness: There are well-known risks involved in cryptocurrency trading.We make sure to inform and educate our users on potential risks through certain terms and conditions , helping them be safe and make informed decisions .We prioritize safeguarding our users cryptoassets by encouraging safe trading practices and up to date security measures

IV. Design

1. High-Level Design

1.1 Outline

The design of this software will encompass a comprehensive analysis of its functional requirements, explicitly stating any assumptions made during the design process and dependencies on external systems or services. This section will visually represent the system's architecture through comprehensive diagrams, including an ER diagram illustrating the relationships between entities within the system's database and a class diagram outlining the structure and interactions of the software's classes. Furthermore, this section will showcase the user interfaces, providing a clear representation of how users will engage with the system. The design will consider security measures to protect sensitive user data, particularly API keys for cryptocurrency exchanges like Binance and Bybit, and will aim for scalability to handle potential increases in user traffic and data volume.

1.2 System Overview

AutoArbitrage is a desktop application designed to capitalize on the fleeting price discrepancies that occur in the cryptocurrency futures market. The volatility of cryptocurrencies makes it very difficult to manually find and execute arbitrage trades across many exchanges, such as Binance and Bybit. By automating the process of identifying these price differences and giving customers the data they require to make prompt trading decisions, AutoArbitrage solves this problem.

The system uses WebSockets, a technology that allows for quick, two-way communication, to continuously monitor real-time price feeds from Binance and Bybit. Since even slight disparities in prices can vanish in milliseconds, this real-time data is essential. After analyzing the data, AutoArbitrage finds potentially lucrative chances and provides the user with an understandable presentation of these options.

With both manual and automated trading options, the application serves a broad spectrum of customers, from novices to experienced traders. The application's user-friendly design, which provides obvious indications of potential arbitrage possibilities, is advantageous to new users. By establishing thresholds for automatic trade execution, seasoned users can personalize the application. This enables individuals to specify their level of risk tolerance and customize the application to fit their own trading methods.

AutoArbitrage has a transactions page that lets users see and manually confirm every trade before it is done, ensuring them keep control over their money. Additionally, in order to prevent AutoArbitrage from executing trades without the user's consent, the program requires users to have accounts and API keys for both Binance and ByBit.

The architecture of the system mainly uses XAML for UI/UX design, Python for machine learning models, and C# for core development. Avalonia is used in the development of the cross-platform user interface, which allows for deployment on Linux and Windows. The application can retrieve both static and real-time data by interacting with the exchanges via a combination of WebSockets and REST APIs.

A thorough summary of the user's cryptocurrency holdings is shown on the wallet page. It clearly shows the amount of money that is available for trading. The overall equity, which shows the cumulative worth of all assets—including available funds and additional tokens or cryptocurrencies held. The composition of the portfolio may be clearly understood thanks to the pie chart, which shows how the entire equity is allocated among different assets.

1.3 Functional requirements

User Interface (UI)

The platform's UI is crafted to provide a seamless and intuitive experience for users of varying technical expertise. It displays essential trading information, such as arbitrage opportunities, real-time price feeds, and account statuses, in an organized and visually appealing format. Users can interact with the system to execute trades, monitor transaction histories, and manage account settings effortlessly. Accessibility across devices ensures that users can manage their trades on desktops, tablets, and smartphones, enhancing flexibility. Furthermore, the UI incorporates interactive charts and visual indicators to help users better understand market trends and make informed decisions.

Data Processing

AutoArbitrage employs efficient data processing mechanisms to manage the high volume and velocity of cryptocurrency market data. Using RESTful APIs and web socket calls, the system retrieves and processes real-time data from multiple exchanges. Advanced data cleaning and validation techniques ensure the accuracy and reliability of the data before analysis. To further enhance trading outcomes, machine learning models are integrated into the platform, analyzing historical data and identifying patterns that aid in predicting market behavior. This integration enables the platform to provide actionable insights and recommend optimal trading strategies, empowering users to make informed decisions in real time.

Exchange Connectivity

The platform ensures seamless and secure connectivity with at least two cryptocurrency exchanges, establishing reliable channels for real-time price feeds. Web socket connections are used to stream data continuously with minimal latency, allowing users to act swiftly on arbitrage opportunities. API integration with exchanges is carried out securely, adhering to best practices for API key management, including encryption and access control. By maintaining robust connections and ensuring compatibility with exchange APIs, AutoArbitrage guarantees uninterrupted access to market data and trading functionalities, even in volatile conditions. The modular nature of this component also allows for the addition of more exchanges in future updates, increasing the platform's scalability and reach.

Trading Automation

One of the core functionalities of AutoArbitrage is trading automation, which simplifies the arbitrage process for users. The platform employs sophisticated algorithmic strategies to execute trades based on predefined criteria, such as target profit margins and risk thresholds. Users can configure these parameters to align with their trading preferences, such as selecting specific cryptocurrency pairs, setting minimum profit percentages, or defining acceptable levels of slippage. Once configured, the system continuously monitors market conditions, identifies arbitrage opportunities, and executes trades automatically. This automation eliminates manual intervention, reducing the risk of missed opportunities due to delays or human error. Additionally, users are provided with the option to review trade logs and modify strategies dynamically, ensuring adaptability to changing market conditions.

1.4 Assumptions and dependencies

- Binance and bybit are our data sources, but their uptime and real-time data feeds are not guaranteed. We expect reliability, but external factors can cause outages.
- Exchange transaction fees are assumed to be consistent, allowing for reliable profit calculations. However, external factors can cause fluctuations.

- Users are expected to maintain adequate funds in both accounts to facilitate seamless trading activities.
- Users are assumed to have proficient trading experience, comprehensive knowledge of cryptocurrency markets, and a thorough understanding of both the potential rewards and inherent risks involved in cryptocurrency trading.

1.5 Architectural strategies

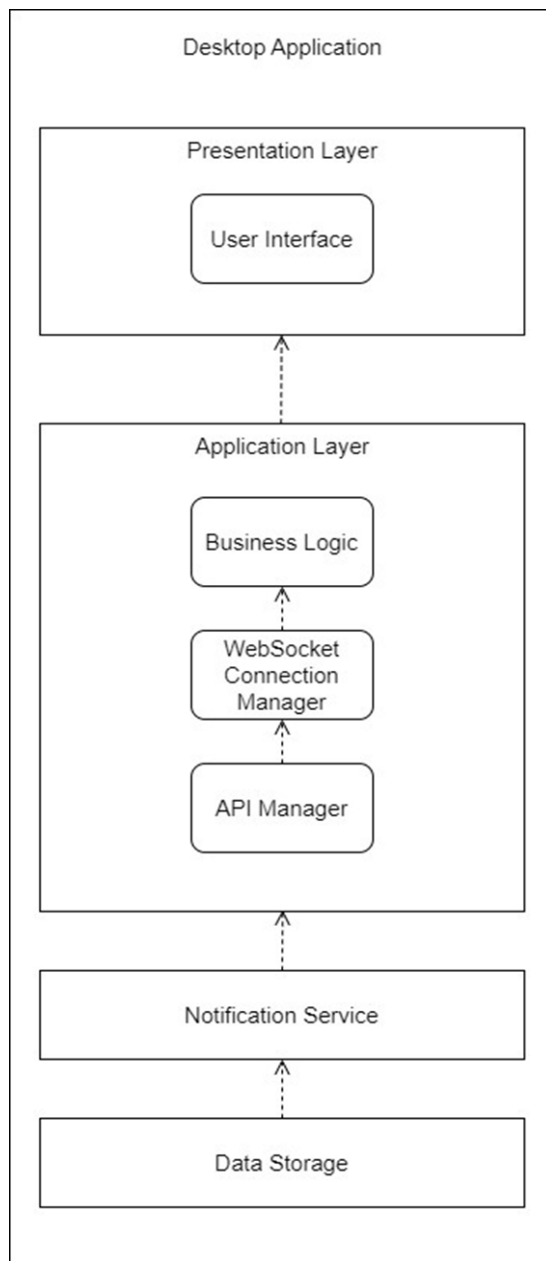


Figure IV-1: **Application Architecture Design**

Desktop Application (C#/.NET Framework)

C# on the .NET Framework is used to build the application's overall container.

Presentation Layer (Avalonia - Cross - Platform WPF XAML)

User Interface (UI)

- Displays Real-Time Price Data: Binance and Bybit's most recent prices are displayed.
- Notifications and Alerts: Informs users of price differences.
- User Settings and Configurations: Gives users the ability to customize notifications, thresholds, and other parameters.

Application Layer

Business Logic

- Process Real-Time Data: Looks for discrepancies in incoming data.
- Detect Price Discrepancies: This method looks for significant price variations using algorithms.
- Choose When to Notify Users: Uses rules to choose when to send alerts.

Web Socket Connection Manager

- Establish WebSocket Connection: Connects to the Bybit and Binance WebSocket APIs to create WebSocket connections.
- Manage incoming streams of data: send data to the Business Logic after processing it.
Manager of APIs
- Handle HTTP API Requests: Retrieves more information from Bybit and Binance on a regular basis.
- Get Required Extra Data: Gets information such as past pricing and account balances.

Notification Service

- Alert Users via UI: Notifies users via the user interface.
- Support for Various Notification Types: Offers a range of alert systems, including pop-ups and noises.

Data Storage

- Log Historical Data: Preserves information for further examination.
- Store User Settings: Preferences and configurations made by the user are saved.

We have opted for an event-driven system architecture with a mediator topology to represent the core functionality of our system. This architecture handles increasing data volumes and facilitates future integration of new processing units.

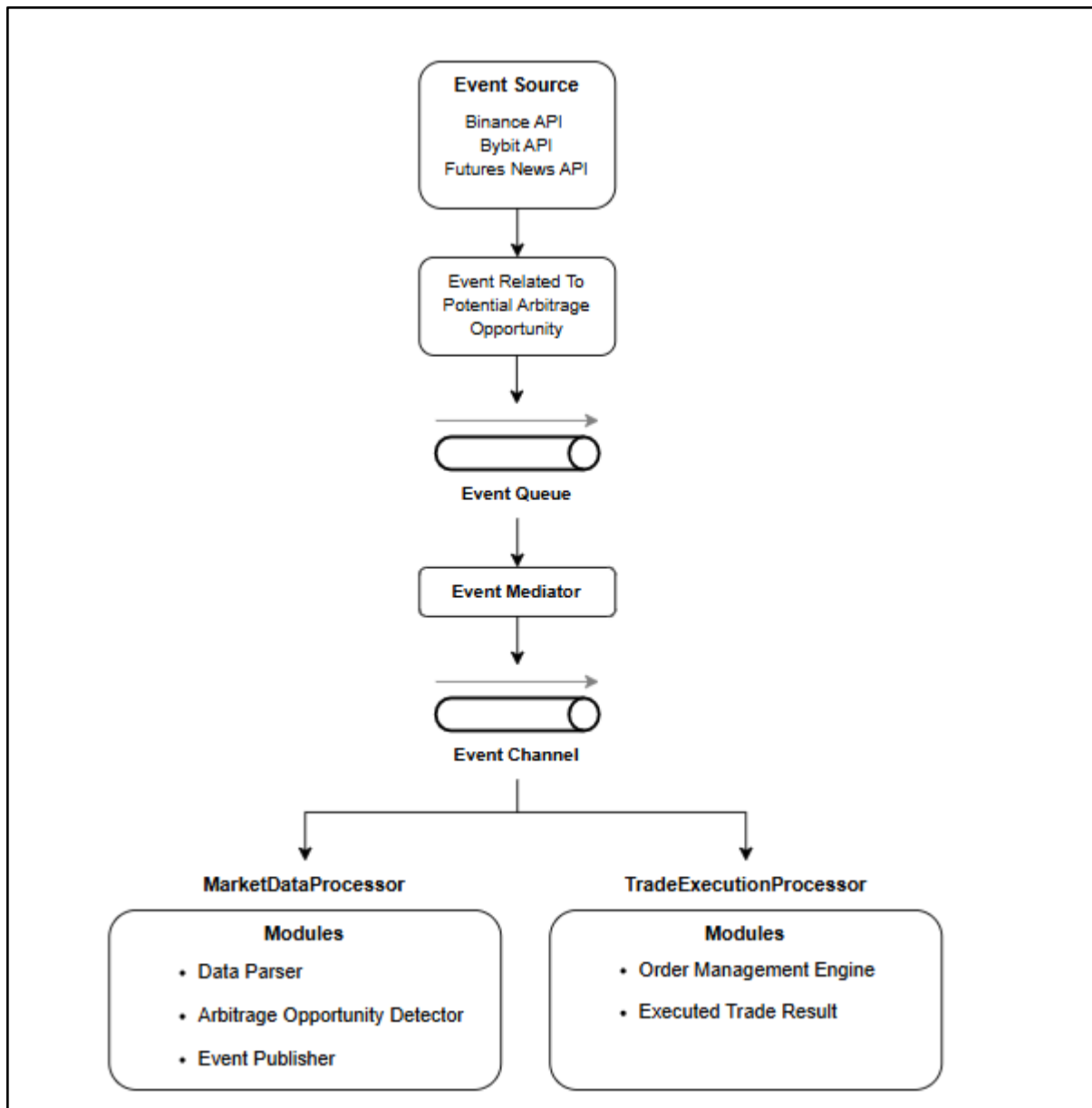


Figure IV-2: **System Architecture Diagram**

Components

- **Event Source:** Retrieves any event related to arbitrage trading focusing on market data from cryptocurrency exchanges like Binance and Bybit and market news APIs through Futures News API sources.

- Event Queue: This component acts as temporary storage for all events published by the Event Source. It maintains a buffer between data feeds and processing units, allowing them to operate synchronously.
- Event Mediator: This central component acts like a conductor, receiving the events from the Event Queue. It analyzes each event to determine its type and directs it to its appropriate processing unit via the Event Channel.
- Event Processor: The system utilizes event processors that consist of different modules, each subscribed to specific events of interest. The following are the processors we will develop for the initial stage:
- Market Data Processor: Analyzes detailed opportunity data from arbitrage events and market trends to confirm validity and potential profit. This processor will include modules such as
 - Data Parser: Parses detailed opportunity data and performs additional checks to validate potential profitability.
 - Arbitrage Opportunity Detector: Identifies arbitrage opportunities based on parsed market data.
 - Event Publisher: Publishes confirmed opportunities as new events back to the queue.
- Trade Execution Processor: Executes trades on the exchanges to capitalize on the confirmed opportunities. This processor will include modules such as
 - Order Management Engine: Interacts with the exchange APIs to buy or sell orders based on published events.
 - Executed Trade Result: Includes details about whether the trade was successful or not and the executed prices for tracking purposes.

Connection and Integrations

- The event source will utilize C# libraries to connect with the exchanges and we will employ C# code to handle REST API requests and responses as well as manage real-time data flow through WebSockets.
- A queuing system operating on a FIFO (First-In-First-Out) principle, acts as the intermediary that interacts with C# code, for retrieving and storing events published.
- Based on the event type, C# code written as the mediator routes events to the appropriate processing unit through the Event Channel
- Each Processor (Market Data Processor, Trade Execution Processor) is a C# module subscribed to an event.
- Executed Trade Results are then recorded into a SQL database to be used for performance analysis and identifying profitable trading strategies.

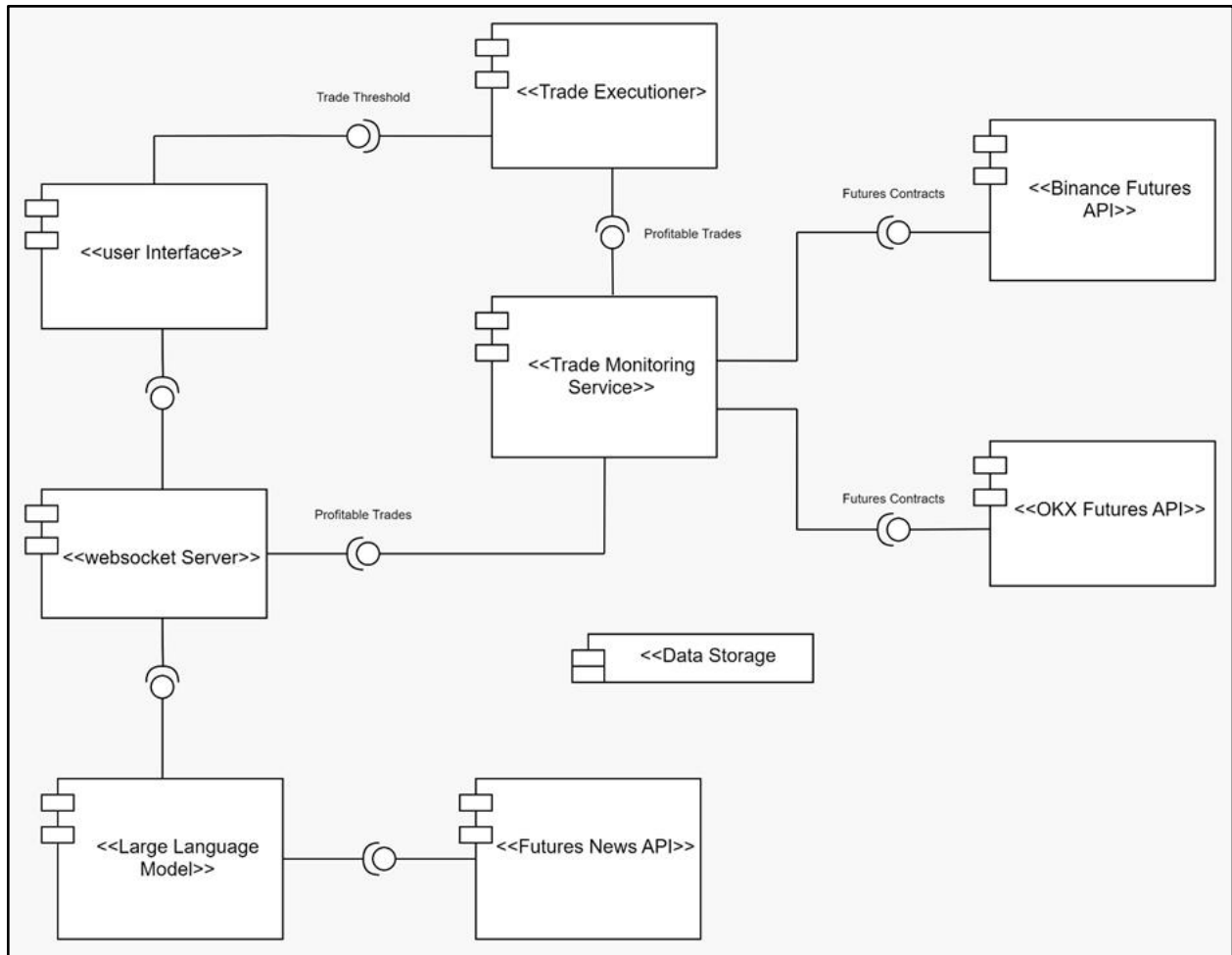


Figure IV-3: **Software Component Diagram**

Core Components:

- **Trade Executioner:** This component is responsible for initiating and managing trades based on user-defined parameters and identified arbitrage opportunities.
- **Trade Monitoring Service:** This component oversees continuously monitoring trades and managing them based on pre-set rules.
- **Large Language Model:** This component likely refers to a machine learning model that's used to analyze data and identify profitable arbitrage opportunities.
- **WebSocket Server:** This component is used to facilitate real-time communication between AutoArbitrage and cryptocurrency exchanges via web sockets.
- **Data Storage:** This component refers to the system's storage for user data, historical data, and model data used by the large language model.

Database Layer:

The data storage component is the database layer in our application as it provides efficient data access and easy retrieval.

APIs & External Connection:

Binance Futures API: This connection allows AutoArbitrage to access real-time price data and execute trades on the Binance exchange.

Bybit Futures API: This connection allows AutoArbitrage to access real-time price data and execute trades on the bybit exchange.

Futures News API: This connection allows AutoArbitrage to access external news articles or data feeds related to futures contracts.

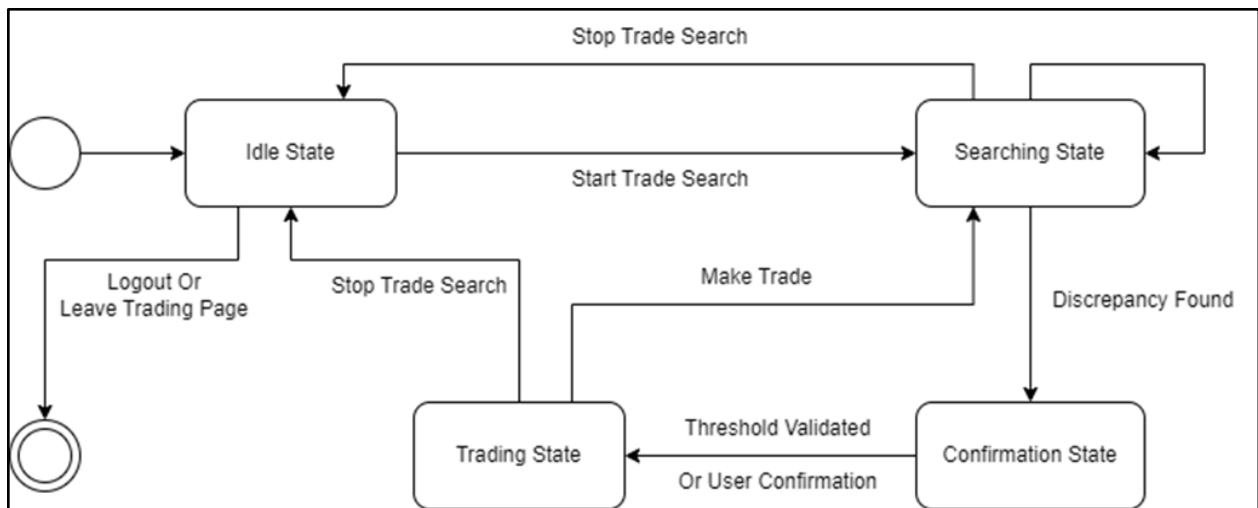


Figure IV-4: **State Diagram**

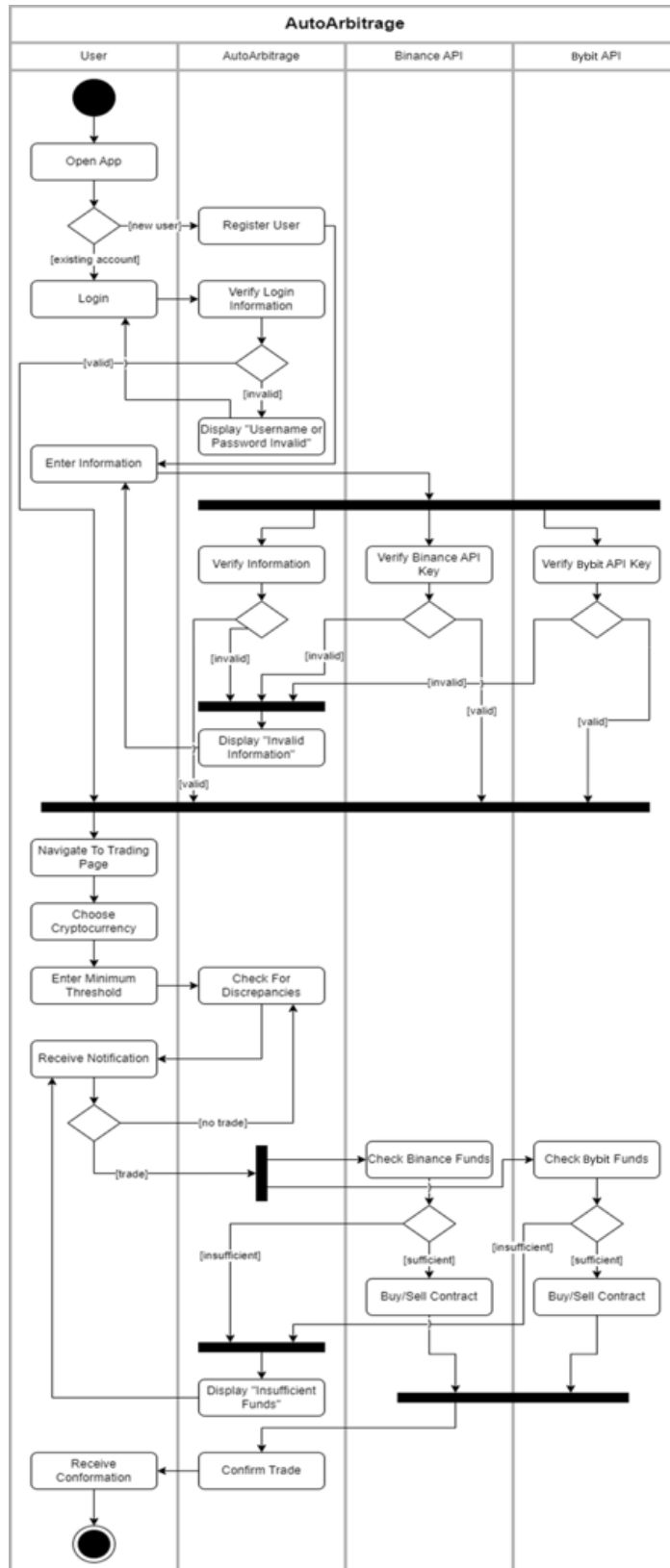


Figure IV-5: Activity Diagram

1.6 ER Diagram

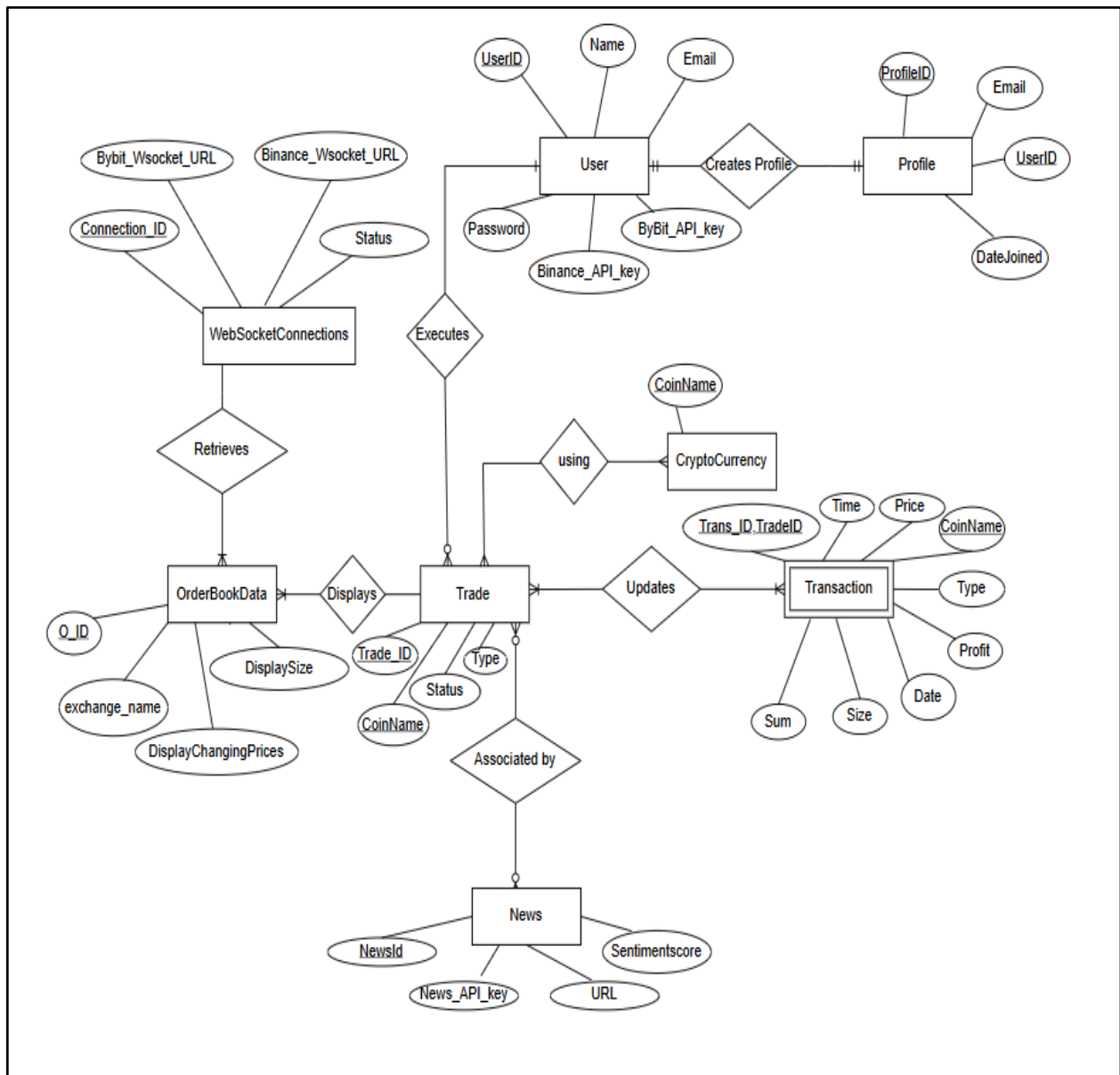


Figure IV-6: ER Diagram

1.7 Class Diagram

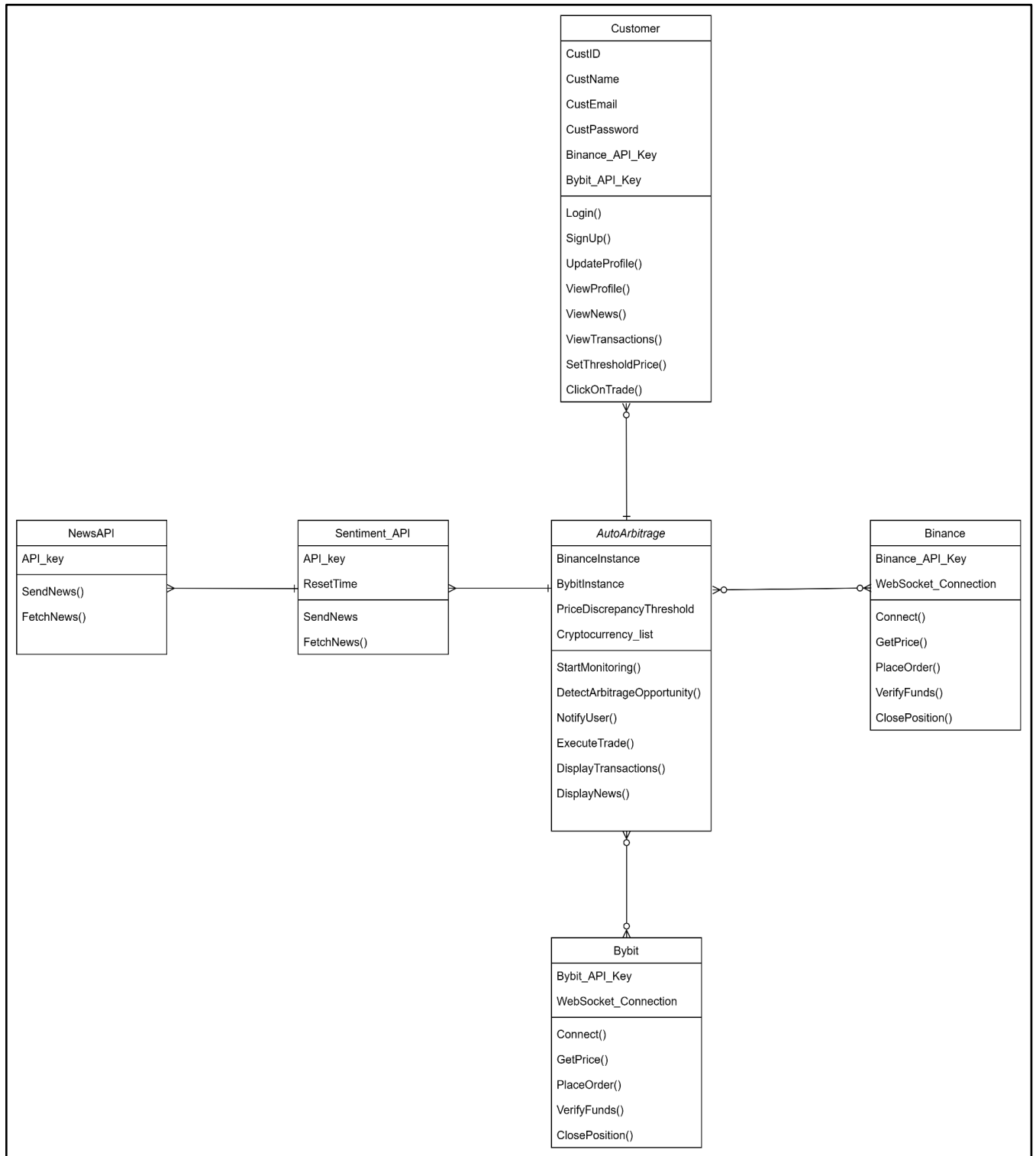
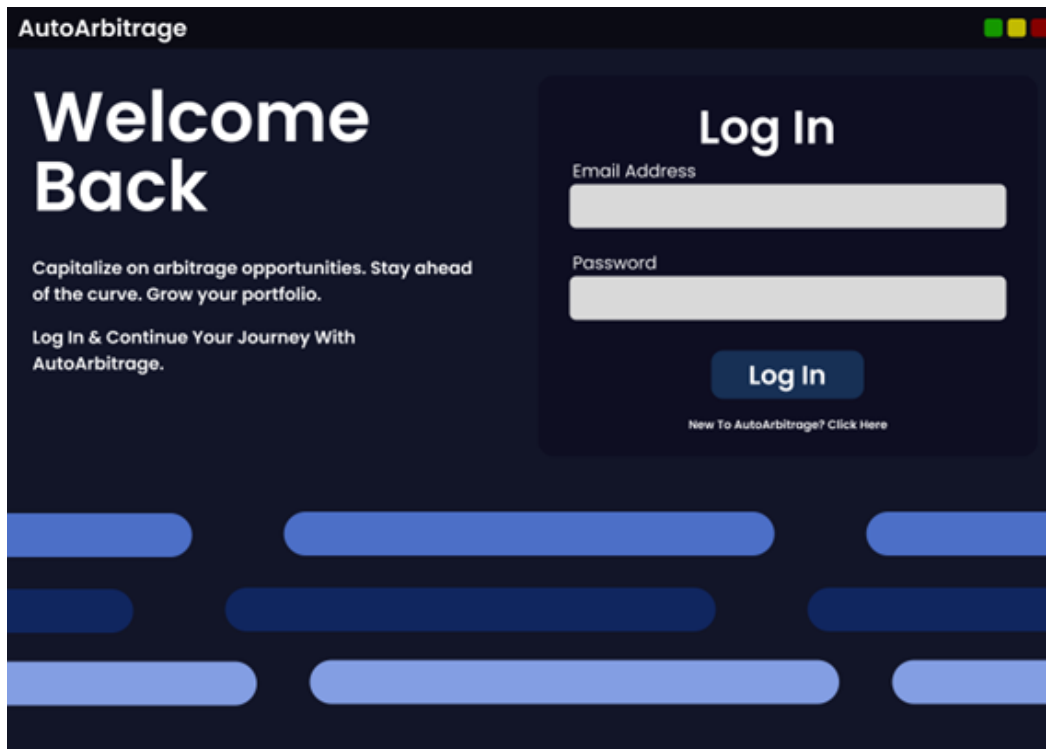


Figure IV-7: **Class Diagram**

1.8 User Interfaces



The image shows a login interface for AutoArbitrage. The header includes the brand name 'AutoArbitrage' and a standard window control bar with green, yellow, and red buttons. The main content area is split into two sections. On the left, a 'Welcome Back' message is displayed in large white text, followed by a motivational quote: 'Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio.' Below this, a smaller line of text reads 'Log In & Continue Your Journey With AutoArbitrage.' On the right, a 'Log In' form is presented. It contains two input fields: 'Email Address' and 'Password'. A blue 'Log In' button is positioned below the password field. At the bottom of the form, a link states 'New To AutoArbitrage? Click Here'. The background features a dark blue gradient with several horizontal bars in varying shades of blue.

AutoArbitrage

Welcome Back

Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio.

Log In & Continue Your Journey With AutoArbitrage.

Log In

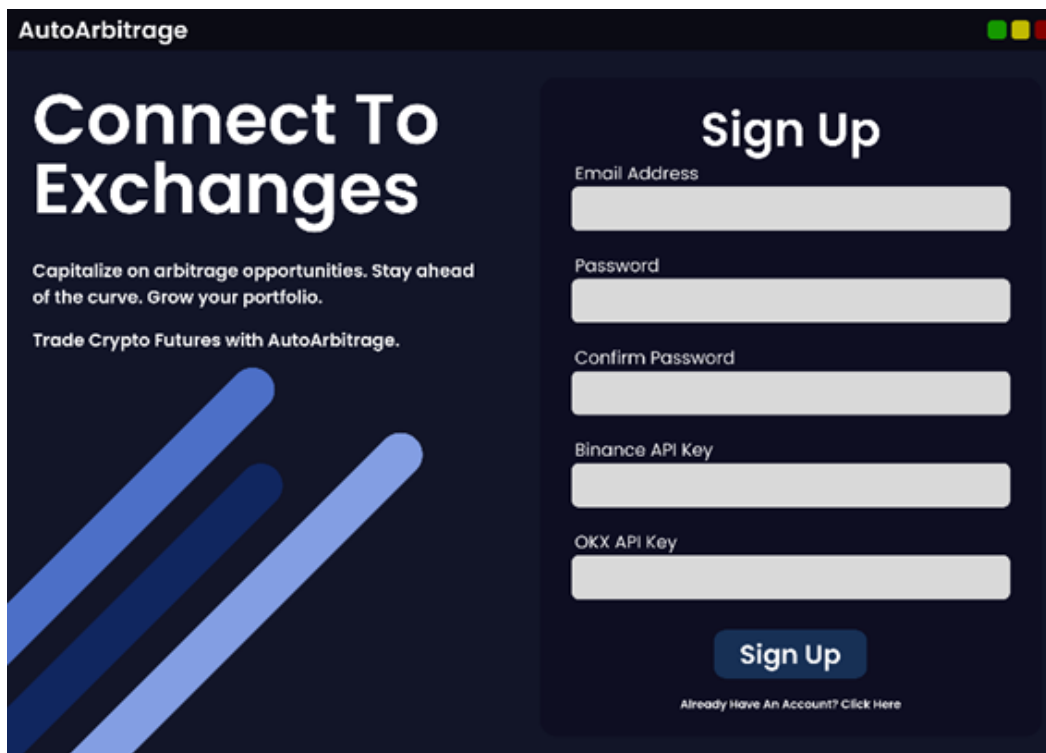
Email Address

Password

Log In

New To AutoArbitrage? Click Here

Figure IV-8: Login



The image shows a sign-up interface for AutoArbitrage. The header includes the brand name 'AutoArbitrage' and a standard window control bar with green, yellow, and red buttons. The main content area is split into two sections. On the left, a 'Connect To Exchanges' message is displayed in large white text, followed by the same motivational quote as the login page: 'Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio.' Below this, a smaller line of text reads 'Trade Crypto Futures with AutoArbitrage.' On the right, a 'Sign Up' form is presented. It contains five input fields: 'Email Address', 'Password', 'Confirm Password', 'Binance API Key', and 'OKX API Key'. A blue 'Sign Up' button is positioned below the 'OKX API Key' field. At the bottom of the form, a link states 'Already Have An Account? Click Here'. The background features a dark blue gradient with several diagonal bars in varying shades of blue.

AutoArbitrage

Connect To Exchanges

Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio.

Trade Crypto Futures with AutoArbitrage.

Sign Up

Email Address

Password

Confirm Password

Binance API Key

OKX API Key

Sign Up

Already Have An Account? Click Here

Figure IV-9: Old Sign Up

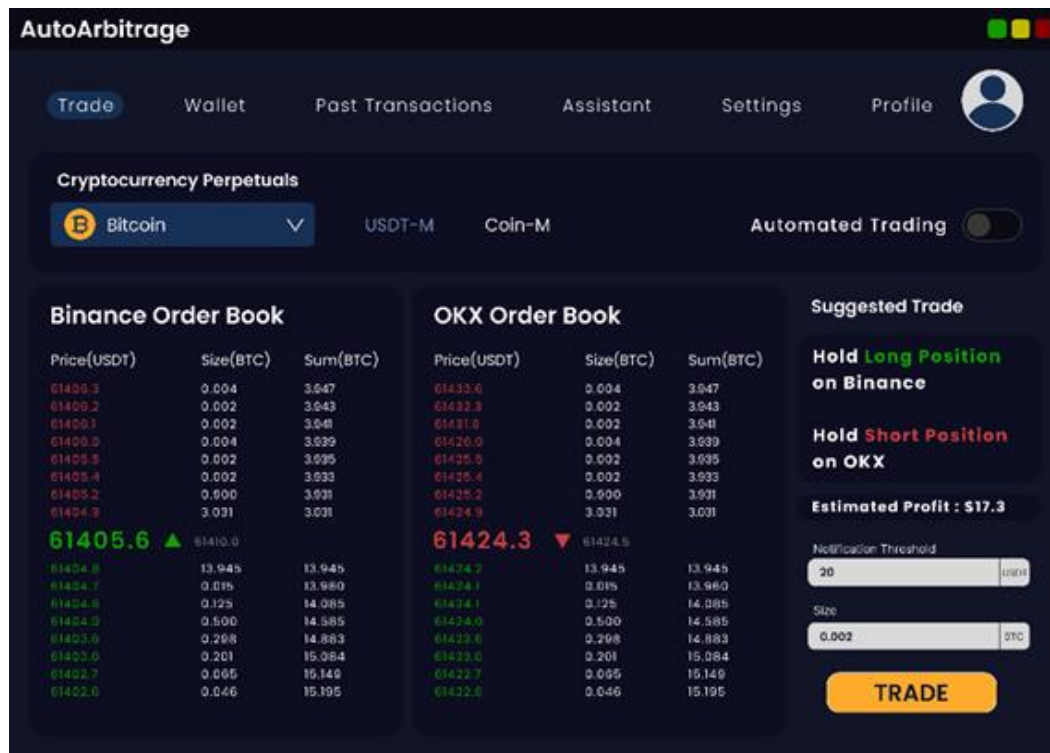


Figure IV-10: Old Trading Page

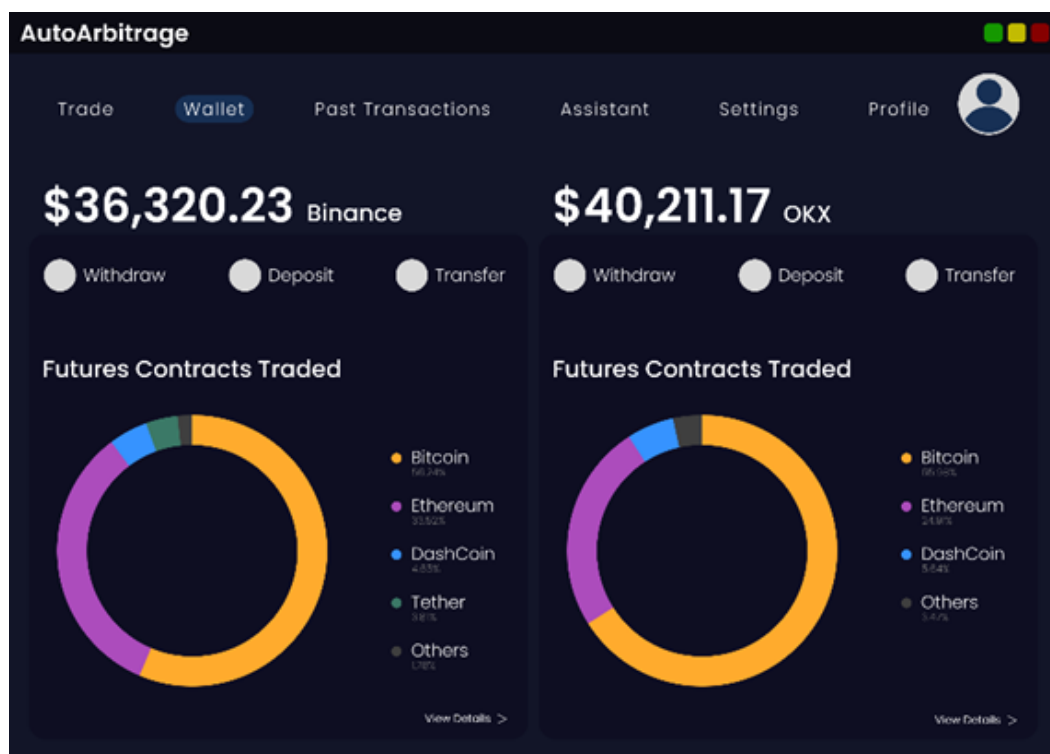


Figure IV-11: Wallet

AutoArbitrage

TradeWalletPast TransactionsAssistantSettingsProfile

Past Transactions

Price(USDT)	Exchange	Coin	Action	Profit	Size	Sum	Date	Time
61384.3	Binance	BTC	Long	\$20.2	0.004	3.947	25/06	08:43
61406.2	OKX	BTC	Short		0.002	3.943	25/06	08:43
36406.1	Binance	ETH	Long	\$18.4	0.002	3.941	24/06	15:56
36426.0	OKX	ETH	Short		0.004	3.939	24/06	15:56
61405.5	OKX	BTC	Long		0.002	3.935	24/06	11:09
61435.4	Binance	BTC	Short	\$27.9	0.002	3.933	24/06	11:10
61405.2	Binance	BTC	Long		0.900	3.931	21/06	13:01
61444.9	OKX	BTC	Short	\$43.4	3.031	3.031	21/06	13:01

Figure IV-12: Past Transactions

AutoArbitrage

TradeWalletPast TransactionsAssistantSettingsProfile

Updates

BitMEX Launches XBTQ24 Bitcoin Futures Contract with 100x Leverage

BitMEX has officially launched its latest monthly Bitcoin (BTC) futures contract, XBTQ24, which began trading at 04:00 UTC on June 25, 2024. The new contract offers up to 100x leverage, according to BitMEX Blog.

June 26, 2024
04:03 AM

[View More >](#)

Tips

We Suggest You Don't Trade With High Leverage

High Leverage might sound attractive and it does have very high rewards but it also comes with massive risks attached with it. You might wipe out your entire portfolio.

June 26, 2024
04:05 AM

[View More >](#)

Figure IV-13: Assistant

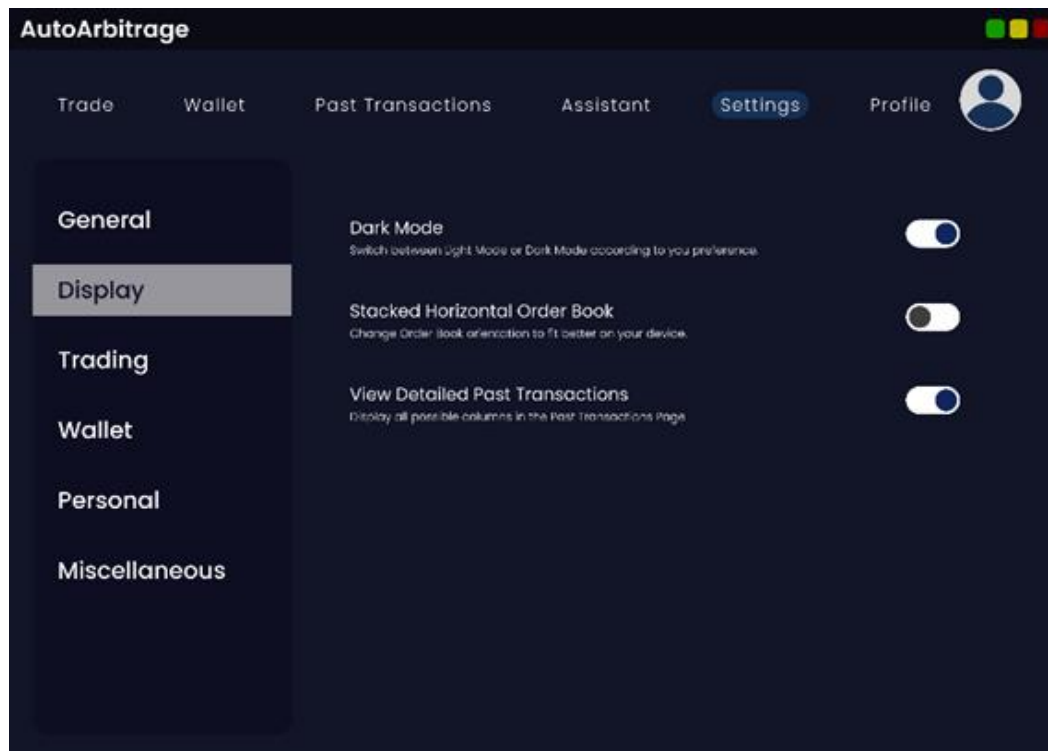


Figure IV-14: **Settings**

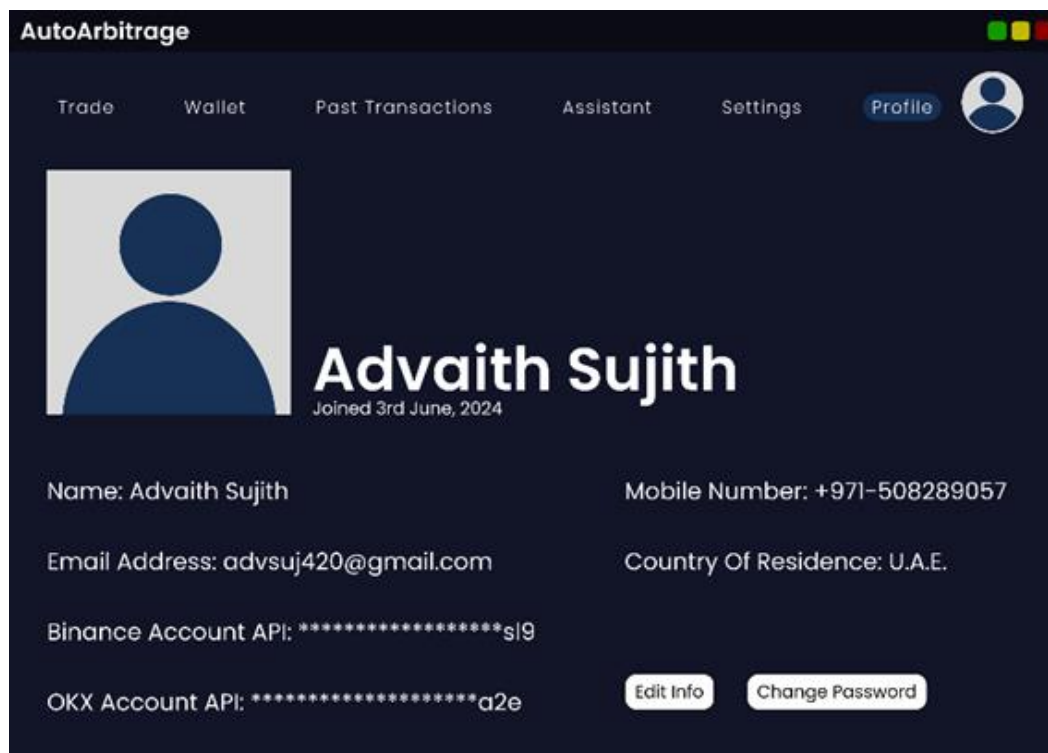
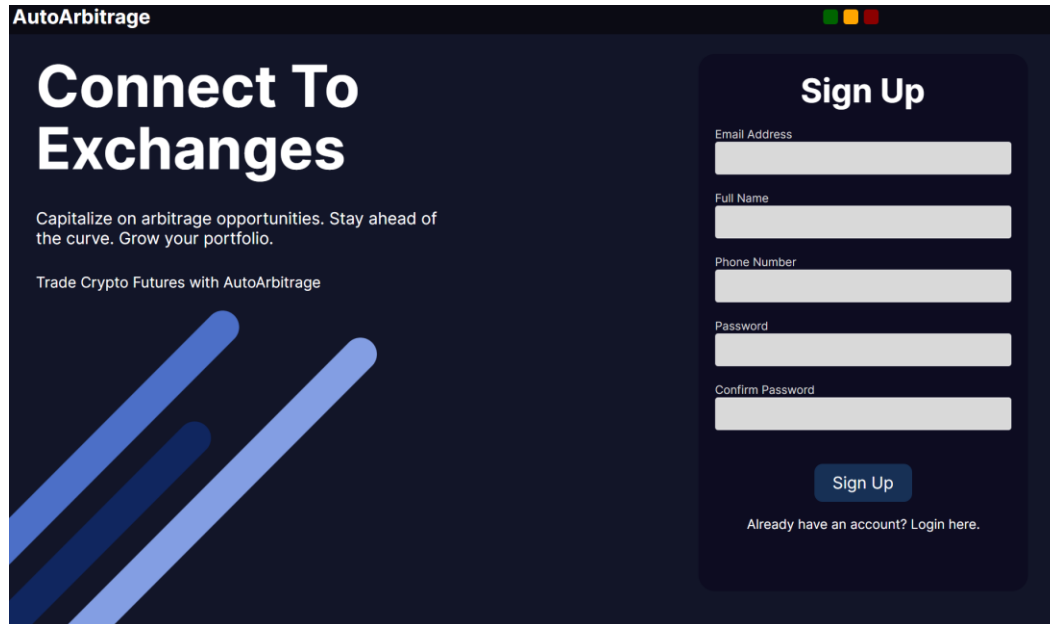


Figure IV-15: **Old Profile**

Over the development time we made changes in the Application's User Interface for better quality and efficiency.

Changes: -



The image shows the 'Sign Up' form of the AutoArbitrage application. The form is located on the right side of a dark-themed interface. It includes input fields for Email Address, Full Name, Phone Number, Password, and Confirm Password. A 'Sign Up' button is positioned below the input fields. At the bottom of the form, there is a link that says 'Already have an account? Login here.' The background of the interface features the text 'Connect To Exchanges' and 'Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio.' along with the tagline 'Trade Crypto Futures with AutoArbitrage'.

AutoArbitrage

Connect To Exchanges

Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio.

Trade Crypto Futures with AutoArbitrage

Sign Up

Email Address

Full Name

Phone Number

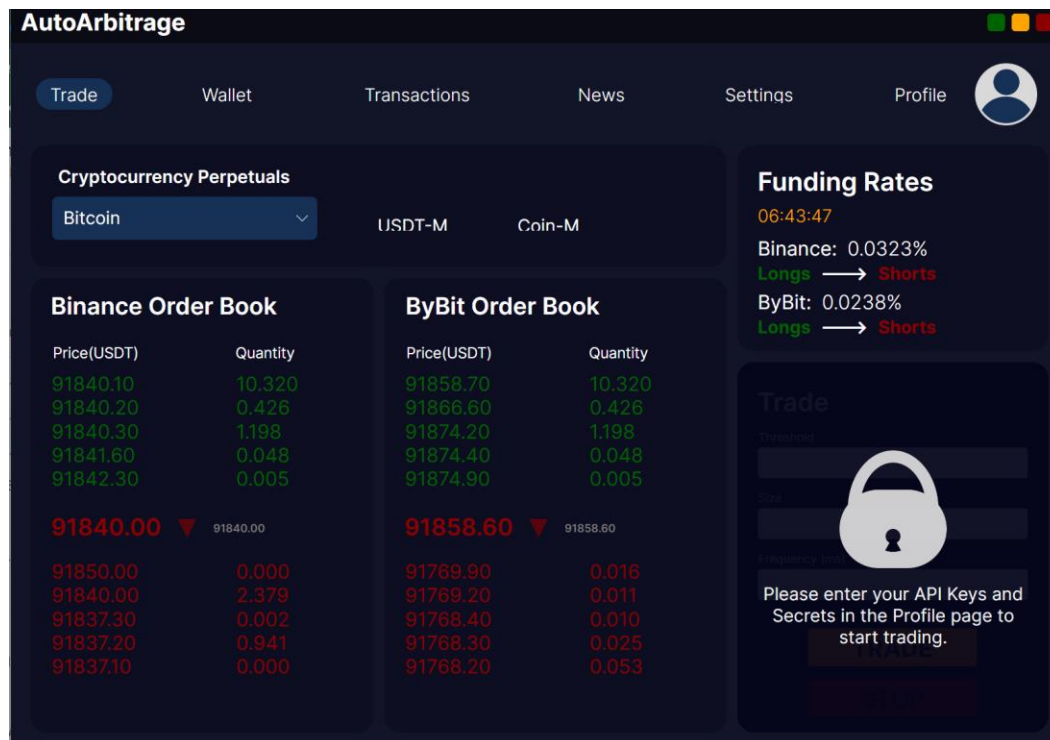
Password

Confirm Password

Sign Up

Already have an account? Login here.

Figure IV-16: New Sign Up



The image shows the 'Bitcoin Trading' interface of the AutoArbitrage application. The interface is dark-themed and features a navigation bar at the top with links for Trade, Wallet, Transactions, News, Settings, and Profile. The main content area is divided into several sections. On the left, there is a 'Cryptocurrency Perpetuals' section with a dropdown menu set to 'Bitcoin'. Below this, there are two order books: 'Binance Order Book' and 'ByBit Order Book'. Each order book displays a list of prices and quantities. On the right, there is a 'Funding Rates' section showing the current time (06:43:47) and the funding rates for Binance (0.0323%) and ByBit (0.0238%). Below the funding rates, there is a 'Trade' section with a large padlock icon and a message that says 'Please enter your API Keys and Secrets in the Profile page to start trading.'

AutoArbitrage

Trade Wallet Transactions News Settings Profile

Cryptocurrency Perpetuals

Bitcoin

USDT-M Coin-M

Binance Order Book

Price(USDT)	Quantity
91840.10	10.320
91840.20	0.426
91840.30	1.198
91841.60	0.048
91842.30	0.005
91840.00	91840.00
91850.00	0.000
91840.00	2.379
91837.30	0.002
91837.20	0.941
91837.10	0.000

ByBit Order Book

Price(USDT)	Quantity
91858.70	10.320
91866.60	0.426
91874.20	1.198
91874.40	0.048
91874.90	0.005
91858.60	91858.60
91769.90	0.016
91769.20	0.011
91768.40	0.010
91768.30	0.025
91768.20	0.053

Funding Rates

06:43:47

Binance: 0.0323%

Longs → Shorts

ByBit: 0.0238%

Longs → Shorts

Trade

Please enter your API Keys and Secrets in the Profile page to start trading.

Figure IV-17: New Bitcoin Trading

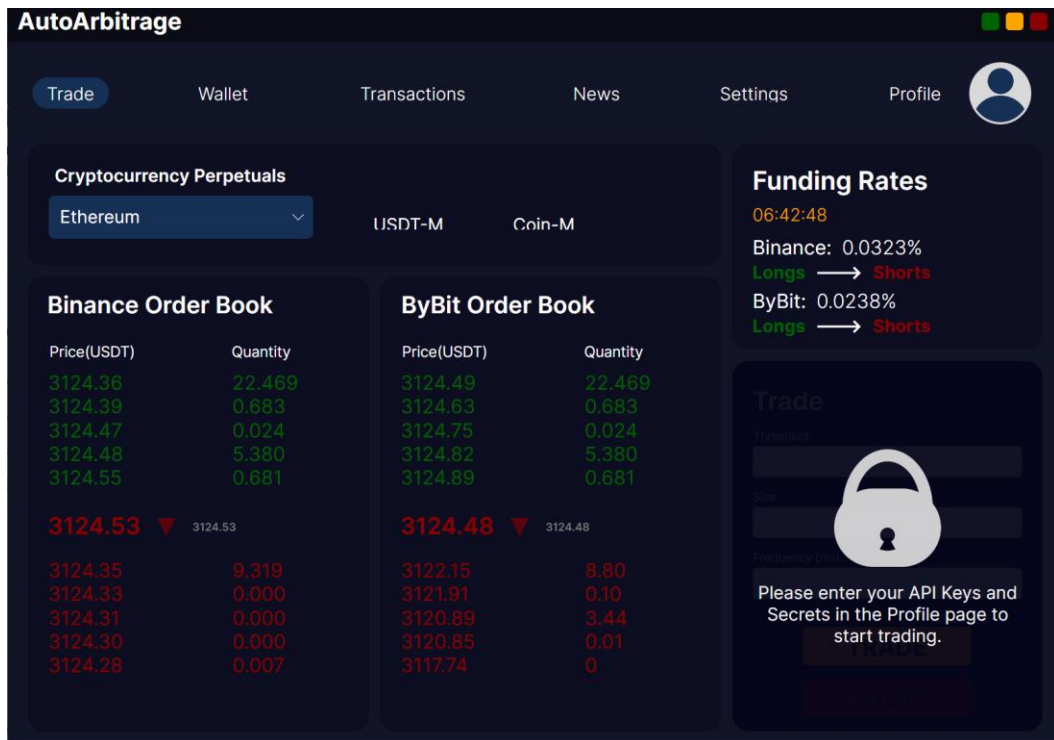
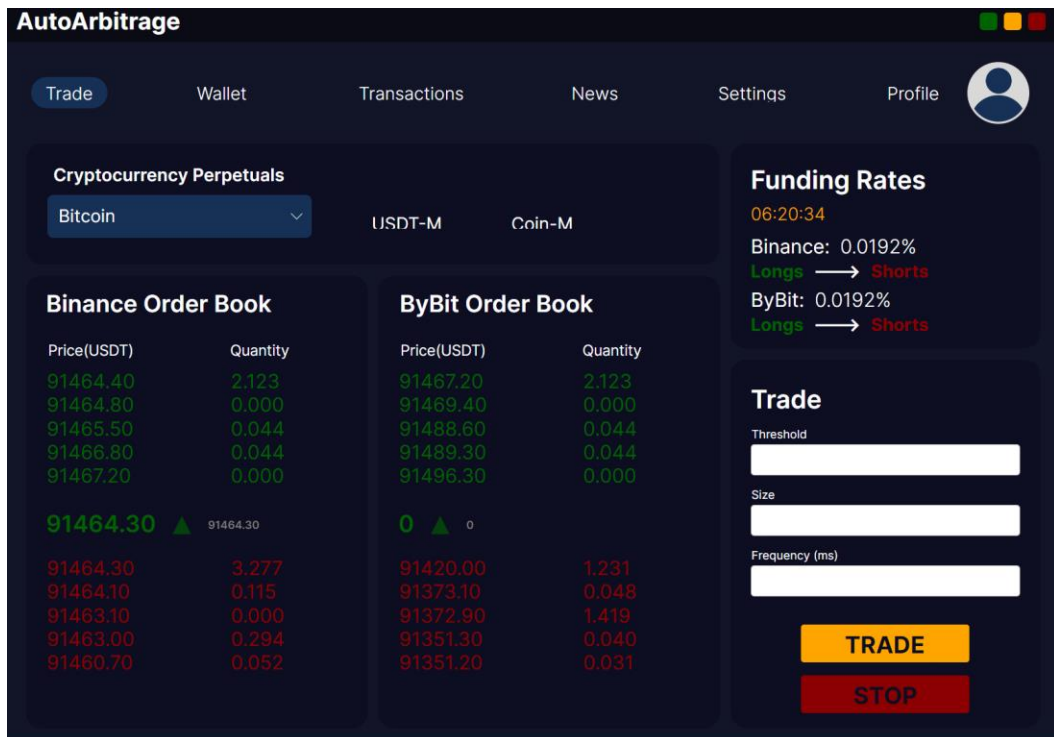


Figure IV-18: *New Trading Ethereum*



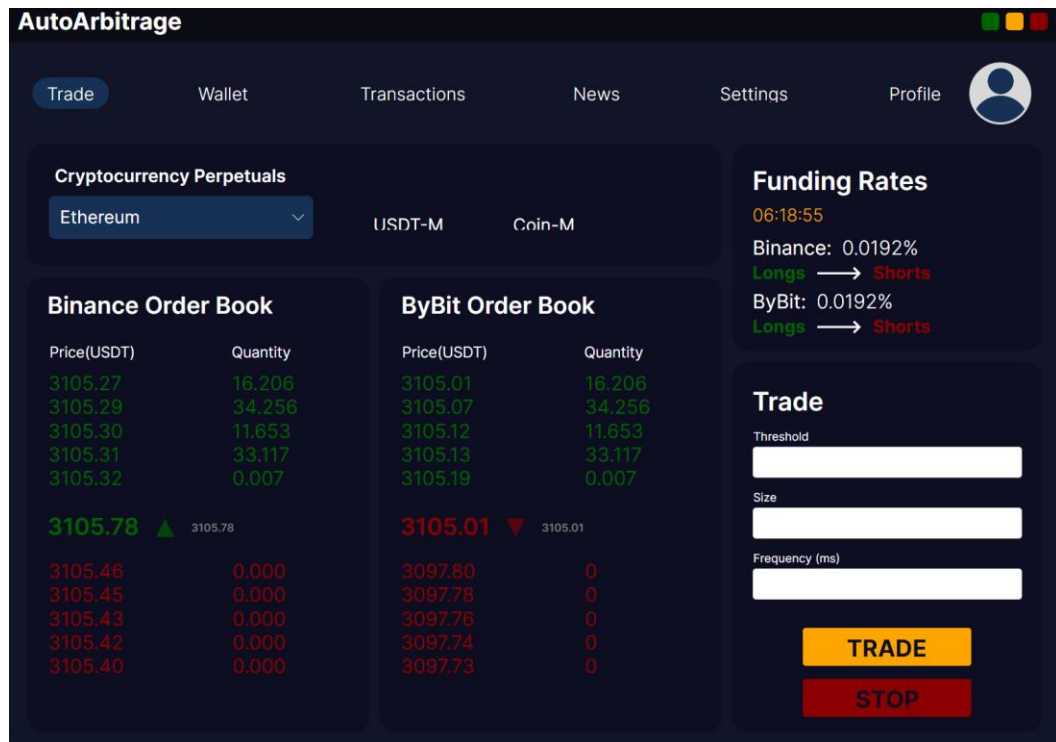


Figure IV-19: **New Profile**

AutoArbitrage

Trade

Wallet

Transactions

News

Settings

Profile

Fahad Nawaz

Joined 19th November, 2024

Email Address:

fahad@gmail.com

Name:

Fahad Nawaz

Phone Number:

0534226765

Binance Secret:

1

Binance API Key:

1

Bybit Secret:

1

Bybit API Key:

1

Save Info

AutoArbitrage

Trade

Wallet

Transactions

News

Settings

Profile

Fahad Nawaz

Joined 19th November, 2024

Email Address:

fahad@gmail.com

Phone Number:

0534226765

Binance API Key:

1

Bybit API Key:

1

Edit Info

2. Low-Level Design

2.1 Outline

This section focuses on the low-level design of the AutoArbitrage document which includes wireframes. Wireframes provide a visual representation of the user interface and experience. These wireframes showcase the application's layout and functionality. It will include our key screens such as login page, sign up page, trading page, news page, settings page, wallet page and profile page. These visual aids give a clear understanding of how users will interact with the application. The wireframes will serve as a blueprint for the development of the user interface, ensuring a user-friendly intuitive experience.

2.2 Wireframes



Figure IV-20: **Login Wireframe**

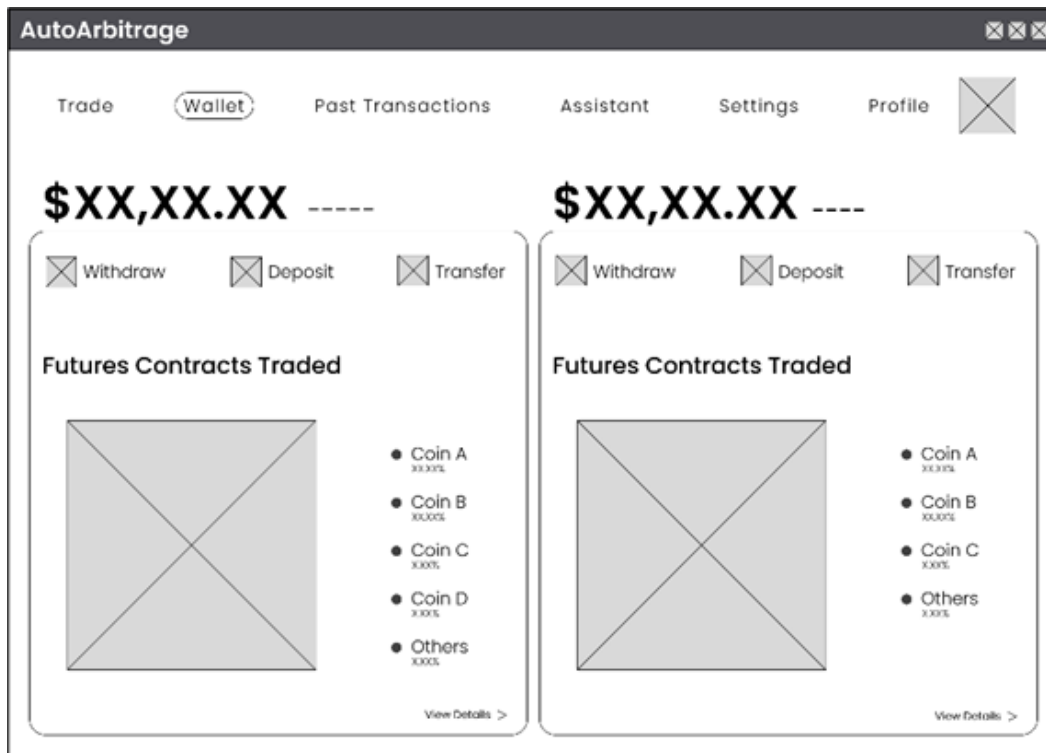


Figure IV-23: **Wallet Wireframe**

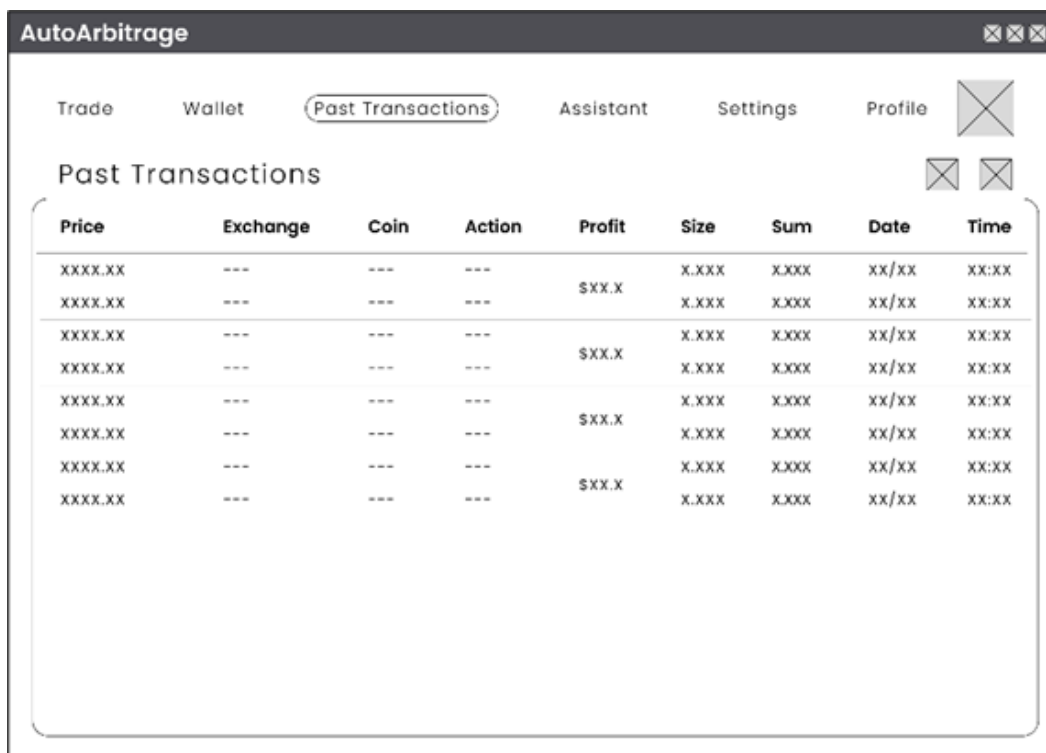


Figure IV-24: **Transactions Wireframe**

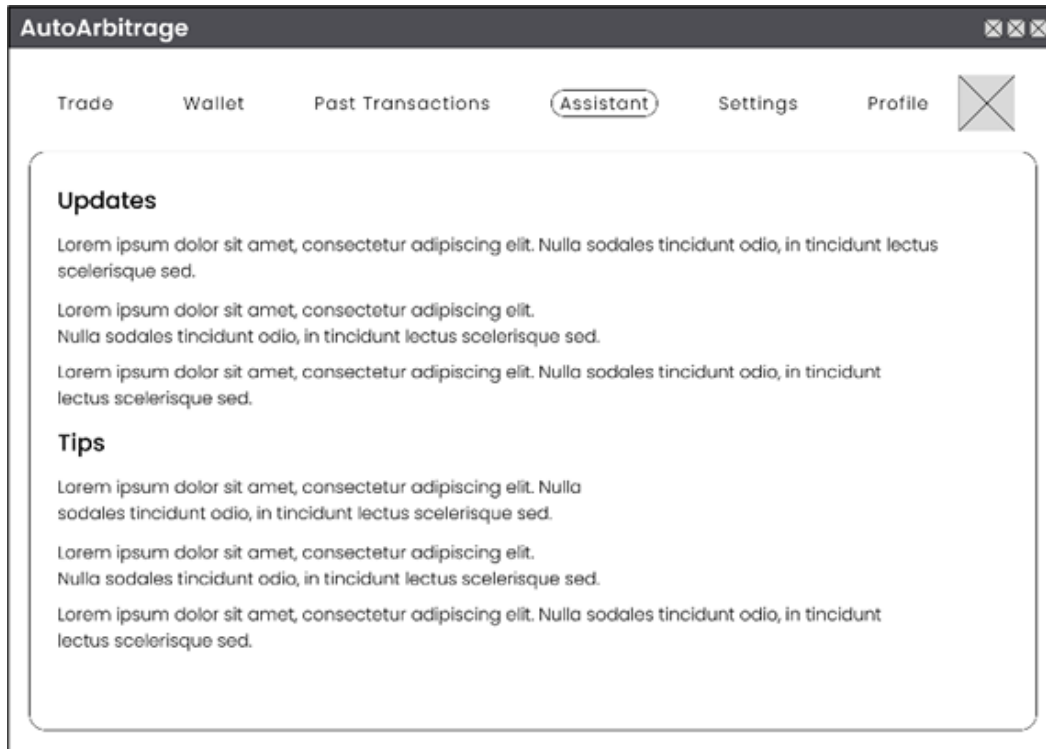


Figure IV-24: **News Wireframe**

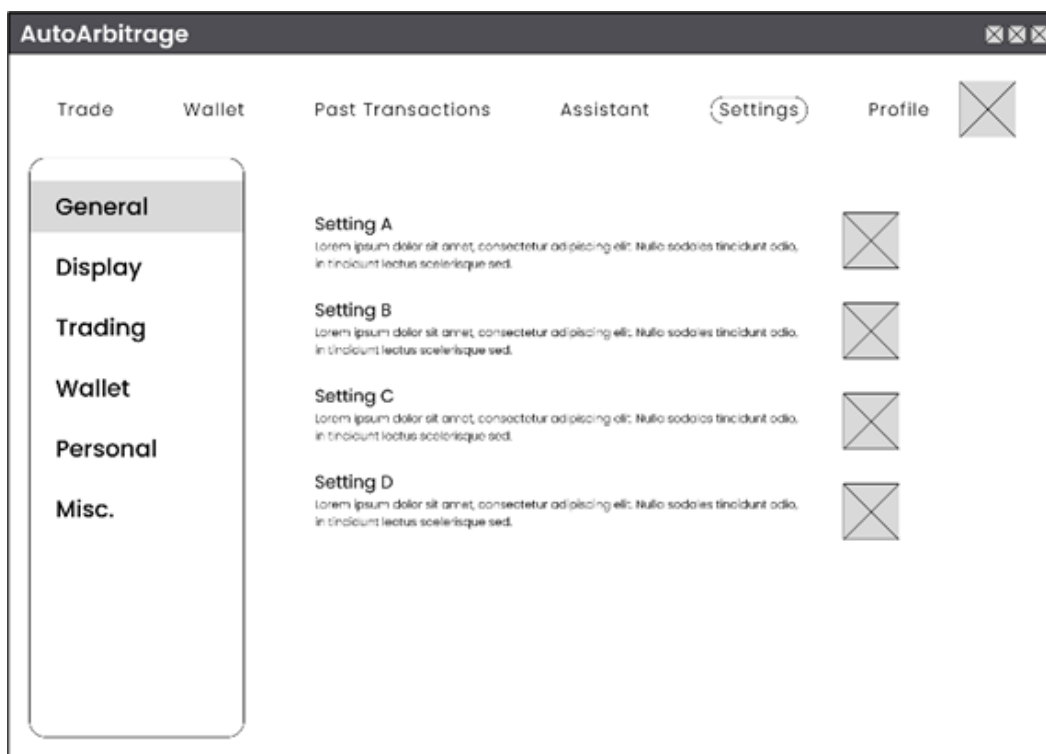


Figure IV-25: **Settings Wireframe**

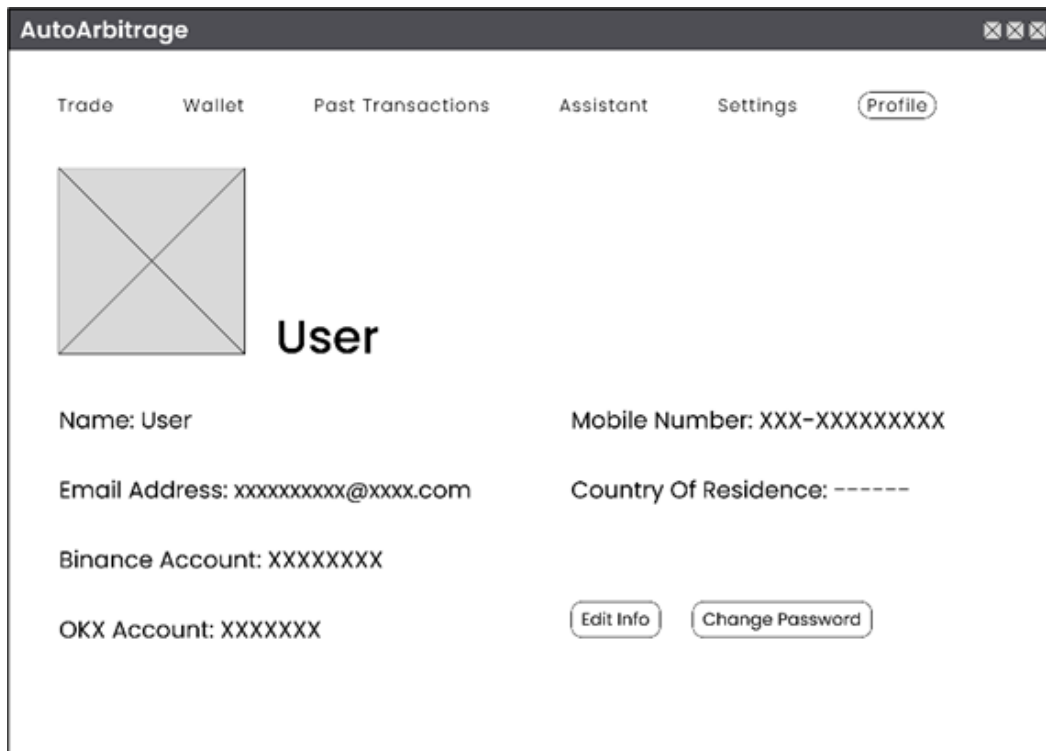


Figure IV-26: **Profile Wireframe**

V. Security

1. Security in Cryptocurrency systems

The importance of security in cryptocurrency trading systems cannot be overstated due to the potential financial and reputational risks involved. Cryptocurrency systems, which rely heavily on decentralized and digital platforms, are inherently vulnerable to cyber-attacks that can result in substantial financial losses. For instance, breaches in such systems can lead to the direct loss of user funds, as seen in notable incidents where attackers exploited system vulnerabilities, causing millions of dollars in losses (Quamara & Singh, 2022).

The reputational damage following a security breach is also profound, as user trust is essential for platforms dealing with high-value digital assets. A compromised trading system can lead users to perceive the platform as unreliable, prompting them to withdraw their assets and seek alternatives. Once a security breach occurs, regaining user confidence is challenging, which could undermine a platform's long-term sustainability (Kizza, 2024).

In addition to financial and reputational impacts, security breaches may expose the trading system to legal liabilities. With stringent regulations like GDPR and AML laws governing data protection and anti-money laundering practices, any breach that compromises personal or financial data can result in regulatory penalties. These legal consequences highlight the importance of maintaining rigorous security protocols to ensure compliance and protect against liabilities.

Thus, protecting a cryptocurrency trading system against potential breaches is crucial not only for safeguarding user assets but also for preserving the platform's reputation and ensuring adherence to legal standards.

2. Security risks

API Key Management

API security is essential in preventing numerous vulnerabilities and threats that arise in its lifecycle. Issues like outdated or misconfigured APIs, unauthorized access, and sensitive data exposure are significant risks. API management involves cataloguing assets, tracking their permissions, and ensuring they conform to security standards. Effective API management is challenging due to the extensive and often hidden nature of APIs, as well as the rapid pace of development in digital transformations, where outdated APIs, unauthorized access, and version control add layers of risk (Sun, Wang & Guo, 2022).

Key management techniques are critical to API security. Secure handling of API keys includes using unique keys per user and storing them securely to prevent unauthorized access. By applying advanced security protocols and controlled access, API security frameworks help manage authentication, authorization, and monitoring tasks, essential in mitigating API-related security challenges and protecting sensitive assets.

Data Breaches

A data breach is a security incident that results in the unauthorized access, disclosure, modification, or destruction of sensitive data. It can occur due to various factors, including:

- Technical vulnerabilities: such as software bugs or weak encryption algorithms
- Human error: such as accidentally sharing passwords or clicking on phishing links
- Malicious attacks: such as hacking, malware, or social engineering

Cryptocurrency systems are particularly vulnerable to data breaches because they often involve the storage and transfer of large amounts of valuable digital assets.

- 51% Attacks: An attacker with control over a majority of the network's computing power can manipulate the blockchain, potentially reversing transactions or double-spending coins.
- Sybil Attacks: An attacker creates multiple fake identities to disrupt network consensus mechanisms, affecting block finalization and validator selection.
- Double-Spending Attacks: Exploiting vulnerabilities to spend the same cryptocurrency units multiple times.
- Smart Contract Vulnerabilities: Bugs or security flaws in smart contracts can be exploited to drain funds or manipulate contract logic.
- Exchange Platform Attacks: Targeting cryptocurrency exchanges to steal funds or user data.
- Wallet Software Attacks: Exploiting vulnerabilities in software wallets to gain access to private keys and steal cryptocurrencies (Vallabhaneni, 2024).

Denial of Service Attack

A Denial of Service (DoS) attack disrupts the normal operation of a computer, service, or network, making it unavailable to its intended users. Instead of attempting to steal information or gain access to sensitive data, DoS attacks aim to overwhelm the target with a flood of traffic, rendering it incapable of responding to legitimate requests.

While DoS attacks can target various systems, they pose a significant threat to cryptocurrency platforms due to their reliance on continuous availability and accessibility. (Kizza, 2024)

- Disruption of Trading and Transactions: Cryptocurrency exchanges and trading platforms rely on uninterrupted service to facilitate trades and transactions. A successful DoS attack can halt trading activities, preventing users from buying, selling, or transferring cryptocurrencies. This can lead to financial losses for traders who may miss out on favourable market movements or be unable to execute time-sensitive transactions.
- Impact on Mining Operations: Cryptocurrency mining involves complex computations to verify and add transactions to the blockchain. DoS attacks can disrupt mining pools, impacting the network's overall processing power and potentially slowing down transaction confirmation times.
- Exploitation by Competing Entities: The sources note that competing mining pools may use DoS attacks to sabotage rivals and gain an unfair advantage in earning mining rewards. By disrupting the operations of other pools, attackers can increase their own chances of successfully mining blocks and collecting associated rewards.
- Smurf Attack: Exploits the trust relationship in IP networks by sending a large number of spoofed ping packets with the victim's IP address as the source. The victim is then flooded with responses from multiple subnets, overwhelming its resources.
- ICMP Protocol Attack: Utilizes the ICMP protocol, commonly used for network diagnostics, to flood the target with ICMP messages, exceeding its processing capacity.
- Ping of Death: Sends oversized or malformed ping packets to the victim, causing it to crash or become unresponsive.
- SYN Flood: Exploits the TCP handshake process by sending a high volume of SYN packets to the target. The victim allocates resources for each connection request but never receives the final ACK packet to complete the handshake, eventually exhausting its resources.
- Land/Latierra Attack: Sends forged SYN packets with identical source and destination IP addresses and ports, creating a loop that can crash the victim (Kizza, 2024).

3. Security in AutoArbitrage

Implementing comprehensive cybersecurity measures in AutoArbitrage is vital due to the platform's handling of sensitive user data, particularly API keys for cryptocurrency exchanges. Robust security practices not only safeguard data but also maintain user trust and compliance with regulatory standards.

Authentication and Authorization

- Secure Login Mechanisms: AutoArbitrage enforces stringent password policies to ensure user accounts are secure from unauthorized access. Users are required to create passwords that are at least eight characters long, incorporating a combination of uppercase and lowercase letters, numbers, and special characters. This policy significantly reduces the risk of brute-force attacks and weak password exploitation. Furthermore, the system employs additional login protections, such as account lockout mechanisms after multiple failed login attempts, to thwart unauthorized access attempts. Future implementations may include Two-Factor Authentication (2FA) for an added layer of security, requiring users to verify their identities using a secondary device or code.

Data Protection

- Encryption: To protect sensitive information, AutoArbitrage utilizes advanced encryption techniques. Critical user data, including API keys, credentials, and transaction history, is encrypted both at rest and in transit using industry-standard algorithms such as AES-256 and TLS (Transport Layer Security). Encryption ensures that, even if intercepted or accessed by malicious entities, this data remains unreadable and secure. By implementing encryption at multiple levels, AutoArbitrage ensures the confidentiality and safety of user information.
- Data Integrity: To prevent unauthorized modifications to stored data, AutoArbitrage employs hashing techniques. Hashing creates a unique, fixed-size representation of data, often referred to as a "digital fingerprint," which can be used to detect tampering. If data is altered, its hash value will no longer match the stored hash, signaling a potential breach. This mechanism ensures that stored data remains consistent, accurate, and tamper-proof.
- Secure Data Storage: Sensitive user data, including API keys, is stored in a secure database configured with strict access controls. Access is limited to authorized users and processes based on the principle of least privilege, minimizing the exposure of sensitive information. The database is further fortified with encryption and regular security updates, ensuring it remains resilient against emerging threats. Backup systems are also employed to provide data redundancy and recovery capabilities in case of a breach or system failure.
- Protection Against SQL Injection: As part of its security protocols, AutoArbitrage has undergone rigorous testing to ensure resilience against SQL injection attacks. SQL injection is a technique used by attackers to exploit vulnerabilities in a system's database query mechanisms, often gaining unauthorized access to sensitive data. By implementing input validation, parameterized queries, and prepared statements, the platform effectively neutralizes such threats. These safeguards ensure the system can only execute legitimate database queries, thereby protecting sensitive user information from exploitation.

- **AWS Security Protocol Integration:** Since AutoArbitrage is deployed within the Amazon Web Services (AWS) ecosystem, it benefits from AWS's robust built-in security features. These include network security measures, such as firewalls and virtual private networks (VPNs), to restrict unauthorized access. Data stored in AWS is protected with server-side encryption, and the environment supports secure key management through AWS Key Management Service. AWS Identity and Access Management enables fine-grained access control, ensuring that only authenticated and authorized users can access specific resources. Additionally, AWS routinely undergoes security audits and compliance checks, ensuring that its infrastructure meets stringent security and regulatory standards.

VI. Implementation

This section includes the code of the main Logic of AutoArbitrage

1. Sign up ViewModel

```
using System;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;
using AutoArbitrage_MVVM.Services;
using AutoArbitrage_MVVM.Views;
using Avalonia.Controls;
using Avalonia.Threading;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using MySql.Data.MySqlClient;
using BCrypt.Net;

namespace AutoArbitrage_MVVM.ViewModels
{
    5 references
    public partial class SignUpViewModel : ObservableObject
    {
        private string connectionString = "Server=autoarbitrage.cri2yu84sa9j.me-central-1.rds.amazonaws.com;Database=userdb;User ID=admin;Password=autoarbitrage12;";

        [ObservableProperty] private string email;
        [ObservableProperty] private string password;
        [ObservableProperty] private string confirmPassword;
        [ObservableProperty] private string fullName;
        [ObservableProperty] private string phoneNumber;
        [ObservableProperty] private string errorMessage;

        private Window currentWindow;

        1 reference
        public SignUpViewModel(Window window)
        {
            currentWindow = window;
        }
    }
}
```

```
[RelayCommand]
2 references
public async Task SignUp()
{
    if (string.IsNullOrEmpty(Email) || string.IsNullOrEmpty>Password) ||
        string.IsNullOrEmpty(ConfirmPassword) ||
        string.IsNullOrEmpty(FullName) || string.IsNullOrEmpty(PhoneNumber))
    {
        ErrorMessage = "Please fill in all fields.";
        return;
    }

    if (Password != ConfirmPassword)
    {
        ErrorMessage = "The passwords do not match. Please try again.";
        return;
    }
}
```

```

if (Password.Length < 8)
{
    ErrorMessage = "Password must be at least 8 characters long.";
    return;
}

try
{
    using var connection = new MySqlConnection(connectionString);
    await connection.OpenAsync();

    // Check if the email already exists
    string checkUserQuery = "SELECT COUNT(*) FROM users WHERE email = @Email";
    using (var checkCmd = new MySqlCommand(checkUserQuery, connection))
    {
        checkCmd.Parameters.AddWithValue("@Email", Email);
        var count = (long)await checkCmd.ExecuteScalarAsync();
        if (count > 0)
        {
            ErrorMessage = "Email already exists. Please use a different email.";
            return;
        }
    }
}

```

```

// Generate a salt and hash the password
var salt = GenerateSalt();
var hashedPassword = HashPassword(Password, salt);

// Insert the new user with the hashed password and salt
string insertUserQuery = @"INSERT INTO users (email, password_hash, salt, fullname, phone)
VALUES (@Email, @PasswordHash, @Salt, @FullName, @PhoneNumber)";
using (var insertCmd = new MySqlCommand(insertUserQuery, connection))
{
    insertCmd.Parameters.AddWithValue("@Email", Email);
    insertCmd.Parameters.AddWithValue("@PasswordHash", hashedPassword);
    insertCmd.Parameters.AddWithValue("@Salt", salt);
    insertCmd.Parameters.AddWithValue("@FullName", FullName);
    insertCmd.Parameters.AddWithValue("@PhoneNumber", PhoneNumber);
    await insertCmd.ExecuteNonQueryAsync();
}

// Clear error message and open main window after successful registration
ErrorMessage = string.Empty;
var mainWindow = new MainWindow();
mainWindow.Show();
Dispatcher.UIThread.InvokeAsync(() => currentWindow.Close());
}
catch (Exception ex)
{
    ErrorMessage = $"Error: {ex.Message}";
}

UserService.Instance.Email = this.Email;
}

```

```

// Generate a unique salt for each user
1 reference
private static string GenerateSalt()
{
    var rng = new RNGCryptoServiceProvider();
    byte[] saltBytes = new byte[32];
    rng.GetBytes(saltBytes);
    return Convert.ToBase64String(saltBytes);
}

// Hash the password using SHA256 with the provided salt
1 reference
private static string HashPassword(string password, string salt)
{
    using var sha256 = SHA256.Create();
    var saltedPassword = salt + password;
    var hashBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(saltedPassword));
    return BitConverter.ToString(hashBytes).Replace("-", "").ToLower();
}

[RelayCommand]
2 references
public void ToLogin()
{
    var loginWindow = new Login();
    loginWindow.Show();

    Dispatcher.UIThread.InvokeAsync(() => currentWindow.Close());
}
}

```

2. Login ViewModel

```

using System;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;
using AutoArbitrage_MVVM.Services;
using AutoArbitrage_MVVM.Views;
using Avalonia.Controls;
using Avalonia.Threading;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using MySql.Data.MySqlClient;

namespace AutoArbitrage_MVVM.ViewModels;

5 references
public partial class LoginViewModel : ObservableObject
{
    private string connectionString = "Server=autoarbitrage.cri2yu04sa9j.me-central-1.rds.amazonaws.com;Database=userdb;User ID=admin;Password=autoarbitrage12;";

    [ObservableProperty]
    private string email;

    [ObservableProperty]
    private string password;

    [ObservableProperty]
    private string errorMessage;

    private Window currentWindow;

    2 references
    public LoginViewModel(Window window)
    {
        currentWindow = window;
    }

    [RelayCommand]
    2 references

```

2 references

```
public async Task Login()
{
    if (string.IsNullOrEmpty(Email) || string.IsNullOrEmpty>Password))
    {
        ErrorMessage = "Please fill in all fields.";
        return;
    }

    try
    {
        using var connection = new MySqlConnection(connectionString);
        await connection.OpenAsync();

        // Retrieve the salt and password hash from the database
        string query = "SELECT password_hash, salt FROM users WHERE email = @Email";
        using var cmd = new MySqlCommand(query, connection);
        cmd.Parameters.AddWithValue("@Email", Email);

        using var reader = await cmd.ExecuteReaderAsync();
        if (!reader.HasRows)
        {
            ErrorMessage = "Invalid email or password.";
            return;
        }

        await reader.ReadAsync();
        var storedHash = reader.GetString(reader.GetOrdinal("password_hash"));
        var storedSalt = reader.GetString(reader.GetOrdinal("salt"));

        // Hash the entered password with the stored salt
        var enteredHash = HashPassword>Password, storedSalt);

        if (enteredHash != storedHash)
        {
            ErrorMessage = "Invalid email or password.";
            return;
        }
    }
}
```

```

        // Successful login
        ErrorMessage = string.Empty;
        var mainWindow = new MainWindow();
        mainWindow.Show();
        Dispatcher.UIThread.InvokeAsync(() => currentWindow.Close());
    }
    catch (Exception ex)
    {
        ErrorMessage = $"Error: {ex.Message}";
    }

    UserService.Instance.Email = this.Email;
    Console.WriteLine(UserService.Instance.Email);
}

```

// Hash the password with the salt

1 reference

```

private static string HashPassword(string password, string salt)
{
    using var sha256 = SHA256.Create();
    var saltedPassword = salt + password;
    var hashBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(saltedPassword));
    return BitConverter.ToString(hashBytes).Replace("-", "").ToLower();
}

```

0 references

```

public void ToSignUp()
{
    var signupWindow = new SignUp();
    signupWindow.Show();

    Dispatcher.UIThread.InvokeAsync(() => currentWindow.Close());
}

```

}

3. Trading ViewModel

```
using System.Linq;
using System.Net.Http;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Input;
using AutoArbitrage_MVVM.Services;
using Avalonia.Controls;
using Avalonia.Media;
using Avalonia.Threading;
using Binance.Net;
using Binance.Net.Clients;
using Bybit.Net;
using Bybit.Net.Clients;
using Bybit.Net.Enums;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using CryptoExchange.Net.Authentication;
using CryptoExchange.Net.CommonObjects;
using Newtonsoft.Json.Linq;
using WebSocketSharp;
using MySql.Data.MySqlClient;

namespace AutoArbitrage_MVVM.ViewModels;

7 references
public partial class TradeViewModel : ObservableObject
{
    // ***Non-Observable Properties**
    private WebSocket _binanceFRWebSocket;

    private static readonly HttpClient client = new HttpClient();

    private BinanceSocketClient _binanceSocketClient;
    private BybitSocketClient _bybitSocketClient;

    private decimal _btcPreviousPrice_binance;
    private decimal _ethPreviousPrice_binance;

    private decimal _btcPreviousPrice_bybit;
    private decimal _ethPreviousPrice_bybit;

    private DateTime _nextTargetTime;

    private static readonly TimeSpan[] TargetTimes =
    {
        new TimeSpan(4, 0, 0), // 4:00 AM
        new TimeSpan(12, 0, 0), // 12:00 PM
        new TimeSpan(20, 0, 0) // 8:00 PM
    };

    private static readonly string apiUrl = "https://api.bybit.com/v5/market/tickers";

    private string _selectedCurrency;

    private decimal _btcPrice_binance;
    private decimal _ethPrice_binance;

    private decimal _btcPrice_bybit;
    private decimal _ethPrice_bybit;

    private string connectionString = "Server=autoarbitrage.cri2yu04sa9j.me-central-1.rds.amazonaws.com;Database=userdb;User ID=admin;Password=autoarbitrage12;";

    private string? _email;
    5 references
    public string? Email
    {
        get => _email;
        set => SetProperty(ref _email, value);
    }

    private bool _isRunning = false;

    // ***Observable Properties***

    // Binance
    [ObservableProperty]
    private decimal _binancePrice;
```



```

[ObservableProperty]
private string _binanceAskPrice;

[ObservableProperty]
private string _binanceAskQuantity;

[ObservableProperty]
private string _binanceBidPrice;

[ObservableProperty]
private string _binanceBidQuantity;

[ObservableProperty]
private string _binanceFundingRate;

[ObservableProperty]
private string _bPaymentFrom;

[ObservableProperty]
private string _bPaymentTo;

[ObservableProperty]
private string _bArrow;

[ObservableProperty]
private IBrush _bPaymentFromColor;

[ObservableProperty]
private IBrush _bPaymentToColor;

[ObservableProperty]
private IBrush _binancePriceColor = Brushes.Gray;

[ObservableProperty]
private double _binanceUpArrowOpacity = 0;

[ObservableProperty]
private double _binanceDownArrowOpacity = 0;

[ObservableProperty]
private decimal _binancePreviousPrice;

```

```

// Bybit
[ObservableProperty]
private decimal _bybitPrice;

[ObservableProperty]
private string _bybitAskPrice;

[ObservableProperty]
private string _bybitAskQuantity;

[ObservableProperty]
private string _bybitBidPrice;

[ObservableProperty]
private string _bybitBidQuantity;

[ObservableProperty]
private string _bybitFundingRate;

[ObservableProperty]
private string _byPaymentFrom;

[ObservableProperty]
private string _byPaymentTo;

[ObservableProperty]
private string _byArrow;

[ObservableProperty]
private IBrush _byPaymentFromColor;

[ObservableProperty]
private IBrush _byPaymentToColor;

[ObservableProperty]
private IBrush _bybitPriceColor = Brushes.Gray;

[ObservableProperty]
private double _bybitUpArrowOpacity = 0;

[ObservableProperty]
private double _bybitDownArrowOpacity = 0;

[ObservableProperty]
private decimal _bybitPreviousPrice;

// General
[ObservableProperty]
private string _countdown;

[ObservableProperty]
private string _threshold;

[ObservableProperty]
private string _frequency;

```

```

[RelayCommand]
3 references
public async Task StartDiscrepancyCheckerAsync()
{
    _isRunning = true;
    await Task.Delay(100);
    while (_isRunning)
    {
        await CheckPriceDiscrepancyAsync();
        await Task.Delay(int.Parse(Frequency));
    }
}

[RelayCommand]
2 references
public void StopDiscrepancyChecker()
{
    _isRunning = false;
}

10 references
public string SelectedCurrency
{
    get => _selectedCurrency;
    set
    {
        SetProperty(ref _selectedCurrency, value, nameof(SelectedCurrency));
        OnCurrencyChanged();

        // Update the database with the new selected currency
        _ = SetCurrency(value);
    }
}

1 reference
public List<string> CurrencyOptions { get; }

2 references
public TradeViewModel()
{
    _binanceSocketClient = new BinanceSocketClient();
    _bybitSocketClient = new BybitSocketClient();

    CurrencyOptions = new List<string> { "Bitcoin", "Ethereum" };

    UserService.Instance.OnEmailChanged += (sender, args) =>
    {
        Email = UserService.Instance.Email;
        Console.WriteLine($"TradeViewModel loaded with email: {Email}");
    };

    Email = UserService.Instance.Email;

    _selectedCurrency = "Bitcoin";

    InitializeWebSocketsAsync();
    InitializeCurrency();

    StartDiscrepancyCheckerAsync();
}

1 reference
public async Task InitializeCurrency()
{
    string dbCurrency = await GetCurrency();
    if (dbCurrency != null)
    {
        SelectedCurrency = dbCurrency;
        OnPropertyChanged(nameof(SelectedCurrency)); // Ensures ComboBox updates
    }
    else
    {
        Console.WriteLine("No currency set in database; using default.");
    }
}

1 reference
private async Task SetCurrency(string currency)
{
    string updateQuery = "UPDATE current_currency SET currency = @currency WHERE email = @Email";
    string insertQuery = "INSERT INTO current_currency (currency, email) VALUES (@currency, @Email)";

    using (var connection = new MySqlConnection(connectionString))
    {
        await connection.OpenAsync();

        using (var updateCommand = new MySqlCommand(updateQuery, connection))
        {
            updateCommand.Parameters.AddWithValue("@currency", currency);
            updateCommand.Parameters.AddWithValue("@Email", Email);
        }
    }
}

```

```

    try
    {
        // Try to update the currency
        int rowsAffected = await updateCommand.ExecuteNonQueryAsync();

        // If no rows were updated, insert a new row with the email and currency
        if (rowsAffected == 0)
        {
            using (var insertCommand = new MySqlCommand(insertQuery, connection))
            {
                insertCommand.Parameters.AddWithValue("@currency", currency);
                insertCommand.Parameters.AddWithValue("@Email", Email);
                await insertCommand.ExecuteNonQueryAsync();
            }

            Console.WriteLine($"Current currency '{currency}' set in database for email {Email}.");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error setting current currency: {ex.Message}");
    }
}
}

```

```

1 reference
private async Task<string> GetCurrency()
{
    string query = "SELECT currency FROM current_currency WHERE email = @Email LIMIT 1";
    string currency = null;

    using (var connection = new MySqlConnection(connectionString))
    {
        await connection.OpenAsync();

        using (var command = new MySqlCommand(query, connection))
        {
            command.Parameters.AddWithValue("@Email", Email);

            try
            {
                // Execute the query and read the result asynchronously
                var result = await command.ExecuteScalarAsync();
                currency = result?.ToString();
                Console.WriteLine($"Retrieved currency '{currency}' from database for email {Email}.");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error retrieving current currency: {ex.Message}");
            }
        }
    }

    return currency;
}

```

```

1 reference
private async Task InitializeWebSocketsAsync()
{
    var currencies = new[] { "BTCUSDOT", "ETHUSDOT" };

    var tasks = new List<Task>
    {
        BinanceFuturesSocketAsync(currencies[0]),
        BinanceFuturesSocketAsync(currencies[1]),
        BybitFuturesSocketAsync(currencies[0]),
        BybitFuturesSocketAsync(currencies[1]),
        ConnectToBinanceFundingRate(currencies[0]),
        ConnectToBinanceFundingRate(currencies[1]),
        ConnectToBybitFundingRate(currencies[0]),
        ConnectToBybitFundingRate(currencies[1])
    };

    await Task.WhenAll(tasks);
}

```

```

3 references
private string OnCurrencyChanged()
{
    if (_selectedCurrency == "Bitcoin")
    {
        Console.WriteLine("Selected currency: Bitcoin (BTCUSDOT)");
        BinancePrice = _btcPrice_binance;
        BybitPrice = _btcPrice_bybit;
        return "BTCUSDOT";
    }
    else if (_selectedCurrency == "Ethereum")
    {
        Console.WriteLine("Selected currency: Ethereum (ETHUSDOT)");
        BinancePrice = _ethPrice_binance;
    }
}

```

```

    {
        Console.WriteLine("Selected currency: Ethereum (ETHUSD)");
        BinancePrice = _ethPrice_binance;
        BybitPrice = _ethPrice_bybit;
        return "ETHUSD";
    }

    return null;
}

2 references
private async Task BinanceFuturesSocketAsync(string currency)
{
    var tickerSubscriptionResult =
        await _binanceSocketClient.UsdFuturesApi.ExchangeData.SubscribeToTickerUpdatesAsync(currency,
            (update) =>
            {
                var newPrice = update.Data.LastPrice;

                // Keep track of the previous price for comparison
                if (currency == "BTCUSD")
                {
                    BinanceChangeColor(newPrice, _btcPreviousPrice_binance);
                    _btcPreviousPrice_binance = newPrice;

                    // Only update BinancePreviousPrice if Bitcoin is selected
                    if (SelectedCurrency == "Bitcoin")
                    {
                        BinancePreviousPrice = _btcPreviousPrice_binance;
                    }
                }
                else if (currency == "ETHUSD")
                {
                    BinanceChangeColor(newPrice, _ethPreviousPrice_binance);
                    _ethPreviousPrice_binance = newPrice;

                    // Only update BinancePreviousPrice if Ethereum is selected
                    if (SelectedCurrency == "Ethereum")
                    {
                        BinancePreviousPrice = _ethPreviousPrice_binance;
                    }
                }

                Dispatcher.UIThread.InvokeAsync(() =>
                {
                    if (SelectedCurrency == "Bitcoin")
                    {
                        BinancePrice = _btcPreviousPrice_binance;
                    }
                    else if (SelectedCurrency == "Ethereum")
                    {
                        BinancePrice = _ethPreviousPrice_binance;
                    }
                });
            });

    var tickerOrderBook = _binanceSocketClient.UsdFuturesApi.ExchangeData.SubscribeToOrderBookUpdatesAsync(currency,
        500,
        (update) =>
        {
            var orderBook = update.Data;

            var askPrice = new StringBuilder();
            var askQuantity = new StringBuilder();
            var bidPrice = new StringBuilder();
            var bidQuantity = new StringBuilder();

            foreach (var ask in orderBook.Asks.Take(5))
            {
                askPrice.AppendLine($"{ask.Price}");
                askQuantity.AppendLine($"{ask.Quantity}");
            }

            var bids = orderBook.Bids
                .TakeLast(5)
                .OrderByDescending(bid => bid.Price);

            foreach (var bid in bids)
            {

```

```

        bidPrice.AppendLine($"{bid.Price}");
        bidQuantity.AppendLine($"{bid.Quantity}");
    }

    Dispatcher.UIThread.InvokeAsync(() =>
    {
        if (SelectedCurrency == "Bitcoin" && currency == "BTCUSD")
        {
            BinanceAskPrice = askPrice.ToString();
            BinanceAskQuantity = askQuantity.ToString();
            BinanceBidPrice = bidPrice.ToString();
            BinanceBidQuantity = bidQuantity.ToString();
        }
        else if (SelectedCurrency == "Ethereum" && currency == "ETHUSD")
        {
            BinanceAskPrice = askPrice.ToString();
            BinanceAskQuantity = askQuantity.ToString();
            BinanceBidPrice = bidPrice.ToString();
            BinanceBidQuantity = bidQuantity.ToString();
        }
    });
});

```

2 references

```

private Task ConnectToBinanceFundingRate(string currency)
{
    string urlCurrency = currency.ToLower();
    return Task.Run(() =>
    {
        FundingRateTimer();
        string binanceWebSocketUrl = $"wss://fstream.binance.com/ws/{urlCurrency}@markPrice";

        _binanceFRWebSocket = new WebSocket(binanceWebSocketUrl);
        _binanceFRWebSocket.OnMessage += BinanceWebSocket_OnMessage;
        _binanceFRWebSocket.OnOpen += BinanceWebSocket_OnOpen;
        _binanceFRWebSocket.OnClose += BinanceWebSocket_OnClose;

        _binanceFRWebSocket.Connect();
    });
}

```

1 reference

```

private void BinanceWebSocket_OnMessage(object sender, WebSocketSharp.MessageEventArgs e)
{
    // Parse the message data
    var data = JObject.Parse(e.Data);
    var fundingRate = data["r"]?.ToObject<double>() ?? 0.0;
    var fundingAmount = data["p"]?.ToString();

    // Convert funding rate to percentage
    var fundingRatePercentage = fundingRate * 100;

    // Determine the payment direction and colors
    string paymentFrom;
    string paymentTo;
    var paymentFromColor = Brushes.Gray;
    var paymentToColor = Brushes.Gray;
    string arrowText;

    if (fundingRate > 0)
    {
        paymentFrom = "Longs";
        paymentTo = "Shorts";
        paymentFromColor = Brushes.DarkGreen;
        paymentToColor = Brushes.DarkRed;
        arrowText = "---->";
    }
    else if (fundingRate < 0)
    {
        paymentFrom = "Shorts";
        paymentTo = "Longs";
        paymentFromColor = Brushes.DarkRed;
        paymentToColor = Brushes.DarkGreen;
        arrowText = "---->";
    }
    else
    {
    }
}

```

```

    {
        paymentFrom = "No payment";
        paymentTo = "(Rate is 0)";
        paymentFromColor = Brushes.Gray;
        paymentToColor = Brushes.Gray;
        arrowText = "";
    }

    Dispatcher.UIThread.InvokeAsync(() =>
    {
        if (SelectedCurrency == "Bitcoin")
        {
            BinanceFundingRate = $"{FundingRatePercentage:F4}%";
            BPaymentFrom = paymentFrom;
            BPaymentTo = paymentTo;
            BArrow = arrowText;
            BPaymentFromColor = paymentFromColor;
            BPaymentToColor = paymentToColor;
        }
        else if (SelectedCurrency == "Ethereum")
        {
            BinanceFundingRate = $"{FundingRatePercentage:F4}%";
            BPaymentFrom = paymentFrom;
            BPaymentTo = paymentTo;
            BArrow = arrowText;
            BPaymentFromColor = paymentFromColor;
            BPaymentToColor = paymentToColor;
        }
    });
}

1 reference
private void BinanceWebSocket_OnOpen(object sender, EventArgs e)
{
    Console.WriteLine("Connected to Binance WebSocket.");
}

1 reference
private void BinanceWebSocket_OnClose(object sender, EventArgs e)
{
    Console.WriteLine("Disconnected from Binance WebSocket.");
}

2 references
private async Task BybitFuturesSocketAsync(string currency)
{
    var tickerSubscriptionResult =
        await _bybitSocketClient.V5LinearApi.SubscribeToTickerUpdatesAsync(currency,
            (update) =>
            {
                var newPrice = (decimal)update.Data.LastPrice;

                // Keep track of the previous price for comparison
                if (currency == "BTCUSDT")
                {
                    BybitChangeColor(newPrice, _btcPreviousPrice_bybit);
                    _btcPreviousPrice_bybit = newPrice;

                    // Only update BybitPreviousPrice if Bitcoin is selected
                    if (SelectedCurrency == "Bitcoin")
                    {
                        BybitPreviousPrice = _btcPreviousPrice_bybit;
                    }
                }
                else if (currency == "ETHUSDT")
                {
                    BybitChangeColor(newPrice, _ethPreviousPrice_bybit);
                    _ethPreviousPrice_bybit = newPrice;

                    // Only update BybitPreviousPrice if Ethereum is selected
                    if (SelectedCurrency == "Ethereum")
                    {
                        BybitPreviousPrice = _ethPreviousPrice_bybit;
                    }
                }

                // Update UI-bound properties
                Dispatcher.UIThread.InvokeAsync(() =>
                {

```

```

        if (SelectedCurrency == "Bitcoin")
        {
            BybitPrice = _btcPreviousPrice_bybit; // Use Bitcoin's price
        }
        else if (SelectedCurrency == "Ethereum")
        {
            BybitPrice = _ethPreviousPrice_bybit; // Use Ethereum's price
        }
    });
});

var tickerOrderBook = _bybitSocketClient.V5LinearApi.SubscribeToOrderbookUpdatesAsync(currency, 500,
(update) =>
{
    var orderBook = update.Data;

    var askPrice = new StringBuilder();
    var askQuantity = new StringBuilder();
    var bidPrice = new StringBuilder();
    var bidQuantity = new StringBuilder();

    foreach (var ask in orderBook.Asks.Take(5))
    {
        askPrice.AppendLine($"{ask.Price}");
        askQuantity.AppendLine($"{ask.Quantity}");
    }

    var bids = orderBook.Bids
        .TakeLast(5)
        .OrderByDescending(bid => bid.Price);

    foreach (var bid in bids)
    {
        bidPrice.AppendLine($"{bid.Price}");
        bidQuantity.AppendLine($"{bid.Quantity}");
    }

    Dispatcher.UIThread.InvokeAsync(() =>
    {
        if (SelectedCurrency == "Bitcoin" && currency == "BTCUSDT")
        {
            BybitAskPrice = askPrice.ToString();
            BybitAskQuantity = askQuantity.ToString();
            BybitBidPrice = bidPrice.ToString();
            BybitBidQuantity = bidQuantity.ToString();
        }
        else if (SelectedCurrency == "Ethereum" && currency == "ETHUSDT")
        {
            BybitAskPrice = askPrice.ToString();
            BybitAskQuantity = askQuantity.ToString();
            BybitBidPrice = bidPrice.ToString();
            BybitBidQuantity = bidQuantity.ToString();
        }
    });
});
}

2 references
public async Task ConnectToBybitFundingRate(string currency)
{
    // Set the symbol and limit
    string symbol = currency.ToUpper();
    int limit = 1; // Limit to get only the most recent data

    // Build the URL with query parameters
    string url = $"{apiUrl}?category=linear&symbol={symbol}";

    // Create an HttpClient instance
    using (HttpClient client = new HttpClient())
    {
        try
        {
            // Send an HTTP GET request to the Bybit API
            HttpResponseMessage response = await client.GetAsync(url);
            response.EnsureSuccessStatusCode();

            // Parse the response content
            string responseBody = await response.Content.ReadAsStringAsync();

```



```

    if (FundingData != null)
    {
        // Get the Funding rate directly
        double fundingRate = fundingData["FundingRate"]?.ToObject<double>() ?? 0.0;

        // Convert to percentage if needed
        double fundingRatePercentage = fundingRate * 100; // This was modified to reflect correct scaling

        string paymentFrom;
        string paymentTo;
        var paymentFromColor = Brushes.Gray;
        var paymentToColor = Brushes.Gray;
        string arrowText;

        if (fundingRate > 0)
        {
            paymentFrom = "Longs";
            paymentTo = "Shorts";
            paymentFromColor = Brushes.DarkGreen;
            paymentToColor = Brushes.DarkRed;
            arrowText = "---->";
        }
        else if (fundingRate < 0)
        {
            paymentFrom = "Shorts";
            paymentTo = "Longs";
            paymentFromColor = Brushes.DarkRed;
            paymentToColor = Brushes.DarkGreen;
            arrowText = "---->";
        }
        else
        {
            paymentFrom = "No payment";
            paymentTo = "(Rate is 0)";
            paymentFromColor = Brushes.Gray;
            paymentToColor = Brushes.Gray;
            arrowText = "";
        }

        Dispatcher.UIThread.InvokeAsync(() =>
        {
            if (SelectedCurrency == "Bitcoin")
            {
                BybitFundingRate = $"{fundingRatePercentage:F4}%";
                ByPaymentFrom = paymentFrom;
                ByPaymentTo = paymentTo;
                ByArrow = arrowText;
                ByPaymentFromColor = paymentFromColor;
                ByPaymentToColor = paymentToColor;
            }
            else if (SelectedCurrency == "Ethereum")
            {
                BybitFundingRate = $"{fundingRatePercentage:F4}%";
                ByPaymentFrom = paymentFrom;
                ByPaymentTo = paymentTo;
                ByArrow = arrowText;
                ByPaymentFromColor = paymentFromColor;
                ByPaymentToColor = paymentToColor;
            }
        });
    }
    else
    {
        Console.WriteLine("No funding history data available.");
    }
}
catch (HttpRequestException e)
{
    // Handle any HTTP request errors
    Console.WriteLine($"Request error: {e.Message}");
}
}

```

2 references

```

private void BinanceChangeColor(decimal newPrice, decimal previousPrice)
{
    Dispatcher.UIThread.InvokeAsync(() =>
    {

```



```

Dispatcher.UIThread.InvokeAsync(() =>
{
    // Set color based on price comparison
    if (newPrice >= previousPrice)
    {
        BinancePriceColor = Brushes.DarkGreen;
        BinanceUpArrowOpacity = 100; // Show up arrow
        BinanceDownArrowOpacity = 0; // Hide down arrow
    }
    else if (newPrice < previousPrice)
    {
        BinancePriceColor = Brushes.DarkRed;
        BinanceUpArrowOpacity = 0; // Hide up arrow
        BinanceDownArrowOpacity = 100; // Show down arrow
    }
    else
    {
        BinancePriceColor = Brushes.Gray; // Neutral color for no change
    }
});
}

```

2 references

```

private void BybitChangeColor(decimal newPrice, decimal previousPrice)
{
    Dispatcher.UIThread.InvokeAsync(() =>
    {
        // Set color based on price comparison
        if (newPrice >= previousPrice)
        {
            BybitPriceColor = Brushes.DarkGreen;
            BybitUpArrowOpacity = 100; // Show up arrow
            BybitDownArrowOpacity = 0; // Hide down arrow
        }
        else if (newPrice < previousPrice)
        {
            BybitPriceColor = Brushes.DarkRed;
            BybitUpArrowOpacity = 0; // Hide up arrow
            BybitDownArrowOpacity = 100; // Show down arrow
        }
        else
        {
            BybitPriceColor = Brushes.Gray; // Neutral color for no change
        }
    });
}

```

1 reference

```

private void FundingRateTimer()
{
    DispatcherTimer.Run(() =>
    {
        UpdateCountdown();
        return true; // Keep running
    }, TimeSpan.FromSeconds(1));
}

```

1 reference

```

private void UpdateCountdown()
{
    var now = DateTime.Now;
    var timeRemaining = _nextTargetTime - now;

    if (timeRemaining.TotalSeconds <= 0)
    {
        UpdateNextTargetTime();
        timeRemaining = _nextTargetTime - now;
    }

    Countdown = FormatTimeRemaining(timeRemaining);
}

```

1 reference

```

private void UpdateNextTargetTime()
{
    var now = DateTime.Now.TimeOfDay;
    foreach (var target in TargetTimes)
    {

```

```

private void UpdateNextTargetTime()
{
    var now = DateTime.Now.TimeOfDay;
    foreach (var target in TargetTimes)
    {
        if (now < target)
        {
            _nextTargetTime = DateTime.Today.Add(target);
            return;
        }
    }

    _nextTargetTime = DateTime.Today.Add(TargetTimes[0]).AddDays(1);
}

1 reference
private string FormatTimeRemaining(TimeSpan timeSpan)
{
    return $"{timeSpan.Hours:D2}:{timeSpan.Minutes:D2}:{timeSpan.Seconds:D2}";
}

1 reference
private async Task CheckPriceDiscrepancyAsync()
{
    // Define fee rates
    decimal binanceTakerFeeRate = 0.0005m; // 0.0500% taker fee rate
    decimal bybitTakerFeeRate = 0.00055m; // 0.055% taker fee rate

    decimal bybitPrice = BybitPrice;
    decimal binancePrice = BinancePrice;
    decimal discrepancy = Math.Abs(bybitPrice - binancePrice);

    // Check if the discrepancy is greater than or equal to the threshold
    if (discrepancy >= decimal.Parse(Threshold))
    {
        string higherExchange, lowerExchange;
        decimal higherPrice, lowerPrice;
        bool isBybitHigher = bybitPrice > binancePrice;

        decimal totalFees = 0;

        if (isBybitHigher)
        {
            higherExchange = "Bybit";
            higherPrice = bybitPrice;
            lowerExchange = "Binance";
            lowerPrice = binancePrice;

            decimal orderQuantity = decimal.Parse(Quantity); // Adjust according to your order quantity

            // Calculate the order values
            decimal bybitOrderValue = orderQuantity / bybitPrice;
            decimal binanceOrderValue = orderQuantity / binancePrice;

            // Calculate taker fees for Bybit and maker fees for Binance
            decimal bybitTakerFee = bybitOrderValue * bybitTakerFeeRate;
            decimal binanceTakerFee = binanceOrderValue * binanceTakerFeeRate;

            // Total fees are the sum of the taker fee (Bybit) and the maker fee (Binance)
            totalFees = bybitTakerFee + binanceTakerFee;
        }
        else
        {
            higherExchange = "Binance";
            higherPrice = binancePrice;
            lowerExchange = "Bybit";
            lowerPrice = bybitPrice;

            decimal orderQuantity = decimal.Parse(Quantity); // Adjust according to your order quantity

            // Calculate the order values
            decimal binanceOrderValue = orderQuantity / binancePrice;
            decimal bybitOrderValue = orderQuantity / bybitPrice;

            // Calculate maker fee for Binance and taker fee for Bybit
            decimal binanceTakerFee = binanceOrderValue * binanceTakerFeeRate;
            decimal bybitTakerFee = bybitOrderValue * bybitTakerFeeRate;

            // Total fees are the sum of the taker fee (Binance) and the maker fee (Bybit)
            totalFees = binanceTakerFee + bybitTakerFee;
        }
    }
}

```

```

else
{
    higherExchange = "Binance";
    higherPrice = binancePrice;
    lowerExchange = "Bybit";
    lowerPrice = bybitPrice;

    decimal orderQuantity = decimal.Parse(Quantity); // Adjust according to your order quantity

    // Calculate the order values
    decimal binanceOrderValue = orderQuantity / binancePrice;
    decimal bybitOrderValue = orderQuantity / bybitPrice;

    // Calculate maker fee for Binance and taker fee for Bybit
    decimal binanceTakerFee = binanceOrderValue * binanceTakerFeeRate;
    decimal bybitTakerFee = bybitOrderValue * bybitTakerFeeRate;

    // Total fees are the sum of the taker fee (Binance) and the maker fee (Bybit)
    totalFees = binanceTakerFee + bybitTakerFee;
}

// Adjust the discrepancy by subtracting total fees
decimal netDiscrepancy = discrepancy - totalFees;

// Proceed only if the net discrepancy meets or exceeds the threshold
if (netDiscrepancy >= decimal.Parse(Threshold))
{
    Console.WriteLine("\n");
    Console.WriteLine($"Found net discrepancy of {netDiscrepancy} dollars after fees");
    Console.WriteLine($"*{higherExchange}: {higherPrice} dollars : Sell/Short position");
    Console.WriteLine($"*{lowerExchange}: {lowerPrice} dollars : Buy/Long position");

    await HandleDiscrepancyAndTradeAsync(isBybitHigher);

    await MonitorAndClosePositionAsync(isBybitHigher, decimal.Parse(Threshold) / 2);
}
}

```

```

1 reference
private async Task MonitorAndClosePositionAsync(bool isBybitHigher, decimal closeThreshold)
{
    decimal currentDiscrepancy;

    // Monitor until the discrepancy falls below the close threshold
    do
    {
        decimal bybitPrice = BybitPrice;
        decimal binancePrice = BinancePrice;
        currentDiscrepancy = Math.Abs(bybitPrice - binancePrice);

        // Wait for a short interval before rechecking
        await Task.Delay(500);
    } while (currentDiscrepancy >= closeThreshold);

    Console.WriteLine($"Discrepancy fell below {closeThreshold}. Closing positions.");

    // Close the open positions once the discrepancy is below the close threshold
    await ClosePositionsAsync(isBybitHigher);
}

```

```

1 reference
private async Task ClosePositionsAsync(bool isBybitHigher)
{
    // If Bybit was higher at the opening, now close the Bybit short and Binance long
    if (isBybitHigher)
    {
        await OpenCloseBybitPositions(OrderSide.Buy); // Close short on Bybit
        await OpenCloseBinancePositions(Binance.Net.Enums.OrderSide.Sell); // Close long on Binance
    }
    // If Binance was higher at the opening, close the Binance short and Bybit long
    else
    {
        await OpenCloseBybitPositions(OrderSide.Sell); // Close long on Bybit
        await OpenCloseBinancePositions(Binance.Net.Enums.OrderSide.Buy); // Close short on Binance
    }
}

```

1 reference

```
private async Task ClosePositionsAsync(bool isBybitHigher)
{
    // If Bybit was higher at the opening, now close the Bybit short and Binance long
    if (isBybitHigher)
    {
        await OpenCloseBybitPositions(OrderSide.Buy); // Close short on Bybit
        await OpenCloseBinancePositions(Binance.Net.Enums.OrderSide.Sell); // Close long on Binance
    }
    // If Binance was higher at the opening, close the Binance short and Bybit long
    else
    {
        await OpenCloseBybitPositions(OrderSide.Sell); // Close long on Bybit
        await OpenCloseBinancePositions(Binance.Net.Enums.OrderSide.Buy); // Close short on Binance
    }
}
```

1 reference

```
private async Task HandleDiscrepancyAndTradeAsync(bool isBybitHigher)
```

```
{
    // If Bybit is higher, short Bybit and long Binance
    if (isBybitHigher)
    {
        await OpenCloseBybitPositions(OrderSide.Sell);
        await OpenCloseBinancePositions(Binance.Net.Enums.OrderSide.Buy);
    }
    // If Binance is higher, short Binance and long Bybit
    else
    {
        await OpenCloseBybitPositions(OrderSide.Buy);
        await OpenCloseBinancePositions(Binance.Net.Enums.OrderSide.Sell);
    }
}
```

```
public static readonly string Bybit_API_KEY = "aVcZaYygMMUqRwDTdJ";
public static readonly string Bybit_API_SECRET = "Hsv94KqPncici7DdHBOVuIcSwvypPn9jVnt";
public static readonly BybitEnvironment By_env = BybitEnvironment.Testnet;

public static readonly string Binance_API_KEY = "e84bee016bbe783cc1a965bd189c2ca18fe0686449a6dce0eb9546800632d3f3";
public static readonly string Binance_API_SECRET = "ba3d8443f3ce00335d91feca0a0ffafa44ad2c7dbbae0184b10b2453a3a61c3";
public static readonly BinanceEnvironment Bin_env = BinanceEnvironment.Testnet;
```

```
[ObservableProperty]
private string _quantity;
```

4 references

```
private async Task OpenCloseBybitPositions(OrderSide side)
```

```
{
    var currency = OnCurrencyChanged();

    var bybitClient = new BybitRestClient(options =>
    {
        options.ApiCredentials = new ApiCredentials(Bybit_API_KEY, Bybit_API_SECRET);
        options.Environment = By_env;
    });

    var newOrder = await bybitClient.V5Api.Trading.PlaceOrderAsync(
        category: Category.Linear,
        symbol: currency,
        side: side,
        type: NewOrderType.Market,
        quantity: decimal.Parse(Quantity),
        timeInForce: TimeInForce.ImmediateOrCancel
    );

    if (!newOrder.Success)
    {
        Console.WriteLine($"Failed to create an order on Bybit! Error: {newOrder.Error.Message}");
        return;
    }

    Console.WriteLine($"Opened {side} position on Bybit. Order ID: {newOrder.Data.OrderId}");
    await SetTransactionData(newOrder.Data.OrderId, BybitPrice, decimal.Parse(Quantity), "Bybit", currency, side.ToString());
}
```

4 references

```
private async Task OpenCloseBinancePositions(Binance.Net.Enums.OrderSide side)
{
    var currency = OnCurrencyChanged();

    var binanceClient = new BinanceRestClient(options =>
    {
        options.ApiCredentials = new ApiCredentials(Binance_API_KEY, Binance_API_SECRET);
        options.Environment = Bin_env;
    });

    // Set position side explicitly based on the order side
    var positionSide = side == Binance.Net.Enums.OrderSide.Buy
        ? Binance.Net.Enums.PositionSide.Long
        : Binance.Net.Enums.PositionSide.Short;

    var newOrder = await binanceClient.UsdFuturesApi.Trading.PlaceOrderAsync(
        symbol: currency,
        side: side,
        type: Binance.Net.Enums.FuturesOrderType.Market,
        quantity: decimal.Parse(Quantity),
        positionSide: positionSide
    );

    if (!newOrder.Success)
    {
        Console.WriteLine($"Failed to create an order on Binance! Error: {newOrder.Error.Message}");
        return;
    }

    Console.WriteLine($"Opened {side} position on Binance. Order ID: {newOrder.Data.Id}");

    await SetTransactionData(newOrder.Data.Id.ToString(), BinancePrice, decimal.Parse(Quantity), "Binance", currency, side.ToString());
}
```

2 references

```
private async Task SetTransactionData(string orderId, decimal price, decimal quantity, string exchange, string currency, string action, decimal? profit = null)
{
    string insertQuery = @"
INSERT INTO transactions
(order_id, price, quantity, exchange, currency, action, profit, date, time, email)
VALUES
(@orderId, @price, @quantity, @exchange, @currency, @action, @profit, @date, @time, @Email)";

    using (var connection = new MySqlConnection(connectionString))
    {
        await connection.OpenAsync();

        using (var command = new MySqlCommand(insertQuery, connection))
        {
            // Adding parameters to prevent SQL injection
            command.Parameters.AddWithValue("@orderId", orderId);
            command.Parameters.AddWithValue("@price", price);
            command.Parameters.AddWithValue("@quantity", quantity);
            command.Parameters.AddWithValue("@exchange", exchange);
            command.Parameters.AddWithValue("@currency", currency);
            command.Parameters.AddWithValue("@action", action);
            command.Parameters.AddWithValue("@profit", profit.HasValue ? (object)profit.Value : DBNull.Value);
            command.Parameters.AddWithValue("@date", DateTime.UtcNow.Date);
            command.Parameters.AddWithValue("@time", DateTime.UtcNow.TimeOfDay);
            command.Parameters.AddWithValue("@Email", Email);

            try
            {
                await command.ExecuteNonQueryAsync();
                Console.WriteLine($"Transaction recorded for {exchange} on {currency}: {action} position, Order ID: {orderId}");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error recording transaction: {ex.Message}");
            }
        }
    }
}
```

4. Transactions ViewModel

```
using System;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Diagnostics;
using CommunityToolkit.Mvvm.ComponentModel;

namespace AutoArbitrage_MVVM_ViewModels;

// 16 references
public class Order
{
    // 16 references
    public string Price { get; set; }
    // 16 references
    public string Quantity { get; set; }
    // 16 references
    public string Exchange { get; set; }
    // 16 references
    public string Currency { get; set; }
    // 16 references
    public string Action { get; set; }
    // 16 references
    public string Profit { get; set; }
    // 16 references
    public string Date { get; set; }
    // 16 references
    public string Time { get; set; }
}

// 4 references
public partial class TransactionsViewModel : ObservableObject
{
    // 2 references
    public ObservableCollection<Order> Orders { get; set; }

    // 2 references
    public TransactionsViewModel()
    {
        Orders = new ObservableCollection<Order>
        {
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "q", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "usdtbtcUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSdsdSDT", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSDassada", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUsasdasdsdT", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00"),
            new Order(Price = "12000", Quantity = "180", Exchange = "Binance", Currency = "BTCUSD", Action = "Long", Profit = "48%", Date = "Wednesday", Time = "12:00")
        };

        Console.WriteLine($"Orders count: {Orders.Count}");
    }

    public event PropertyChangedEventHandler? PropertyChanged;

    [PropertyChanged] protected virtual void OnPropertyChanging(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```


5. Wallet ViewModel

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
using Binance.Net;
using Binance.Net.Clients;
using Bybit.Net;
using Bybit.Net.Clients;
using Bybit.Net.Enums;
using CommunityToolkit.Mvvm.ComponentModel;
using CryptoExchange.Net.Authentication;
using DynamicData.Kernel;
using Google.Protobuf.WellKnownTypes;
using LiveChartsCore;
using LiveChartsCore.SkiaSharpView;
using LiveChartsCore.SkiaSharpView.Extensions;

namespace AutoArbitrage_MVVM.ViewModels;

5 references
public partial class WalletViewModel : ObservableObject
{
    public static readonly string Bybit_API_KEY = "aVcZaYygMMUqRwDT4J";
    public static readonly string Bybit_API_SECRET = "Hsv944KqPncici7DdHBOVuIcSwvypPn9jVnt";
    public static readonly BybitEnvironment By_env = BybitEnvironment.Testnet;

    public static readonly string Binance_API_KEY = "e84bee016bbe783cc1a965bd189c2ca18fe0686449a6dce0eb9546800632d3f3";
    public static readonly string Binance_API_SECRET = "ba3d8443f3ce00335d91feca0a0ffalaf41ad2c7dbbae0184b10b2453a3ae1c3";
    public static readonly BinanceEnvironment Bin_env = BinanceEnvironment.Testnet;

    0 references
    public IEnumerable<ISeries> Series { get; set; } =
        new[] { 2, 4, 1, 4, 3 }.AsPieSeries((value, series) =>
        {
            series.MaxRadialColumnWidth = 60;
        });
}
```

```
[ObservableProperty]
private string _bybitBalance;

[ObservableProperty]
private string _bybitEquity;

[ObservableProperty]
private List<ISeries> _bybitSeries = new();

[ObservableProperty]
private string _binanceBalance;

[ObservableProperty]
private string _binanceEquity;

[ObservableProperty]
private List<ISeries> _binanceSeries = new();

2 references
public WalletViewModel()
{
    GetBybitBalance();
    GetBinanceBalance();
}

2 references
private int GetSortOrder(string asset)
{
    // Custom sort order
    var customOrder = new Dictionary<string, int>
    {
        { "BTC", 1 },
        { "ETH", 2 },
        { "USDT", 3 },
        { "USDC", 4 },
        { "Others", int.MaxValue } // Ensure "Others" always comes last
    };
}
```

7. Profile ViewModel

```
using System;
using System.Threading.Tasks;
using AutoArbitrage_MVVM.Services;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using MySql.Data.MySqlClient;

namespace AutoArbitrage_MVVM.ViewModels
{
    4 references
    public class User
    {
        3 references
        public int Id { get; set; }
        2 references
        public string Email { get; set; }
        1 reference
        public string PasswordHash { get; set; }
        1 reference
        public string Salt { get; set; }
        2 references
        public string? BinanceKey { get; set; }
        2 references
        public string? BybitKey { get; set; }
        2 references
        public string? BinanceSecret { get; set; }
        2 references
        public string? BybitSecret { get; set; }
        2 references
        public string? FullName { get; set; }
        2 references
        public string? Phone { get; set; }
        2 references
        public DateTime CreatedAt { get; set; }
    }

    6 references
    public partial class ProfileViewModel : ObservableObject
    {
        private string connectionString = "Server=autoarbitrage.cri2yu04sa9j.me-central-1-rds.amazonaws.com;Database=userdb;User ID=admin;Password=autoarbitrage12;";

        private string? _email;
        8 references
        public string? Email
        {
            get => _email;
            set => SetProperty(ref _email, value);
        }

        private int? _id;
        1 reference
        public int? Id
        {
            get => _id;
            set => SetProperty(ref _id, value);
        }

        // Observable Objects
        [ObservableProperty] private string displayEmail;
        [ObservableProperty] private string name;
        [ObservableProperty] private string phone;
        [ObservableProperty] private string joined;
        [ObservableProperty] private string binanceKey;
        [ObservableProperty] private string bybitKey;
        [ObservableProperty] private string binanceSecret;
        [ObservableProperty] private string bybitSecret;

        2 references
        public ProfileViewModel()
        {
            // Initialize Email immediately if available
            Email = UserService.Instance.Email;

            // Register for email updates
            UserService.Instance.OnEmailChanged += (sender, args) =>
            {
                Email = UserService.Instance.Email;
                Console.WriteLine($"ProfileViewModel loaded with email: {Email}");
            };
        }
    }
}
```


[RelayCommand]

2 references

public async Task SaveInfo()

```
{
    int userId = await GetIDAsync();
    Console.WriteLine($"Attempting to update user with Id: {userId}");

    if (userId == 0)
    {
        Console.WriteLine("Error: Id is null or invalid, unable to update user information.");
        return;
    }

    try
    {
        using var connection = new MySqlConnection(connectionString);
        await connection.OpenAsync();

        // Define the SQL query to update user info
        string query = @"
UPDATE users SET
    fullname = @Name,
    email = @Email,
    phone = @Phone,
    binance_key = @BinanceKey,
    bybit_key = @BybitKey,
    binance_secret = @BinanceSecret,
    bybit_secret = @BybitSecret
WHERE id = @UserId";

        // Prepare the command with parameters to avoid SQL injection
        using var command = new MySqlCommand(query, connection);
        command.Parameters.AddWithValue("@Name", Name ?? string.Empty);
        command.Parameters.AddWithValue("@Email", DisplayEmail ?? string.Empty);
        command.Parameters.AddWithValue("@Phone", Phone ?? string.Empty);
        command.Parameters.AddWithValue("@BinanceKey", BinanceKey ?? string.Empty);
        command.Parameters.AddWithValue("@BybitKey", BybitKey ?? string.Empty);
        command.Parameters.AddWithValue("@BinanceSecret", BinanceSecret ?? string.Empty);
        command.Parameters.AddWithValue("@BybitSecret", BybitSecret ?? string.Empty);
        command.Parameters.AddWithValue("@UserId", userId);

        // Execute the update command
        int affectedRows = await command.ExecuteNonQueryAsync();

        if (affectedRows > 0)
        {
            Console.WriteLine("User information updated successfully.");
        }
        else
        {
            Console.WriteLine("Failed to update user information. No rows affected.");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error updating user information: {ex.Message}");
    }
}
```

1 reference

private async Task<int> GetIDAsync()

```
{
    var user = await GetInfo();
    return user?.Id ?? 0;
}
```

2 references

private async Task<User?> GetInfo()

```
{
    if (string.IsNullOrEmpty(Email))
    {
        Console.WriteLine("Email is not set. Unable to query the user.");
        return null;
    }

    Console.WriteLine($"Attempting to find user with email: {Email}");
    string query = "SELECT * FROM users WHERE email = @Email LIMIT 1";
    User? user = null;
```

```

    try
    {
        using (var reader = await command.ExecuteReaderAsync())
        {
            if (await reader.ReadAsync())
            {
                user = new User
                {
                    Id = reader.GetInt32(reader.GetOrdinal("id")),
                    Email = reader.GetString(reader.GetOrdinal("email")),
                    PasswordHash = reader.GetString(reader.GetOrdinal("password_hash")),
                    Salt = reader.GetString(reader.GetOrdinal("salt")),
                    BinanceKey = reader.IsDBNull(reader.GetOrdinal("binance_key")) ? null : reader.GetString(reader.GetOrdinal("binance_key")),
                    BybitKey = reader.IsDBNull(reader.GetOrdinal("bybit_key")) ? null : reader.GetString(reader.GetOrdinal("bybit_key")),
                    BinanceSecret = reader.IsDBNull(reader.GetOrdinal("binance_secret")) ? null : reader.GetString(reader.GetOrdinal("binance_secret")),
                    BybitSecret = reader.IsDBNull(reader.GetOrdinal("bybit_secret")) ? null : reader.GetString(reader.GetOrdinal("bybit_secret")),
                    FullName = reader.IsDBNull(reader.GetOrdinal("fullname")) ? null : reader.GetString(reader.GetOrdinal("fullname")),
                    Phone = reader.IsDBNull(reader.GetOrdinal("phone")) ? null : reader.GetString(reader.GetOrdinal("phone")),
                    CreatedAt = reader.GetDateTime(reader.GetOrdinal("created_at"));
                };
                Console.WriteLine($"Retrieved user info for email {Email}.");
            }
            else
            {
                Console.WriteLine($"No user found with email {Email}.");
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error retrieving user info: {ex.Message}");
    }
}

return user;
}

//reference
private async Task DisplayInfo()
{
    User? user = await GetInfo();
    if (user != null)
    {
        Id = user.Id;
        DisplayEmail = $"{user.Email}";
        Name = $"{user.FullName}";
        Phone = $"{user.Phone}";
        BinanceKey = $"{user.BinanceKey}";
        BybitKey = $"{user.BybitKey}";
        BinanceSecret = $"{user.BinanceSecret}";
        BybitSecret = $"{user.BybitSecret}";
        Joined = $"{user.CreatedAt}";
    }
    else
    {
        Console.WriteLine("No user found.");
    }
}

//reference
private async Task<string> FormatDate(DateTime dateTime)
{
    int day = dateTime.Day;
    string monthName = dateTime.ToString("MMMM"); // Gets the full month name
    int year = dateTime.Year;

    string suffix = day % 10 == 1 && day % 100 != 11 ? "st"
        : day % 10 == 2 && day % 100 != 12 ? "nd"
        : day % 10 == 3 && day % 100 != 13 ? "rd"
        : "th";

    return $"{day}{suffix} {monthName}, {year}";
}

```

VII. Test Plans

1. Testing Goals

This section outlines the testing strategy for AutoArbitrage, intended for the development and testing teams. The primary aim is to ensure the platform functions seamlessly, delivering a secure and efficient cryptocurrency arbitrage trading experience. Given the reliance on real-time data processing, API integrations, and advanced security protocols, testing will proactively identify and address potential issues. The system will be validated against its requirements to ensure all functions and sub-functions operate as expected. Particular focus will be placed on the system's ability to handle exceptions gracefully, such as unexpected user inputs or API failures, without compromising functionality or security. The platform's performance will also be tested in terms of uptime, response times, and arbitrage calculation accuracy.

2. Test Cases

Log in

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Empty Username Field	Username: (empty) Password: valid password	Error message "Please fill in all fields."	Error message "Please fill in all fields."	Pass
002	Empty Password Field	Username: valid username Password: (empty)	Error message "Please fill in all fields."	Error message "Please fill in all fields."	Pass
003	Empty Username and Password	Username: (empty) Password: (empty)	Error message "Please fill in all fields."	Error message "Please fill in all fields."	Pass
004	Invalid Username	Username: invalid username Password: valid password	Error message "Invalid email or password."	Error message "Invalid email or password."	Pass
005	Invalid Password	Username: valid username Password: invalid password	Error message "Invalid email or password."	Error message "Invalid email or password."	Pass
006	Valid Credentials	Username: valid username Password: valid password	Logs in	Logs in	Pass

Sign up

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Empty Email Address Field	Leaves 'Email Address' field blank	Error Message below "Please fill in all fields."	Error Message below "Please fill in all fields."	Pass
002	Empty Password Field	Leaves 'Password' field blank	Error Message below "Please fill in all fields."	Error Message below "Please fill in all fields."	Pass
003	Empty Confirm Password Field	Leaves 'Confirm Password' field blank	Error Message below "Please fill in all fields."	Error Message below "Please fill in all fields."	Pass
004	Empty Full Name Field	Leaves 'Full Name' field blank	Error Message below "Please fill in all fields."	Error Message below "Please fill in all fields."	Pass
005	Empty Phone Numbers Field	Leaves 'Phone Number' field blank	Error Message below "Please fill in all fields."	Error Message below "Please fill in all fields."	Pass
006	User attempts to sign-up with two different passwords	Password: "P@sswOrd1", Confirm Password: "P@sswOrd2"	System should prevent form submission and display an error message like: "Passwords do not match. Please try again."	System should prevent form submission and display an error message like: "Passwords do not match. Please try again."	Pass
007	User attempts to sign up with a weak password (e.g., too short)	Password: "pass123", Confirm Password: "pass123"	System should prevent form submission and display an error message "Password must be at least 8 characters long."	System should prevent form submission and display an error message "Password must be at least 8 characters long."	Pass
008	User attempts to sign up with a valid password	Password: "S3cur3P@\$\$wOrd!", Confirm Password: "S3cur3P@\$\$wOrd!"	System should allow form submission and proceed to the accessing the features of the App	As expected	Pass
009	User attempts to sign up with an invalid email	Email: abcdedfg	System should prevent sign-up	System prevents sign-up	Pass

AI - News API

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Fetch news articles from API	Send API requests for news articles	API returns a list of news articles with relevant data	API returns a list of news articles with relevant data	Pass
002	Send fetched news articles to flask app	Receive successful API response with articles	News articles with titles, descriptions, etc., are sent to the flask server	News articles with titles, descriptions, etc., are sent to the flask server	Pass
003	Handle API error responses	Receive an error response from the API	App displays an error in the command line	App displays an error in the command line	Pass
004	Measure news retrieval time	Send API request for news	API response time is within acceptable limits (e.g., under 10 seconds)	API response time is within acceptable limits (e.g., under 10 seconds)	Pass
005	Send fetched news articles to flask app	Receive successful API response with articles	News articles with titles, descriptions, etc., are sent to the flask server	News articles with titles, descriptions, etc., are sent to the flask server	Pass

AI - Ngrok Server (Acts as API gateway)

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Start ngrok server	Execute the command to start ngrok	Ngrok server starts successfully and displays connection status and forwarding URLs	Ngrok server starts successfully and displays connection status and forwarding URLs	Pass

AI - Flask Server

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Start Flask Server	Run the Python file	API returns a list of news articles with sentiment	API returns a list of news articles with sentiment	Pass
002	Verify ngrok tunnel to flask server	Access the ngrok URL to flask app	Successful connection to the Flask development server, content from Flask app is displayed in the browser	Successful connection to the Flask development server, content from Flask app is displayed in the browser	Pass
003	Start Flask Server	Run the Python file	API returns a list of news articles with sentiment	API returns a list of news articles with sentiment	Pass
004	Handle a large number of requests,	Send multiple simultaneous	News articles are displayed in the	News articles are displayed	Pass
	around 50 a second	requests	app UI with titles, description, etc.	in the app UI with titles, description, etc.	
005	Verify ngrok tunnel to flask server	Access the ngrok URL to flask app	Successful connection to the Flask development server, content from Flask app is displayed in the app	Successful connection to the Flask development server, content from Flask app is displayed in the app	Pass

Binance & Bybit API

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	API completely unavailable	Attempt to place any order (buy, sell, limit, market, etc)	Order Placement fails. Error message is displayed, indicating API is currently unavailable and advising the user to try again	Order Placement hangs indefinitely, with no feedback to the user or system crashes without a clear message.	Pass
002	User holds position	Attempt to place any order (buy, sell, limit, market, etc)	Order Placement succeeds	Order Placement succeeds	Pass
003	User holds position without Setting API keys and API Secrets	Attempt to place any order without set API Keys and API Secrets	Unable to place order. Trade section locked.	Unable to place order. Trade section locked.	Pass

Latency

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Place Market Order	Buy X BTC	Order Execution within 100 milliseconds	Order Execution within 100 milliseconds	Pass
002	Place Limit Order	Buy X ETH at \$1500	Order Execution within 200 milliseconds NO	Order not executed	Pass
003	Stop Holding Position	Close position	Message confirming that the position has been closed	Position not closed	Pass
004	Receive market data update(BTC)	Simulate BTC price change on text exchange feed	Price update displayed	Price not updated frequently	Pass

Performance

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Measure page load time	Access a specific page	Page load time should be within acceptable limits (e.g., under 3 seconds) to ensure a responsive user experience	Page load time should be within acceptable limits (e.g., under 3 seconds) to ensure a responsive user experience	Pass
002	Measure API response time	Send an API request	API response time should be within acceptable limits (e.g., under 1 second) to ensure quick and efficient data retrieval.	API response time should be within acceptable limits (e.g., under 1 second) to ensure quick and efficient data retrieval.	Pass
003	Test database query performance	Execute database queries relevant to App functionality	Database queries should be executed efficiently	Database queries should be executed efficiently	Pass
004	Measure news retrieval time	Send API request for news	API response time is within acceptable limits (e.g., under 10 seconds)	API response time is within acceptable limits (e.g., under 10 seconds)	Pass

Load

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Test system with multiple transactions	Simulate a large number of transactions, around 20	System should handle the increased load without significant performance degradation.	System should handle the increased load without significant performance degradation	Pass
002	Test database scalability	Increase the database size to 1000 records or load to 20 or more	Database should scale effectively to handle the increased data volume without performance issues.	Database should scale effectively to handle the increased data volume without performance issues.	Pass

Usability

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Test App navigation	Attempt to navigate to different sections of the app.	App navigation should be intuitive and easy to use, allowing users to find the information they need quickly and efficiently	App navigation should be intuitive and easy to use, allowing users to find the information they need quickly and efficiently	Pass
002	Test content clarity and readability	Read and understand the content on different pages of the app.	App content should be clear, concise, and easy to read and understand, using appropriate language and terminology for the target audience.	App content should be clear, concise, and easy to read and understand, using appropriate language and terminology for the target audience.	Pass
003	Test error handling and feedback	Intentionally make errors while using the app.	Error messages should be clear, informative, and helpful, guiding users on how to correct the errors.	Error messages should be clear, informative, and helpful, guiding users on how to correct the errors.	Pass
004	Test cross-platform compatibility	Access the app from different Os's	App should be compatible with both Linux and windows.	App should be compatible with both Linux and windows.	Pass

Accessibility

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Verify screen reader compatibility	Navigate the app using a screen reader	Screen reader should accurately announce all page elements, including text, images, and interactive components	Screen reader not compatible	Fail
002	Verify keyboard navigation	Attempt to navigate the entire app using only the keyboard	All interactive elements, including buttons, links, and form fields, should be accessible and operable using the keyboard.	All interactive elements, including buttons, links, and form fields, are accessible and operable using the keyboard.	Pass
003	Verify color contrast	Analyse the color contrast of text and background elements using a color contrast tool	The color contrast ratio between text and background elements should meet WCAG (Web Content Accessibility Guidelines) standards for sufficient contrast, ensuring readability for users with visual impairments.	The color contrast ratio between text and background elements should meet WCAG (Web Content Accessibility Guidelines) standards for sufficient contrast, ensuring readability for users with visual impairments.	Pass

004	Verify alternative text for images	Inspect the C# or use developer tools to check for alternative text attributes	All images should have appropriate alternative text (alt text) that provides a textual description of the image content, making the content accessible to users who are blind or visually impaired.	All images should have appropriate alternative text (alt text) that provides a textual description of the image content, making the content accessible to users who are blind or visually impaired.	Pass
005	Verify form accessibility	Test form input fields and labels using a screen reader	Form labels should be correctly associated with their corresponding input fields, and error messages should be clear and informative, ensuring accessibility for users with disabilities.	Form labels should be correctly associated with their corresponding input fields, and error messages should be clear and informative, ensuring accessibility for users with disabilities.	Pass

Cybersecurity

Test Case ID	Scenario	Input	Expected Output	Actual Output	Result
001	Test password hashing with salt	Attempt to sign in with a password	Password should be hashed with a randomly generated salt, ensuring unique hash values for users.	Password should be hashed with a randomly generated salt, ensuring unique hash values for users.	Pass
002	Test MySQL injection vulnerability	Attempt SQL injection via sign-up/login text boxes	Application should block all SQL injection attempts, keeping database secure from unauthorized access.	Application should block all SQL injection attempts, keeping database secure from unauthorized access.	Pass
003	Test AWS security protocol integration	Check system security protocols for AWS compliance	System should inherit AWS security protocols, including data encryption and access controls.	System should inherit AWS security protocols, including data encryption and access controls.	Pass

KLM

1. Making a trade	m h p b p b p b p b h k h p b h k h p b	Time: 10.96 sec
2. Check News	m h p b m p b	Time: 5.20 sec

3. Traceability Matrix

Requirement No.	Requirement Description	Test Case ID	Test Result
1	Log in	001 002 003 004 005 006	ALL PASS
2	Sign up	001 002 003 004 005 006 007 008 009	ALL PASS
3	AI - News API	001 002 003 004 005	ALL PASS
4	AI - Ngrok Server	001	ALL PASS
5	AI - Flask Server	001 002 003 004 005	ALL PASS

6	Binance & Bybit	001	ALL PASS
---	-----------------	-----	----------

		002 003	
7	Latency	001 002 003 004	ALL PASS
8	Performance	001 002 003 004	ALL PASS
9	Load	001 002	ALL PASS
10	Usability	001 002 003 004	ALL PASS
11	Accessibility	001 002 003 004 005	FAIL PASS PASS PASS PASS
12	Cybersecurity	001 002 003	ALL PASS

VIII. Conclusion

1. Summary

AutoArbitrage exemplifies the transformative potential of financial technology by implementing algorithmic trading to address inefficiencies in cryptocurrency markets. This platform integrates with leading exchanges, Binance and Bybit, using WebSockets to deliver real-time data crucial for identifying arbitrage opportunities. By automating trade execution, AutoArbitrage eliminates the delays and inefficiencies inherent in manual trading, enabling users to capitalize on fleeting price discrepancies with unparalleled speed and accuracy.

The system employs advanced technologies, including C# for core development, Python for sentiment analysis through machine learning, and SQL for database management. Its modular architecture ensures efficient data handling and facilitates seamless integration with multiple exchanges. Additionally, AutoArbitrage addresses critical challenges like market volatility, data accuracy, scalability, and cybersecurity through solutions such as multi-threading, distributed computing, and robust encryption. With its intuitive user interface, the platform is accessible to novice traders while offering advanced customization for experienced investors, thus appealing to a broad target audience.

By providing actionable insights and automated trading capabilities, AutoArbitrage empowers users to navigate the complexities of cryptocurrency futures markets efficiently and securely. This comprehensive approach not only enhances trading outcomes but also sets the foundation for the platform's ongoing evolution in the fast-paced world of cryptocurrency trading.

2. Future Growth

The future of AutoArbitrage lies in expanding its capabilities to meet the evolving needs of cryptocurrency traders. One key area of growth is the addition of more cryptocurrency exchanges to the platform, enabling users to access a broader range of arbitrage opportunities across global markets. Enhanced scalability will be achieved by using advanced cloud solutions like AWS Lambda for serverless computing, ensuring the platform can handle increased data volumes and user demand without performance degradation.

The integration of predictive analytics and machine learning models will further refine trading strategies, providing users with insights into market trends and sentiment. This could include the incorporation of natural language processing (NLP) techniques to analyze news and social media for more accurate market predictions. Additionally, the development of mobile applications will extend the platform's accessibility, allowing users to monitor trades and execute strategies on the go.

In terms of security, future enhancements will include integrating blockchain-based authentication mechanisms and implementing zero-trust architecture for user and API interactions. This will further protect sensitive data and fortify the system against emerging cyber threats.

Finally, as the cryptocurrency landscape continues to evolve, AutoArbitrage will adapt to meet regulatory requirements across different jurisdictions. Compliance with laws such as GDPR, CCPA, and AML/KYC standards will ensure the platform's sustainability and trustworthiness. By staying at the forefront of innovation and addressing user needs proactively, AutoArbitrage aims to remain a leading solution in the competitive field of cryptocurrency arbitrage trading.

IX. References

1. Abimbola, O.A. (2022) Bitsgap review 2022: My experience with Bitsgap Bot: Spot trading bot, futures trading bot and..., Medium. Available at: <https://contentking.medium.com/bitsgap-review-2022-my-experience-with-bitsgapbot-spot-trading-bot-futures-trading-bot-and-77fd215ff372> (Accessed: 31 May 2024).
2. Graw, M. (2024) Cryptohopper review 2024. Available at: <https://www.techopedia.com/cryptocurrency/cryptohopper-review> (Accessed: 31 May 2024).
3. Major, J. (2024) Is Pionex safe? what beginners should know, Finbold. Available at: <https://finbold.com/guide/is-pionex-safe/> (Accessed: 31 May 2024).
4. Martin, V. (2024) What is the best crypto arbitrage scanner? Traders Union. Available at: <https://tradersunion.com/crypto-trading-strategies/arbitrage-trading/crypto-arbitragescanner/> (Accessed: 31 May 2024).
5. Nielson, J. (2024) 7 best crypto arbitrage scanners in 2024, CoinCodex. Available at: <https://coincodex.com/article/27241/crypto-arbitrage-scanner/> (Accessed: 31 May 2024).
6. PWC (2021) 3rd Annual Global Crypto Hedge Fund Report 2021. Available at: [https://www.pwc.com/gx/en/financial-services/pdf/3rd-annual-pwc-elwood-aimacrypto-hedge-fund-report-\(may-2021\).pdf](https://www.pwc.com/gx/en/financial-services/pdf/3rd-annual-pwc-elwood-aimacrypto-hedge-fund-report-(may-2021).pdf) (Accessed: 31 May 2024).
7. Pooja (2024) All you need to know about momentum trading, Samco securities limited. Available at: <https://www.samco.in/knowledge-center/articles/everything-about-momentum-trading/> (Accessed: 29 June 2024).
8. Saha, V. (2023) Predicting future cryptocurrency prices using machine learning algorithms, SCIRP. Available at: <https://www.scirp.org/journal/paperinformation?paperid=128965> (Accessed: 29 June 2024).
9. Software architecture diagramming and patterns (2024) Educative. Available at: <https://www.educative.io/blog/software-architecture-diagramming-and-patterns> (Accessed: 30 June 2024).
10. Team, T.I. (2021) What is momentum trading?, Investopedia. Available at: <https://www.investopedia.com/trading/introduction-to-momentum-trading/> (Accessed: 29 June 2024).

11. Quamara, S & Singh, AK (2022), 'A systematic survey on security concerns in cryptocurrencies: State-of-the-art and perspectives', *Computers & security*, vol. 113, pp. 102548-.
12. Kizza, JM (2024), 'Blockchains, Cryptocurrency, and Smart Contracts Technologies: Security Considerations', in *Guide to Computer Network Security*, Springer International Publishing AG, Switzerland
13. Sun, R, Wang, Q, Guo, L, Li, C, Zhang, Y, Wen, W, Lu, W & Yan, H (2022), 'Research Towards Key Issues of API Security', in *CYBER SECURITY, CNCERT 2021*, Springer Nature, SINGAPORE, pp. 179–192.
14. Vallabhaneni, R., (2024). Effects of Data Breaches on Internet of Things (IoT) Devices within the Proliferation of Daily-Life Integrated Devices.
15. Anderson, T. (2021). *Cross-platform Windows Presentation Framework, anyone? The short answer: yes. Unpacking Avalonia*. [online] Theregister.com. Available at: https://www.theregister.com/2021/03/30/crossplatform_wpf_anyone_unpacking_avolonia/ [Accessed 19 Nov. 2024].
16. Rider (2021). *ReSharper & Rider Improvements For Avalonia | The JetBrains Blog*. [online] The JetBrains Blog. Available at: <https://blog.jetbrains.com/dotnet/2021/04/12/improvements-for-resharper-rider-avalonia/> [Accessed 19 Nov. 2024].
17. Avaloniaui.net. (2024). *The MVVM Pattern | Avalonia Docs*. [online] Available at: <https://docs.avaloniaui.net/docs/concepts/the-mvvm-pattern/> [Accessed 19 Nov. 2024].
18. JAM (2011). *How to show 'if' condition on a sequence diagram?* [online] Stack Overflow. Available at: <https://stackoverflow.com/questions/8114770/how-to-show-if-condition-on-a-sequence-diagram>.
19. Vicente, V. (2024). *Risk Assessment Matrix: Overview and Guide*. [online] AuditBoard. Available at: <https://www.auditboard.com/blog/what-is-a-risk-assessment-matrix/>.

X. Appendices

Login View:

Login.axaml:

```
<Window xmlns="https://github.com/avaloniaui"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:viewModels="clr-namespace:AutoArbitrage_MVVM.ViewModels"
  mc:Ignorable="d" d:DesignWidth="1888" d:DesignHeight="768"
  Width="1888" Height="768"
  x:Class="AutoArbitrage_MVVM.Views.Login"
  Title="Login"
  WindowStartupLocation="CenterOwner"
  ExtendClientAreaToDecorationsHint="True"
  ExtendClientAreaChromeHints="NoChrome"
  ExtendClientAreaTitleBarHeightHint="0"
  x:DataType="viewModels:LoginViewModel">

  <Grid Background="#121528">

    <Grid.RowDefinitions>42.75,585,*</Grid.RowDefinitions>

    <StackPanel Grid.Row="0" Grid.Column="0" Orientation="Horizontal" Background="#888B14">
      <TextBlock
        Text="AutoArbitrage"
        FontSize="24"
        FontWeight="Bold"
        Foreground="White"
        VerticalAlignment="Center"
        Margin="11.25,0,813,0"/>

      <Button
        Name="Min"
        Click="Min_OnClick"
        Margin="2.625,0"
        Height="18.75"
        Width="18.75"
        CornerRadius="3.75"
        Background="DarkGreen"/>

      <Button
        Name="Stack"
        Click="Stack_OnClick"
        Margin="2.625,0"
        Height="18.75"
        Width="18.75"
        CornerRadius="3.75"
        Background="Orange"/>

      <Button
        Name="Close"
        Click="Close_OnClick"
        Margin="2.625,0"
        Height="18.75"
        Width="18.75"
        CornerRadius="3.75"
        Background="DarkRed"/>
    </StackPanel>

    <StackPanel Margin="20" Grid.Row="1">
      <TextBlock
        Text="Welcome"
        FontSize="70"
        FontWeight="Bold"
        Foreground="White"
        VerticalAlignment="Center"
        HorizontalAlignment="Left"
        Margin="20,0,0,0"/>

      <TextBlock
        Text="Back"
        FontSize="70"
        FontWeight="Bold"
        Foreground="White"
        VerticalAlignment="Center"
        HorizontalAlignment="Left"
        Margin="20,-10,0,0"/>

      <TextBlock
        Text="Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio."
        FontSize="20"
        Margin="20,0,0,0"/>
    </StackPanel>
  </Grid>

```

```

<TextBlock
    Text="Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio."
    FontSize="20"
    Foreground="White"
    VerticalAlignment="Center"
    HorizontalAlignment="Left"
    Margin="20,30,0,0"
    Width="500"
    TextWrapping="Wrap"/>

<TextBlock
    Text="Log In and Continue Your Journey with AutoArbitrage."
    FontSize="18"
    Foreground="White"
    VerticalAlignment="Center"
    HorizontalAlignment="Left"
    Margin="20,30,0,0"
    Width="500"
    TextWrapping="Wrap"/>
</StackPanel>

<StackPanel Grid.Row="2" VerticalAlignment="Bottom" Margin="0,0,0,-15">
    <StackPanel Orientation="Horizontal" HorizontalAlignment="Left" VerticalAlignment="Bottom" Margin="-41.25,0,0,30">
        <Rectangle Fill="#4C6FC7" Height="45" Width="252" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
        <Rectangle Fill="#4C6FC7" Height="45" Width="504" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
        <Rectangle Fill="#4C6FC7" Height="45" Width="375" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
    </StackPanel>

    <StackPanel Orientation="Horizontal" HorizontalAlignment="Left" VerticalAlignment="Bottom" Margin="-82.5,0,0,30">
        <Rectangle Fill="#10265F" Height="45" Width="252" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
        <Rectangle Fill="#10265F" Height="45" Width="504" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
        <Rectangle Fill="#10265F" Height="45" Width="375" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
    </StackPanel>

    <StackPanel Orientation="Horizontal" HorizontalAlignment="Left" VerticalAlignment="Bottom" Margin="-33.75,0,0,30">
        <Rectangle Fill="#839EE3" Height="45" Width="252" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
        <Rectangle Fill="#839EE3" Height="45" Width="504" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
        <Rectangle Fill="#839EE3" Height="45" Width="375" Margin="0,0,52.5,0" RadiusX="22.5" RadiusY="22.5"/>
    </StackPanel>
</StackPanel>

<Border Grid.Row="1"
    Width="400"
    Height="430"
    HorizontalAlignment="Right"
    VerticalAlignment="Bottom"
    Margin="0,0,50,50"
    CornerRadius="20"
    Background="#808C21">

    <StackPanel Margin="20">
        <TextBlock
            Text="Log In"
            FontSize="40"
            FontWeight="Bold"
            Foreground="White"
            HorizontalAlignment="Center"/>

        <TextBlock
            Text="Email Address"
            Foreground="#D9D9D9"
            Margin="0,20,0,0"/>

        <TextBox
            Name="Email"
            Text="{Binding Email, Mode=TwoWay}"
            Height="40"
            Foreground="Black"
            Background="#D9D9D9"
            FontSize="20"
            GotFocus="Email_OnGotFocus"
            LostFocus="Email_OnLostFocus"/>

        <TextBlock
            Text="Password"
            Foreground="#D9D9D9"
            Margin="0,20,0,0"/>
    </StackPanel>

```

```

        Foreground="White"
        HorizontalAlignment="Center"/>

<TextBlock
    Text="Email Address"
    Foreground="#D9D9D9"
    Margin="0,20,0,0"/>

<TextBox
    Name="Email"
    Text="{Binding Email, Mode=TwoWay}"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="Email_OnGotFocus"
    LostFocus="Email_OnLostFocus"/>

<TextBlock
    Text="Password"
    Foreground="#D9D9D9"
    Margin="0,20,0,0"/>

<TextBox
    Name="Password"
    Text="{Binding Password, Mode=TwoWay}"
    PasswordChar="*"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="Password_OnGotFocus"
    LostFocus="Password_OnLostFocus"/>

<Button
    Name="LoginButton"
    Content="Log In"
    Command="{Binding Login}"
    Background="#173055"
    Margin="0,40,0,0"
    Width="120"
    FontSize="20"
    Padding="10"
    Foreground="White"
    CornerRadius="10"
    HorizontalAlignment="Center"
    HorizontalContentAlignment="Center"
/>

<Button
    Name="ToLoginButton"
    Content="New To AutoArbitrage? Click Here"
    Command="{Binding ToSignUp}"
    Background="#808C21"
    Margin="0,10,0,0"
    FontSize="16"
    Foreground="White"
    HorizontalAlignment="Center"/>

<TextBlock
    Name="ErrorMessage"
    Text="{Binding ErrorMessage}"
    Margin="0,15,0,0"
    FontSize="14"
    FontWeight="SemiBold"
    Foreground="DarkRed"
    HorizontalAlignment="Center"
    TextWrapping="Wrap"/>

</StackPanel>
</Border>

</Grid>
</Window>

```

Login.axaml.cs

```
namespace AutoArbitrage_MVVM.Views;

4 references
public partial class Login : Window
{
    2 references
    public Login()
    {
        InitializeComponent();
        DataContext = new LoginViewModel(this);
    }

    0 references
    private void Min_OnClick(object? sender, RoutedEventArgs e)
    {
        this.WindowState = WindowState.Minimized;
    }

    0 references
    private void Stack_OnClick(object? sender, RoutedEventArgs e)
    {
        if (this.WindowState == WindowState.Maximized)
        {
            this.WindowState = WindowState.Normal;
        }
        else
        {
            this.WindowState = WindowState.Maximized;
        }
    }

    0 references
    private void Close_OnClick(object? sender, RoutedEventArgs e)
    {
        Close();
    }

    0 references
    private void Email_OnGotFocus(object? sender, GotFocusEventArgs e)
    {
        Email.Foreground = Brushes.White;
    }

    0 references
    private void Email_OnLostFocus(object? sender, RoutedEventArgs e)
    {
        Email.Foreground = Brushes.Black;
    }

    0 references
    private void Password_OnGotFocus(object? sender, GotFocusEventArgs e)
    {
        Password.Foreground = Brushes.White;
    }

    0 references
    private void Password_OnLostFocus(object? sender, RoutedEventArgs e)
    {
        Password.Foreground = Brushes.Black;
    }
}
```

MainWindow view:

MainWindow.axaml:

```

<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:vm="using:AutoArbitrage_MVVM.ViewModels"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:views="clr-namespace:AutoArbitrage_MVVM.Views"
        mc:Ignorable="d" d:DesignWidth="1080" d:DesignHeight="768"
        Width="1080" Height="768"
        x:Class="AutoArbitrage_MVVM.Views.MainWindow"
        x:DataType="vm:MainWindowViewModel"
        Title="AutoArbitrage_MVVM"
        WindowStartupLocation="CenterOwner"
        ExtendClientAreaToDecorationsHint="True"
        ExtendClientAreaChromeHints="NoChrome"
        ExtendClientAreaTitleBarHeightHint="0">

    <Design.DataContext>
        <!-- This only sets the DataContext for the previewer in an IDE,
             to set the actual DataContext for runtime, set the DataContext property in code (look at App.axaml.cs) -->
        <vm:MainWindowViewModel/>
    </Design.DataContext>

    <Grid>

        <Grid.RowDefinitions>42.75,104.25,*</Grid.RowDefinitions>

        <StackPanel Grid.Row="0" Grid.Column="0" Orientation="Horizontal" Background="#888814">
            <TextBlock
                Text="AutoArbitrage"
                FontSize="24"
                FontWeight="Bold"
                Foreground="White"
                VerticalAlignment="Center"
                Margin="11.25,0,813,0"/>

            <Button
                Name="Min"
                Click="Min_OnClick"
                Margin="2.625,0"
                Height="18.75"
                Width="18.75"
                CornerRadius="3.75"
                Background="DarkGreen"/>

            <Button
                Name="Stack"
                Click="Stack_OnClick"
                Margin="2.625,0"
                Height="18.75"
                Width="18.75"
                CornerRadius="3.75"
                Background="Orange"/>

            <Button
                Name="Close"
                Click="Close_OnClick"
                Margin="2.625,0"
                Height="18.75"
                Width="18.75"
                CornerRadius="3.75"
                Background="DarkRed"/>
        </StackPanel>

        <Grid Grid.Row="1" Background="#121528" Width="1080">

            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="25"/> <!-- Left margin -->
                <ColumnDefinition Width="78.75"/> <!-- Trade button -->
                <ColumnDefinition Width="72.75"/> <!-- Spacing -->
                <ColumnDefinition Width="85.5"/> <!-- Wallet button -->
                <ColumnDefinition Width="72.75"/> <!-- Spacing -->
                <ColumnDefinition Width="153"/> <!-- Transactions button -->
                <ColumnDefinition Width="72.75"/> <!-- Spacing -->
                <ColumnDefinition Width="74.25"/> <!-- News button -->
                <ColumnDefinition Width="72.75"/> <!-- Spacing -->
                <ColumnDefinition Width="99.75"/> <!-- Settings button -->
                <ColumnDefinition Width="72.75"/> <!-- Spacing -->
            </Grid.ColumnDefinitions>

```



```

        <ColumnDefinition Width="63"/> <!-- Rectangle -->
        <ColumnDefinition Width="25"/> <!-- Right margin -->
    </Grid.ColumnDefinitions>

    <Button
        Grid.Column="1"
        Name="Trade"
        Content="Trade"
        Click="Trade_OnClick"
        Command="{Binding OnNavigate}"
        CommandParameter="Trade"
        Background="#173055"
        Height="33"
        Width="78.75"
        CornerRadius="50"
        Foreground="White"
        FontSize="18"
        FontWeight="Light"
        HorizontalAlignment="Center"
        HorizontalContentAlignment="Center"/>

    <Button
        Grid.Column="3"
        Name="Wallet"
        Content="Wallet"
        Click="Wallet_OnClick"
        Background="Transparent"
        Height="33"
        Width="85.5"
        CornerRadius="50"
        Foreground="White"
        FontSize="18"
        FontWeight="Light"
        HorizontalAlignment="Center"
        HorizontalContentAlignment="Center"/>

    <Button
        Grid.Column="5"
        Name="PastTransactions"
        Content="Transactions"
        Click="PastTransactions_OnClick"
        Command="{Binding OnNavigate}"
        CommandParameter="Transactions"
        Background="Transparent"
        Height="33"
        Width="153"
        CornerRadius="50"
        Foreground="White"
        FontSize="18"
        FontWeight="Light"
        HorizontalAlignment="Center"
        HorizontalContentAlignment="Center"/>

    <Button
        Grid.Column="7"
        Name="Assistant"
        Content="News"
        Click="Assistant_OnClick"
        Command="{Binding OnNavigate}"
        CommandParameter="News"
        Background="Transparent"
        Height="33"
        Width="74.25"
        CornerRadius="50"
        Foreground="White"
        FontSize="18"
        FontWeight="Light"
        HorizontalAlignment="Center"
        HorizontalContentAlignment="Center"/>

    <Button
        Grid.Column="9"
        Name="Settings"
        Content="Settings"
        Command="{Binding OnNavigate}"
        CommandParameter="Settings"
        Click="Settings_OnClick"
        Background="Transparent"
        Height="33"

```



```

        CommandParameter="Transactions"
        Background="Transparent"
        Height="33"
        Width="153"
        CornerRadius="50"
        Foreground="White"
        FontSize="18"
        FontWeight="Light"
        HorizontalAlignment="Center"
        HorizontalContentAlignment="Center"/>

<Button
    Grid.Column="7"
    Name="Assistant"
    Content="News"
    Click="Assistant_OnClick"
    Command="{Binding OnNavigate}"
    CommandParameter="News"
    Background="Transparent"
    Height="33"
    Width="74.25"
    CornerRadius="50"
    Foreground="White"
    FontSize="18"
    FontWeight="Light"
    HorizontalAlignment="Center"
    HorizontalContentAlignment="Center"/>

<Button
    Grid.Column="9"
    Name="Settings"
    Content="Settings"
    Command="{Binding OnNavigate}"
    CommandParameter="Settings"
    Click="Settings_OnClick"
    Background="Transparent"
    Height="33"
    Width="99.75"
    CornerRadius="50"
    Foreground="White"
    FontSize="18"
    FontWeight="Light"
    HorizontalAlignment="Center"
    HorizontalContentAlignment="Center"/>

<Button
    Grid.Column="11"
    Name="Profile"
    Content="Profile"
    Click="Profile_OnClick"
    Command="{Binding OnNavigate}"
    CommandParameter="Profile"
    Background="Transparent"
    Height="33"
    Width="80.25"
    CornerRadius="50"
    Foreground="White"
    FontSize="18"
    FontWeight="Light"
    HorizontalAlignment="Center"
    HorizontalContentAlignment="Center"/>

<Image
    Grid.Column="13"
    Height="63"
    Width="63"
    Source="/Assets/user-circle.png"
    HorizontalAlignment="Left"/>
</Grid>

<Grid Grid.Row="2" Name="MainGrid">
    <ContentControl Name="MainContent"
        Content="{Binding CurrentView}"/>
</Grid>
</Grid>
</Window>

```

MainWindow.axaml.cs

```
using System;
using System.Diagnostics;
using AutoArbitrage_MVVM.ViewModels;
using Avalonia.Controls;
using Avalonia.Interactivity;
using Avalonia.Media;

namespace AutoArbitrage_MVVM.Views;

4 references
public partial class MainWindow : Window
{
    2 references
    public MainWindow()
    {
        InitializeComponent();
        DataContext = new MainWindowViewModel();
    }

    0 references
    private void Min_OnClick(object? sender, RoutedEventArgs e)
    {
        this.WindowState = WindowState.Minimized;
    }

    0 references
    private void Stack_OnClick(object? sender, RoutedEventArgs e)
    {
        if (this.WindowState == WindowState.Maximized)
        {
            this.WindowState = WindowState.Normal;
        }
        else
        {
            this.WindowState = WindowState.Maximized;
        }
    }

    0 references
    private void Close_OnClick(object? sender, RoutedEventArgs e)
    {
        Close();
    }

    0 references
    private void Trade_OnClick(object? sender, RoutedEventArgs e)
    {
        Trade.Background = new SolidColorBrush(Color.Parse("#173855"));
        Wallet.Background = Brushes.Transparent;
        PastTransactions.Background = Brushes.Transparent;
        Assistant.Background = Brushes.Transparent;
        Settings.Background = Brushes.Transparent;
        Profile.Background = Brushes.Transparent;
    }

    0 references
    private void Wallet_OnClick(object? sender, RoutedEventArgs e)
    {
        Trade.Background = Brushes.Transparent;
        Wallet.Background = new SolidColorBrush(Color.Parse("#173855"));
        PastTransactions.Background = Brushes.Transparent;
        Assistant.Background = Brushes.Transparent;
        Settings.Background = Brushes.Transparent;
        Profile.Background = Brushes.Transparent;
    }

    0 references
    private void PastTransactions_OnClick(object? sender, RoutedEventArgs e)
    {
        Trade.Background = Brushes.Transparent;
        Wallet.Background = Brushes.Transparent;
        PastTransactions.Background = new SolidColorBrush(Color.Parse("#173855"));
        Assistant.Background = Brushes.Transparent;
        Settings.Background = Brushes.Transparent;
        Profile.Background = Brushes.Transparent;
    }

    0 references
    private void Assistant_OnClick(object? sender, RoutedEventArgs e)
    {

```

0 references

```
private void Close_OnClick(object? sender, RoutedEventArgs e)
{
    Close();
}
```

0 references

```
private void Trade_OnClick(object? sender, RoutedEventArgs e)
{
    Trade.Background = new SolidColorBrush(Color.Parse("#173055"));
    Wallet.Background = Brushes.Transparent;
    PastTransactions.Background = Brushes.Transparent;
    Assistant.Background = Brushes.Transparent;
    Settings.Background = Brushes.Transparent;
    Profile.Background = Brushes.Transparent;
}
```

0 references

```
private void Wallet_OnClick(object? sender, RoutedEventArgs e)
{
    Trade.Background = Brushes.Transparent;
    Wallet.Background = new SolidColorBrush(Color.Parse("#173055"));
    PastTransactions.Background = Brushes.Transparent;
    Assistant.Background = Brushes.Transparent;
    Settings.Background = Brushes.Transparent;
    Profile.Background = Brushes.Transparent;
}
```

0 references

```
private void PastTransactions_OnClick(object? sender, RoutedEventArgs e)
{
    Trade.Background = Brushes.Transparent;
    Wallet.Background = Brushes.Transparent;
    PastTransactions.Background = new SolidColorBrush(Color.Parse("#173055"));
    Assistant.Background = Brushes.Transparent;
    Settings.Background = Brushes.Transparent;
    Profile.Background = Brushes.Transparent;
}
```

0 references

```
private void Assistant_OnClick(object? sender, RoutedEventArgs e)
{
    Trade.Background = Brushes.Transparent;
    Wallet.Background = Brushes.Transparent;
    PastTransactions.Background = Brushes.Transparent;
    Assistant.Background = new SolidColorBrush(Color.Parse("#173055"));
    Settings.Background = Brushes.Transparent;
    Profile.Background = Brushes.Transparent;
}
```

0 references

```
private void Settings_OnClick(object? sender, RoutedEventArgs e)
{
    Trade.Background = Brushes.Transparent;
    Wallet.Background = Brushes.Transparent;
    PastTransactions.Background = Brushes.Transparent;
    Assistant.Background = Brushes.Transparent;
    Settings.Background = new SolidColorBrush(Color.Parse("#173055"));
    Profile.Background = Brushes.Transparent;
}
```

0 references

```
private void Profile_OnClick(object? sender, RoutedEventArgs e)
{
    Trade.Background = Brushes.Transparent;
    Wallet.Background = Brushes.Transparent;
    PastTransactions.Background = Brushes.Transparent;
    Assistant.Background = Brushes.Transparent;
    Settings.Background = Brushes.Transparent;
    Profile.Background = new SolidColorBrush(Color.Parse("#173055"));
}
```

News View:

New.axaml:

```
<UserControl xmlns="https://github.com/avaloniaui"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
mc:Ignorable="d" d:DesignWidth="1088" d:DesignHeight="621"
xmlns:viewModels="clr-namespace:AutoArbitrage_MVVM.ViewModels"
Width="1088" Height="621"
x:Class="AutoArbitrage_MVVM.Views.News"
x:DataType="viewModels:NewsViewModel">
<Grid Background="#121528">
<Grid.RowDefinitions>*,</Grid.RowDefinitions>
<StackPanel Grid.Row="0" Orientation="Horizontal">
<TextBlock Foreground="White"
FontWeight="SemiBold"
FontSize="22"
Margin="36,10,0,14">News and Updates</TextBlock>
<TextBlock
Text="Next News Update In:"
Foreground="White"
HorizontalAlignment="Center"
FontSize="20"
FontWeight="ExtraLight"
Margin="460,12,0,14"/>
<TextBlock
Text="{Binding Countdown}"
Name="Countdown"
Foreground="Orange"
HorizontalAlignment="Center"
FontSize="22"
FontWeight="Bold"
Margin="10,11,0,0"/>
</StackPanel>
<StackPanel Grid.Row="1" Background="#121528" Orientation="Horizontal">
<ListBox
Name="NewsList" CornerRadius="16" Width="680" Height="550" MaxHeight="600"
Background="Transparent" HorizontalAlignment="Left" Margin="22,0,0,0">
</ListBox>
<StackPanel>
<StackPanel Orientation="Horizontal">
<TextBlock
Margin="20,20,0,0"
Padding="12"
Foreground="White"
Text="News Powered By :"/>
<Image Source="/Assets/coindesk.png"
Height="20"
Margin="0,20,0,0"/>
<TextBlock
Margin="-7.5,20,0,0"
Padding="12"
FontSize="16"
FontWeight="650"
Foreground="White"
Text="CoinDesk"/>
</StackPanel>
<TextBlock
Margin="20,10,0,0"
Padding="12"
FontSize="20"
FontWeight="SemiBold"
Foreground="White"
Text="Disclaimer:"/>
<TextBlock
Margin="20,-10,0,0"
Padding="12"
Width="325"
FontWeight="SemiLight"
Foreground="White"
TextWrapping="Wrap"
LetterSpacing="0.3">
```

```

        LineSpacing="4"
        Text="The sentiment analysis provided on news is done by AI. While useful, it may miss context or important details."/>
    <TextBlock
        Margin="20,-15,0,0"
        Padding="12"
        Width="325"
        FontWeight="SemiLight"
        Foreground="White"
        TextWrapping="Wrap"
        LetterSpacing="0.3"
        LineSpacing="4"
        Text="Always double-check the information yourself. Relying solely on AI can lead to mistakes."/>

    <TextBlock
        Margin="20,-15,0,0"
        Padding="12"
        Width="325"
        FontWeight="SemiLight"
        Foreground="White"
        TextWrapping="Wrap"
        LetterSpacing="0.3"
        LineSpacing="4"
        Text="Do thorough research from multiple sources before making any trading decisions. Don't let automated tools be your only guide."/>

    <TextBlock
        Margin="20,-15,0,0"
        Padding="12"
        Width="325"
        FontWeight="Regular"
        Foreground="Orange"
        TextWrapping="Wrap"
        LetterSpacing="0.3"
        LineSpacing="4"
        Text="Stay informed and cautious."/>
</StackPanel>
</StackPanel>

</Grid>

```

News.axaml.cs:

```

using System;
using System.Diagnostics;
using System.IO;
using System.Net.Http;
using System.Text.Json;
using System.Threading.Tasks;
using AutoArbitrage_MVVM.ViewModels;
using Avalonia;
using Avalonia.Controls;
using Avalonia.Controls.Primitives;
using Avalonia.Controls.Shapes;
using Avalonia.Input;
using Avalonia.Interactivity;
using Avalonia.Layout;
using Avalonia.Markup.Xaml;
using Avalonia.Media;
using Avalonia.Media.Imaging;
using Avalonia.Threading;

namespace AutoArbitrage_MVVM.Views;

2 references
public partial class News : UserControl
{
    0 references
    public News()
    {
        InitializeComponent();
        DataContext = new NewsViewModel();
        Dispatcher.UIThread.InvokeAsync(() => RunPython());
    }

    1 reference
    private async Task RunPython()
    {
        // Path to the Python executable
        string pythonPath = @"D:\JetBrains Rider\Projects\AutoArbitrage_MVVM\venv\Scripts\python.exe";
        // Path to your Python script
        string scriptPath = @"D:\JetBrains Rider\Projects\AutoArbitrage_MVVM\AutoArbitrage_MVVM\Assets\Model.py";

        try
        {
            // Create a new process start info
            ProcessStartInfo start = new ProcessStartInfo
            {
                FileName = pythonPath,
                Arguments = $"\"{scriptPath}\"",
                RedirectStandardOutput = true,
                RedirectStandardError = true,
                UseShellExecute = false,
                CreateNoWindow = true
            };

            // Start the process
            using (Process process = Process.Start(start))
            {
                // Read the output
                string output = await process.StandardOutput.ReadToEndAsync();
                string error = await process.StandardError.ReadToEndAsync();
                process.WaitForExit();

                // Print the output
                Console.WriteLine("Output from Python script:");
                Console.WriteLine(output);

                // Check for errors
                if (!string.IsNullOrEmpty(error))
                {
                    Console.WriteLine("Error occurred:");
                    Console.WriteLine(error);
                }

                // Process the JSON output
                if (!string.IsNullOrEmpty(output))
                {
                    var jsonResponse = JsonSerializer.Deserialize<JsonElement>(output);

                    // Display news in the ListBox
                    await DisplayNews(jsonResponse);
                }
            }
        }
    }
}

```

```

    }
}
catch (Exception ex)
{
    Console.WriteLine("An error occurred while running the Python script:");
    Console.WriteLine(ex.Message);
}
}

// reference
private async Task DisplayNews(JsonElement jsonResponse)
{
    int newsLimit = 50; // Limit to 50 news items
    int count = 0;

    try
    {
        // Assuming the JSON is a list of articles
        foreach (var article in jsonResponse.EnumerateArray())
        {
            if (count >= newsLimit)
                break;

            string url = article.GetProperty("url").GetString();
            string title = article.GetProperty("title").GetString();
            string description = article.GetProperty("description").GetString();
            string img = article.GetProperty("thumbnail").GetString();
            string createdAt = article.GetProperty("createdAt").GetString();
            string sentiment = article.GetProperty("sentiment").GetString();
            decimal score = article.GetProperty("score").GetDecimal();

            DateTime dateTime;
            string formattedDate = createdAt; // Default to the raw string in case of parsing failure
            if (DateTime.TryParse(createdAt, out dateTime))
            {
                string daySuffix = GetDaySuffix(dateTime.Day);
                formattedDate = dateTime.ToString("ddd, d'{'daySuffix}' MMMM yyyy, HH:mm");
            }

            var listBoxItem = new ListBoxItem
            {
                Width = 1020,
                Background = new SolidColorBrush(Color.Parse("#808080")),
                Margin = new Thickness(0, 15, 0, 0),
                CornerRadius = new CornerRadius(15),
            };

            var grid = new Grid();
            grid.ColumnDefinitions.Add(new ColumnDefinition(new GridLength(500)));
            grid.ColumnDefinitions.Add(new ColumnDefinition(GridLength.Auto));

            var stackPanel = new StackPanel { Margin = new Thickness(10, 0, 0, 0) };

            var titleTextBlock = new TextBlock
            {
                Foreground = Brushes.White,
                FontSize = 18,
                TextWrapping = TextWrapping.Wrap,
                Width = 500,
                FontWeight = FontWeight.Bold,
                HorizontalAlignment = HorizontalAlignment.Left,
                Text = title
            };

            var descriptionTextBlock = new TextBlock
            {
                Foreground = Brushes.Gray,
                Margin = new Thickness(0, 10, 0, 0),
                FontSize = 11,
                Width = 400,
                TextWrapping = TextWrapping.Wrap,
                HorizontalAlignment = HorizontalAlignment.Left,
                Text = description
            };

```



```

var sentimentTextBlock = new TextBlock
{
    FontWeight = FontWeight.Bold,
    Foreground = Brushes.White,
    FontSize = 11,
    Margin = new Thickness(0, 10, 0, 0),
    Text = $"{char.ToUpper(sentiment[0])} + sentiment[1..].ToLower()"}
};

if (sentiment.ToLower().Equals("negative"))
{
    sentimentTextBlock.Foreground = Brushes.Red;
}
else if (sentiment.ToLower().Equals("positive"))
{
    sentimentTextBlock.Foreground = Brushes.Green;
}
else if (sentiment.ToLower().Equals("neutral"))
{
    sentimentTextBlock.Foreground = Brushes.Orange;
}
else
{
    sentimentTextBlock.Foreground = Brushes.White;
}

// Add elements to the stack panel
stackPanel.Children.Add(titleTextBlock);
stackPanel.Children.Add(descriptionTextBlock);
stackPanel.Children.Add(sentimentTextBlock);

// Add the new grid for the accuracy bar below the sentiment
var accuracyGrid = new Grid();

var backgroundBar = new Rectangle
{
    Fill = new SolidColorBrush(Color.Parse("#323949")),
    Width = 300,
    Height = 20,
    HorizontalAlignment = HorizontalAlignment.Left,
    RadiusX = 10,
    RadiusY = 10,
    Margin = new Thickness(-5, 4, 0, 0),
};

double barFill = (double) score * 300;

var accBar = new Rectangle
{
    Fill = new LinearGradientBrush
    {
        GradientStops = new GradientStops
        {
            new GradientStop { Offset = 0.1, Color = Color.Parse("#173055") },
            new GradientStop { Offset = 1, Color = Color.Parse("#3C5780") }
        }
    },
    Width = barFill,
    Height = 20,
    HorizontalAlignment = HorizontalAlignment.Left,
    RadiusX = 10,
    RadiusY = 10,
    Margin = new Thickness(-5, 4, 0, 0),
};

var zeroTextBlock = new TextBlock
{
    Foreground = Brushes.White,
    Margin = new Thickness(1, 8, 0, 0),
    Text = "0%",
    FontSize = 10,
    FontWeight = FontWeight.SemiBold
};

var hundredTextBlock = new TextBlock
{
    Foreground = Brushes.White,
    Margin = new Thickness(265, 8, 0, 0),
};

```



```

    }
    },
    Width = barFill,
    Height = 20,
    HorizontalAlignment = HorizontalAlignment.Left,
    RadiusX = 10,
    RadiusY = 10,
    Margin = new Thickness(-5, 4, 0, 0),
};

var zeroTextBlock = new TextBlock
{
    Foreground = Brushes.White,
    Margin = new Thickness(1, 8, 0, 0),
    Text = "0%",
    FontSize = 10,
    FontWeight = FontWeight.SemiBold
};

var hundredTextBlock = new TextBlock
{
    Foreground = Brushes.White,
    Margin = new Thickness(265, 8, 0, 0),
    Text = "100%",
    FontSize = 10,
    FontWeight = FontWeight.SemiBold
};

var accuracyTextBlock = new TextBlock
{
    Foreground = Brushes.White,
    Margin = new Thickness(113, 8, 0, 0),
    Text = "Accuracy",
    FontSize = 10,
    FontWeight = FontWeight.SemiBold
};

// Add elements to the accuracy grid
accuracyGrid.Children.Add(backgroundBar);
accuracyGrid.Children.Add(accBar);
accuracyGrid.Children.Add(zeroTextBlock);
accuracyGrid.Children.Add(hundredTextBlock);
accuracyGrid.Children.Add(accuracyTextBlock);

var dateTextBlock = new TextBlock
{
    Foreground = Brushes.White,
    FontSize = 11,
    Margin = new Thickness(0, 10, 0, 0),
    Text = formattedDate
};

stackPanel.Children.Add(accuracyGrid);
stackPanel.Children.Add(dateTextBlock);

Grid.SetColumn(stackPanel, 0);
grid.Children.Add(stackPanel);

// Download image from URL and set it to the Image control
var newsImage = new Image
{
    Width = 140,
    Height = 140,
    HorizontalAlignment = HorizontalAlignment.Center,
    VerticalAlignment = VerticalAlignment.Center,
    Margin = new Thickness(175, 0, 0, 0)
};

// Download image in a background thread to avoid blocking the UI
var imageBytes = await DownloadImageAsync(img);
if (imageBytes != null)
{
    // Ensure this UI update happens on the UI thread
    await Dispatcher.UIThread.InvokeAsync(() =>
    {
        using (var stream = new MemoryStream(imageBytes))
        {

```

```

    },

    // Download image in a background thread to avoid blocking the UI
    var imageBytes = await DownloadImageAsync(img);
    if (imageBytes != null)
    {
        // Ensure this UI update happens on the UI thread
        await Dispatcher.UIThread.InvokeAsync(() =>
        {
            using (var stream = new MemoryStream(imageBytes))
            {
                newsImage.Source = new Bitmap(stream);
            }
        });
    }

    Grid.SetColumn(newsImage, 1);
    grid.Children.Add(newsImage);

    listBoxItem.Content = grid;

    listBoxItem.PointerReleased += async (s, e) =>
    {
        // Open the URL in the default web browser
        System.Diagnostics.Process.Start(new System.Diagnostics.ProcessStartInfo
        {
            FileName = url,
            UseShellExecute = true
        });
    });

    // Add the ListBoxItem to the ListBox
    await Dispatcher.UIThread.InvokeAsync(() => { NewsList.Items.Add(listBoxItem); });
    count++;
}

}

catch (Exception ex)
{
    Console.WriteLine($"Error displaying news: {ex.Message}");
}

}

// New method to download image bytes
// reference
private async Task<byte[]> DownloadImageAsync(string imageUrl)
{
    try
    {
        using (var httpClient = new HttpClient())
        {
            return await httpClient.GetByteArrayAsync(imageUrl);
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Failed to load image from {imageUrl}: {ex.Message}");
        return null;
    }
}

// reference
private string GetDaySuffix(int day)
{
    if (day >= 11 && day <= 13)
    {
        return "th";
    }

    switch (day % 10)
    {
        case 1: return "st";
        case 2: return "nd";
        case 3: return "rd";
        default: return "th";
    }
}

```

Profile views:

Profile.axaml:

```
<UserControl xmlns="https://github.com/avaloniaui"
              xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
              xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
              xmlns:viewModels="clr-namespace:AutoShiraga_POWM.Views.Models;assembly=AutoShiraga_POWM.Views.Models"
              mc:Ignorable="d" d:DesignWidth="1888" d:DesignHeight="621"
              Width="1888" Height="621"
              x:Class="AutoShiraga_POWM.Views.Profile"
              x:DataType="viewModels:ProfileViewModel"
              Background="#121528">
    <Grid VerticalAlignment="Top">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="*"/>
            <RowDefinition Height="*"/>
        </Grid.RowDefinitions>
        <StackPanel Grid.Row="0" Orientation="Horizontal" HorizontalAlignment="Left">
            <Image Source="/Assets/user-square.png"
                  Height="250.5" Width="250.5" Margin="39,18,0,0"/>
            <StackPanel Orientation="Vertical" Margin="22.5,0,0,0" VerticalAlignment="Center">
                <TextBlock Name="FullName" Foreground="FFFFFF" FontSize="61.0" Text="{Binding Name}" FontWeight="SemiBold" Margin="0,170.75,0,0" />
                <TextBlock Foreground="Orange" FontSize="18" Text="{Binding Joined}" />
            </StackPanel>
        </StackPanel>
        <Grid Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*"/>
                <ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>
            <StackPanel Grid.Column="0" Margin="0,35,0,0">
                <StackPanel Orientation="Horizontal">
                    <TextBlock Foreground="Orange" FontSize="22" Text="Email Address: " Margin="39,39,0,0"/>
                    <TextBlock Name="Email" Foreground="FFFFFF" FontWeight="Light" FontSize="18" Width="300" TextWrapping="Wrap" TextTrimming="CharacterEllipsis" VerticalAlignment="Center" Text="{Binding DisplayEmail}" Margin="0,39,0,0" Opacity="1"/>
                </StackPanel>
                <StackPanel Orientation="Horizontal">
                    <TextBlock Foreground="Orange" FontSize="22" Text="Phone Number: " Margin="39,39,0,0"/>
                    <TextBlock Name="Phone" Foreground="FFFFFF" FontSize="18" Width="300" TextWrapping="Wrap" TextTrimming="CharacterEllipsis" VerticalAlignment="Center" Text="{Binding Phone}" Margin="0,39,0,0" Opacity="1"/>
                </StackPanel>
                <StackPanel Orientation="Horizontal">
                    <TextBlock Foreground="Orange" FontSize="22" Text="Binance API Key: " Margin="39,39,0,0"/>
                    <TextBlock Name="BinanceKey" Foreground="FFFFFF" FontSize="18" Width="300" TextWrapping="Wrap" TextTrimming="CharacterEllipsis" VerticalAlignment="Center" Text="{Binding BinanceKey}" Margin="0,39,0,0" Opacity="1"/>
                </StackPanel>
                <StackPanel Orientation="Horizontal">
                    <TextBlock Foreground="Orange" FontSize="22" Text="Bybit API Key: " Margin="39,39,0,0"/>
                    <TextBlock Name="BybitKey" Foreground="FFFFFF" FontSize="18" Width="300" TextWrapping="Wrap" TextTrimming="CharacterEllipsis" VerticalAlignment="Center" Text="{Binding BybitKey}" Margin="0,39,0,0" Opacity="1"/>
                </StackPanel>
            </StackPanel>
            <StackPanel Grid.Column="1" Margin="0,35,0,0">
                <StackPanel Orientation="Horizontal">
                    <TextBox Text="{Binding DisplayEmail, Mode=TwoWay}" Name="EmailBox" Height="32" FontSize="14" Width="250" Margin="0,39,0,0" VerticalContentAlignment="Top" Opacity="0" Watermark="{Binding DisplayEmail}"/>
                </StackPanel>
                <StackPanel Orientation="Horizontal">
                    <TextBox Text="{Binding Phone, Mode=TwoWay}" Name="PhoneBox" Height="32" FontSize="14" Width="250" Margin="0,39,0,0" VerticalContentAlignment="Top" Opacity="0" Watermark="{Binding Phone}"/>
                </StackPanel>
                <StackPanel Orientation="Horizontal">
                    <TextBox Text="{Binding BinanceKey, Mode=TwoWay}" Name="BinanceKeyBox" Height="32" FontSize="14" Width="250" Margin="0,39,0,0" VerticalContentAlignment="Top" Opacity="0" Watermark="{Binding BinanceKey}"/>
                </StackPanel>
                <StackPanel Orientation="Horizontal">
                    <TextBox Text="{Binding BybitKey, Mode=TwoWay}" Name="BybitKeyBox" Height="32" FontSize="14" Width="250" Margin="0,39,0,0" VerticalContentAlignment="Top" Opacity="0" Watermark="{Binding BybitKey}"/>
                </StackPanel>
            </StackPanel>
        </Grid>
        <StackPanel Grid.Column="2" Margin="0,35,0,0">
            <StackPanel Orientation="Horizontal">
                <TextBlock Name="NameLabel" FontSize="22" Foreground="Orange" Text="Name: " Margin="39,39,0,0" Opacity="0"/>
            </StackPanel>
        </StackPanel>
    </Grid>
</UserControl>
```

```

<StackPanel Grid.Column="2" Margin="0,35,0,0">
    <StackPanel Orientation="Horizontal">
        <TextBlock Name="NameLabel" FontSize="22" Foreground="Orange" Text="Name: " Margin="39,39,0,0" Opacity="0"/>
    </StackPanel>

    <StackPanel Orientation="Horizontal">
        <TextBlock Name="BinSecretLabel" FontSize="22" Foreground="Orange" Text="Binance Secret: " Margin="39,39,0,0" Opacity="0"/>
    </StackPanel>

    <StackPanel Orientation="Horizontal">
        <TextBlock Name="BytSecretLabel" FontSize="22" Foreground="Orange" Text="Bybit Secret: " Margin="39,39,0,0" Opacity="0"/>
    </StackPanel>

</StackPanel>

<StackPanel Grid.Column="3" Margin="0,35,0,0">
    <StackPanel Orientation="Horizontal">
        <TextBox Text="{Binding Name, Mode=TwoWay}" Name="NameBox" Height="32" FontSize="14" Width="250" Margin="0,39,0,0" VerticalContentAlignment="Top" Opacity="0" Watermark="{Binding Name}"</>/TextBox>
    </StackPanel>

    <StackPanel Orientation="Horizontal">
        <TextBox Text="{Binding BinanceSecret, Mode=TwoWay}" Name="BinanceSecretBox" Height="32" FontSize="14" Width="250" Margin="0,39,0,0" VerticalContentAlignment="Top" Opacity="0" Watermark="{Binding BinanceSecret}"</>/TextBox>
    </StackPanel>

    <StackPanel Orientation="Horizontal">
        <TextBox Text="{Binding BybitSecret, Mode=TwoWay}" Name="BybitSecretBox" Height="32" FontSize="14" Width="250" Margin="0,39,0,0" VerticalContentAlignment="Top" Opacity="0" Watermark="{Binding BybitSecret}"</>/TextBox>
    </StackPanel>

</StackPanel>

</Grid>

<Grid Grid.Row="1" Grid.Column="1" >
    <Button
        Name="EditInfo"
        CornerRadius="12"
        Height="77.5" Width="95"
        Background="White"
        Foreground="Orange"
        HorizontalContentAlignment="Center"
        VerticalContentAlignment="Center"
        FontSize="18"
        Margin="424.5,385,0,0"
        Opacity="1"
        Click="EditInfo_OnClick"
        Content="Edit Info"
        ZIndex="1"/>

    <Button
        Name="SaveInfo"
        CornerRadius="12"
        Height="77.5" Width="95"
        Background="Orange"
        Foreground="White"
        HorizontalContentAlignment="Center"
        VerticalContentAlignment="Center"
        FontSize="18"
        Margin="424.5,385,0,0"
        Opacity="0"
        Click="SaveInfo_OnClick"
        Command="{Binding SaveInfo}"
        Content="Save Info"
        ZIndex="0"/>

</Grid>

</Grid>

</UserControl>

```

Profile.axaml.cs:

```

using System.Linq;
using AutoArbitrage_MVVM.ViewModels;
using Avalonia;
using Avalonia.Controls;
using Avalonia.Interactivity;
using Avalonia.Markup.Xaml;
using Avalonia.Media;
using Avalonia.VisualTree;

```

```

namespace AutoArbitrage_MVVM.Views;

```

```

3 references
public partial class Profile : UserControl
{

```

```

1 reference
public Profile()
{
    InitializeComponent();
    DataContext = new ProfileViewModel();
}

```

```

0 references
private void EditInfo_OnClick(object? sender, RoutedEventArgs e)
{
    foreach (var textBox in this.GetVisualDescendants().OfType<TextBox>())
    {
        textBox.Opacity = 1;
    }

    NameLabel.Opacity = 1;
    BiSecLabel.Opacity = 1;
    BySecLabel.Opacity = 1;

    Email.Opacity = 0;
    Phone.Opacity = 0;
    BinanceKey.Opacity = 0;
    BybitKey.Opacity = 0;

    SaveInfo.ZIndex = 1;
    SaveInfo.Opacity = 1;
    EditInfo.ZIndex = 0;
    EditInfo.Opacity = 0;
}

```

```

0 references
private void SaveInfo_OnClick(object? sender, RoutedEventArgs e)
{
    foreach (var textBox in this.GetVisualDescendants().OfType<TextBox>())
    {
        textBox.Opacity = 0;
    }

    NameLabel.Opacity = 0;
    BiSecLabel.Opacity = 0;
    BySecLabel.Opacity = 0;

    Email.Opacity = 1;
    Phone.Opacity = 1;
    BinanceKey.Opacity = 1;
    BybitKey.Opacity = 1;

    SaveInfo.ZIndex = 0;
    SaveInfo.Opacity = 0;
    EditInfo.ZIndex = 1;
    EditInfo.Opacity = 1;
}
}

```

Signup:

```
<Window xmlns="https://github.com/axloriniui"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:viewModels="clr-namespace:AutoArbitrage_WVM.ViewModels"
mc:Ignorable="d" d:DesignWidth="1080" d:DesignHeight="768"
Width="1080" Height="768"
x:Class="AutoArbitrage_WVM.Views.Signup"
Title="Signup"
WindowStartupLocation="CenterOwner"
ExtendClientAreaToDecorationsHint="True"
ExtendClientAreaChromeHints="NoChrome"
ExtendClientAreaTitleBarHeightHint="0"
x:DataType="viewModels:SignupViewModel">

<Grid Background="#121528">

    <Grid.RowDefinitions>42,75,<*/Grid.RowDefinitions>

    <StackPanel Grid.Row="0" Grid.Column="0" Orientation="Horizontal" Background="#888814">
        <TextBlock
            Text="AutoArbitrage"
            FontSize="24"
            FontWeight="Bold"
            Foreground="White"
            VerticalAlignment="Center"
            Margin="11,25,0,813,0"/>

        <Button
            Name="Min"
            Click="Min_OnClick"
            Margin="2,625,0"
            Height="18,75"
            Width="18,75"
            CornerRadius="3,75"
            Background="DarkGreen"/>

        <Button
            Name="Stack"
            Click="Stack_OnClick"
            Margin="2,625,0"
            Height="18,75"
            Width="18,75"
            CornerRadius="3,75"
            Background="Orange"/>

        <Button
            Name="Close"
            Click="Close_OnClick"
            Margin="2,625,0"
            Height="18,75"
            Width="18,75"
            CornerRadius="3,75"
            Background="DarkRed"/>
    </StackPanel>

    <Grid Grid.Row="1">
        <StackPanel Margin="20">
            <TextBlock
                Text="Connect To"
                FontSize="70"
                FontWeight="Bold"
                Foreground="White"
                VerticalAlignment="Center"
                HorizontalAlignment="Left"
                Margin="20,0,0,0"/>

            <TextBlock
                Text="Exchanges"
                FontSize="70"
                FontWeight="Bold"
                Foreground="White"
                VerticalAlignment="Center"
                HorizontalAlignment="Left"
                Margin="20,-10,0,0"/>

            <TextBlock
                Text="Capitalize on arbitrage opportunities. Stay ahead of the curve. Grow your portfolio."
            </TextBlock>
        </StackPanel>
    </Grid>
</Grid>
```

```

        Foreground="White"
        VerticalAlignment="Center"
        HorizontalAlignment="Left"
        Margin="20,30,0,0"
        Width="500"
        TextWrapping="Wrap"/>

<TextBlock
    Text="Trade Crypto Futures with AutoArbitrage"
    FontSize="18"
    Foreground="White"
    VerticalAlignment="Center"
    HorizontalAlignment="Left"
    Margin="20,30,0,0"
    Width="500"
    TextWrapping="Wrap"/>

<Border
    Width="540"
    Height="40"
    Background="#4C6FC7"
    Margin="-200,200,0,0"
    HorizontalAlignment="Left"
    CornerRadius="100">

    <Border.RenderTransform>
        <RotateTransform Angle="-45"/></RotateTransform>
    </Border.RenderTransform>
</Border>

<Border
    Width="500"
    Height="40"
    Background="#10265F"
    Margin="-200,80,0,0"
    HorizontalAlignment="Left"
    CornerRadius="100">

    <Border.RenderTransform>
        <RotateTransform Angle="-45"/></RotateTransform>
    </Border.RenderTransform>
</Border>

<Border
    Width="900"
    Height="40"
    Background="#839EE3"
    Margin="-340,0,0,0"
    HorizontalAlignment="Left"
    CornerRadius="100">

    <Border.RenderTransform>
        <RotateTransform Angle="-45"/></RotateTransform>
    </Border.RenderTransform>
</Border>
</StackPanel>

<Border
    Width="400"
    Height="650"
    HorizontalAlignment="Right"
    VerticalAlignment="Bottom"
    Margin="0,0,50,50"
    CornerRadius="20"
    Background="#000C21">

    <StackPanel Margin="20">
        <TextBlock
            Text="Sign Up"
            FontSize="40"
            FontWeight="Bold"
            Foreground="White"
            HorizontalAlignment="Center"/>

        <TextBlock
            Text="Email Address"
            Foreground="#D9D9D9"
            Margin="0,20,0,0"/>
    </StackPanel>
</Border>

```

```

<TextBox
    Name="Email"
    Text="{Binding Email, Mode=TwoWay}"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="Email_OnGotFocus"
    LostFocus="Email_OnLostFocus"/>

<TextBlock
    Text="Full Name"
    Foreground="#D9D9D9"
    Margin="0,20,0,0"/>

<TextBox
    Name="FullName"
    Text="{Binding FullName, Mode=TwoWay}"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="FullName_OnGotFocus"
    LostFocus="FullName_OnLostFocus"/>

<TextBlock
    Text="Phone Number"
    Foreground="#D9D9D9"
    Margin="0,20,0,0"/>

<TextBox
    Name="PhoneNumber"
    Text="{Binding PhoneNumber, Mode=TwoWay}"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="PhoneNumber_OnGotFocus"
    LostFocus="PhoneNumber_OnLostFocus"/>

<TextBlock
    Text="Password"
    Foreground="#D9D9D9"
    Margin="0,20,0,0"/>

<TextBox
    Name="Password"
    Text="{Binding Password, Mode=TwoWay}"
    PasswordChar="*"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="Password_OnGotFocus"
    LostFocus="Password_OnLostFocus"/>

<TextBlock
    Text="Confirm Password"
    Foreground="#D9D9D9"
    Margin="0,20,0,0"/>

<TextBox
    Name="ConfirmPassword"
    Text="{Binding ConfirmPassword, Mode=TwoWay}"
    PasswordChar="*"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="ConfirmPassword_OnGotFocus"
    LostFocus="ConfirmPassword_OnLostFocus"/>

<Button
    Name="SignUpButton"
    Content="Sign Up"
    Command="{Binding SignUp}"
    Background="#173055"
    Margin="0,40,0,0"
    Width="120"

```



```

<TextBox
    Name="Password"
    Text="{Binding Password, Mode=TwoWay}"
    PasswordChar="*"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="Password_OnGotFocus"
    LostFocus="Password_OnLostFocus"/>

<TextBlock
    Text="Confirm Password"
    Foreground="#D9D9D9"
    Margin="0,20,0,0"/>

<TextBox
    Name="ConfirmPassword"
    Text="{Binding ConfirmPassword, Mode=TwoWay}"
    PasswordChar="*"
    Height="40"
    Foreground="Black"
    Background="#D9D9D9"
    FontSize="20"
    GotFocus="ConfirmPassword_OnGotFocus"
    LostFocus="ConfirmPassword_OnLostFocus"/>

<Button
    Name="SignUpButton"
    Content="Sign Up"
    Command="{Binding SignUp}"
    Background="#173055"
    Margin="0,40,0,0"
    Width="120"
    FontSize="20"
    Padding="10"
    Foreground="White"
    CornerRadius="10"
    HorizontalAlignment="Center"
    HorizontalContentAlignment="Center"
/>

<Button
    Name="ToLoginButton"
    Content="Already have an account? Login here."
    Command="{Binding ToLogin}"
    Background="#800C21"
    Margin="0,10,0,0"
    FontSize="16"
    Foreground="White"
    HorizontalAlignment="Center"/>

<TextBlock
    Name="ErrorMessage"
    Text="{Binding ErrorMessage}"
    Margin="0,15,0,0"
    FontSize="14"
    FontWeight="SemiBold"
    Foreground="DarkRed"
    HorizontalAlignment="Center"
    TextWrapping="Wrap"/>
</StackPanel>
</Border>
</Grid>
</Grid>
</Window>

```

Signup.xaml.cs:

```

using AutoArbitrage_MVVM.ViewModels;
using Avalonia;
using Avalonia.Controls;
using Avalonia.Input;
using Avalonia.Interactivity;
using Avalonia.Markup.Xaml;
using Avalonia.Media;

namespace AutoArbitrage_MVVM.Views;

3 references
public partial class SignUp : Window
{
    1 reference
    public SignUp()
    {
        InitializeComponent();
        DataContext = new SignUpViewModel(this);
    }

    0 references
    private void Min_OnClick(object? sender, RoutedEventArgs e)
    {
        this.WindowState = WindowState.Minimized;
    }

    0 references
    private void Stack_OnClick(object? sender, RoutedEventArgs e)
    {
        if (this.WindowState == WindowState.Maximized)
        {
            this.WindowState = WindowState.Normal;
        }
        else
        {
            this.WindowState = WindowState.Maximized;
        }
    }

    0 references
    private void Close_OnClick(object? sender, RoutedEventArgs e)
    {
        Close();
    }

    0 references
    private void Email_OnGotFocus(object? sender, GotFocusEventArgs e)
    {
        Email.Foreground = Brushes.White;
    }

    0 references
    private void Email_OnLostFocus(object? sender, RoutedEventArgs e)
    {
        Email.Foreground = Brushes.Black;
    }

    0 references
    private void Password_OnGotFocus(object? sender, GotFocusEventArgs e)
    {
        Password.Foreground = Brushes.White;
    }

    0 references
    private void Password_OnLostFocus(object? sender, RoutedEventArgs e)
    {
        Password.Foreground = Brushes.Black;
    }

    0 references
    private void ConfirmPassword_OnGotFocus(object? sender, GotFocusEventArgs e)
    {
        ConfirmPassword.Foreground = Brushes.White;
    }

    0 references
    private void ConfirmPassword_OnLostFocus(object? sender, RoutedEventArgs e)
    {
        ConfirmPassword.Foreground = Brushes.Black;
    }
}

```

```
0 references
private void FullName_OnGotFocus(object? sender, GotFocusEventArgs e)
{
    FullName.Foreground = Brushes.White;
}

0 references
private void FullName_OnLostFocus(object? sender, RoutedEventArgs e)
{
    FullName.Foreground = Brushes.Black;
}

0 references
private void PhoneNumber_OnGotFocus(object? sender, GotFocusEventArgs e)
{
    PhoneNumber.Foreground = Brushes.White;
}

0 references
private void PhoneNumber_OnLostFocus(object? sender, RoutedEventArgs e)
{
    PhoneNumber.Foreground = Brushes.Black;
}
}
```

Trade views:

Trade.axaml:

```

<UserControl xmlns="https://github.com/avaloniaui"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:viewModels="clr-namespace:AutoArbitrage_MVVM.ViewModels"
mc:Ignorable="d" d:DesignWidth="1080" d:DesignHeight="621"
Width="1080" Height="621"
x:Class="AutoArbitrage_MVVM.Views.Trade"
x:DataType="viewModels:TradeViewModel">

    <Grid Background="White">
        <Grid.RowDefinitions>135,486</Grid.RowDefinitions>
        <Grid.ColumnDefinitions>375.75,375.75,*</Grid.ColumnDefinitions>

        <StackPanel Grid.Row="0" Grid.Column="0" Grid.ColumnSpan="3" Background="#121528">

            <Grid>
                <Grid.ColumnDefinitions>*,*,*</Grid.ColumnDefinitions>

                <Rectangle
                    Grid.Column="0"
                    Grid.ColumnSpan="2"
                    Fill="#0E0E22"
                    Height="123.75"
                    Width="708.25"
                    RadiusX="16.5"
                    RadiusY="16.5"
                    HorizontalAlignment="Left"
                    Margin="21,0,0,0"/>

                <StackPanel Grid.Column="0">
                    <TextBlock
                        Text="Cryptocurrency Perpetuals"
                        Foreground="White"
                        FontWeight="SemiBold"
                        FontSize="18"
                        Margin="49.5,21.375,0,0"/>

                    <ComboBox CornerRadius="6"
                        Foreground="White"
                        Background="#173055"
                        Width="270.75"
                        FontSize="18"
                        Height="45"
                        Margin="42.75,9,0,0"
                        BorderBrush="Transparent"
                        PlaceholderForeground="White"
                        PlaceholderText="Choose Cryptocurrency"
                        Name="CurrencyBox"
                        ItemsSource="{Binding CurrencyOptions}"
                        SelectedItem="{Binding SelectedCurrency, Mode=TwoWay}">

                    </ComboBox>

                </StackPanel>

                <StackPanel Grid.Column="1" Orientation="Horizontal">

                    <Button
                        Content="USDT-M"
                        VerticalContentAlignment="Center"
                        HorizontalContentAlignment="Center"
                        FontSize="18"
                        Foreground="White"
                        Name="USD_Margin"
                        Background="Transparent"
                        CornerRadius="6"
                        Height="30"
                        Margin="0,35,0,0"/>

                    <Button
                        Content="Coin-M"
                        VerticalContentAlignment="Center"
                        HorizontalContentAlignment="Center"
                        FontSize="18"
                        Foreground="White"
                        Name="Coin_Margin"

```

```

        Height="30"
        Margin="0,35,0,0"/>

        <Button
            Content="Coin-M"
            VerticalContentAlignment="Center"
            HorizontalContentAlignment="Center"
            FontSize="18"
            Foreground="White"
            Name="Coin_Margin"
            Background="Transparent"
            CornerRadius="6"
            Height="30"
            Margin="51.75,35,0,0"/>
    </StackPanel>
</Grid>

</StackPanel>

<StackPanel Grid.Row="1" Grid.Column="0" Background="#121528">
    <Grid>
        <Rectangle
            HorizontalAlignment="Left"
            Margin="21,0,0,0"
            Width="345.75"
            Height="463.5"
            Fill="#00E0E2"
            RadiusX="16.5"
            RadiusY="16.5"/>

        <StackPanel>
            <TextBlock
                Foreground="White"
                FontSize="24"
                Margin="42,14,25,0,0"
                FontWeight="SemiBold">Binance Order Book
            </TextBlock>
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="{Binding Price(USDT)}" Foreground="White" FontSize="15" Margin="43.75,24,0,0"/></TextBlock>
                <TextBlock Text="Quantity" Foreground="White" FontSize="15" Margin="98.75,24,0,0"/></TextBlock>
            </StackPanel>

            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition/></ColumnDefinition>
                    <ColumnDefinition/></ColumnDefinition>
                </Grid.ColumnDefinitions>

                <TextBlock Text="{Binding BinanceAskPrice}" Name="BinanceAskPrice" Grid.Column="0" Height="143.25" FontSize="20" Foreground="DarkGreen" Margin="42.5,19,0,0"/></TextBlock>
                <TextBlock Text="{Binding BinanceAskQuantity}" Name="BinanceAskQuantity" Grid.Column="1" Height="143.25" FontSize="20" Foreground="DarkGreen" Margin="42.5,19,0,0"/></TextBlock>
            </Grid>

            <StackPanel Orientation="Horizontal">
                <TextBlock Text="{Binding BinancePrice}" Name="BinanceFuturesPrice" Foreground="{Binding BinancePriceColor}" FontSize="24" FontWeight="SemiBold" Margin="42,0,16.875,0"/></TextBlock>
                <Grid>
                    <Polygon Opacity="{Binding BinanceUpArrowOpacity}" Name="B_Up" Fill="DarkGreen" Points="9.375,0 18.75,18.75 0,18.75" Width="18.75" Height="15" />
                    <Polygon Opacity="{Binding BinanceDownArrowOpacity}" Name="B_Down" Fill="DarkRed" Points="0,0 18.75,0 9.375,18.75" Width="18.75" Height="15" />
                </Grid>
                <TextBlock Text="{Binding BinancePreviousPrice}" Name="BinanceLastPrice" Foreground="FFC7C7C" FontSize="11.25" FontWeight="SemiBold" Margin="16.875,9,0,0"/></TextBlock>
            </StackPanel>

            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition/></ColumnDefinition>
                    <ColumnDefinition/></ColumnDefinition>
                </Grid.ColumnDefinitions>

                <TextBlock Text="{Binding BinanceBidPrice}" Name="BinanceBidPrice" Grid.Column="0" Height="143.25" FontSize="20" Foreground="DarkRed" Margin="42.5,19,0,0"/></TextBlock>
                <TextBlock Text="{Binding BinanceBidQuantity}" Name="BinanceBidQuantity" Grid.Column="1" Height="143.25" FontSize="20" Foreground="DarkRed" Margin="42.5,19,0,0"/></TextBlock>
            </Grid>
        </StackPanel>
    </Grid>
</StackPanel>

```

```

<StackPanel Grid.Row="1" Grid.Column="1" Background="#121528">
    <Grid>
        <Rectangle
            HorizontalAlignment="Left"
            Margin="5,0,0,0"
            Width="348.75" Height="463.5"
            Fill="#0E0E22"
            RadiusX="16.5" RadiusY="16.5"/>
        </Rectangle>
        <StackPanel>
            <TextBlock
                Foreground="White"
                FontSize="24"
                Margin="27.5,14.25,0,0"
                FontWeight="SemiBold">Bybit Order Book
            </TextBlock>
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="Price(USDT)" Foreground="White" FontSize="15" Margin="27.75,24,0,0"/></TextBlock>
                <TextBlock Text="Quantity" Foreground="White" FontSize="15" Margin="98.75,24,0,0"/></TextBlock>
            </StackPanel>
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition/><ColumnDefinition>
                    <ColumnDefinition/><ColumnDefinition>
                </Grid.ColumnDefinitions>
                <TextBlock Text="{Binding BybitAskPrice}" Name="BybitAskPrice" Grid.Column="0" Height="143.25" FontSize="20" Foreground="DarkGreen" Margin="26.5,10,0,0"/></TextBlock>
                <TextBlock Text="{Binding BinanceAskQuantity}" Name="BybitAskQuantity" Grid.Column="1" Height="143.25" FontSize="20" Foreground="DarkGreen" Margin="26.5,10,0,0"/></TextBlock>
            </Grid>
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="{Binding BybitPrice}" Name="BybitFuturesPrice" Foreground="{Binding BybitPriceColor}" FontSize="24" FontWeight="SemiBold" Margin="26,0,16.875,0"/></TextBlock>
                <Grid>
                    <Polygon Opacity="{Binding BybitUpArrowOpacity}" Name="By_Up" Fill="DarkGreen" Points="9.375,0 18.75,18.75 0,18.75" Width="18.75" Height="15" />
                    <Polygon Opacity="{Binding BybitDownArrowOpacity}" Name="By_Down" Fill="DarkRed" Points="0,0 18.75,0 9.375,18.75" Width="18.75" Height="15" />
                </Grid>
                <TextBlock Text="{Binding BybitPreviousPrice}" Name="BybitLastPrice" Foreground="#7C7C7C" FontSize="11.25" FontWeight="SemiBold" Margin="16.875,9,0,0"/></TextBlock>
            </StackPanel>
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition/><ColumnDefinition>
                    <ColumnDefinition/><ColumnDefinition>
                </Grid.ColumnDefinitions>
                <TextBlock Text="{Binding BybitBidPrice}" Name="BybitBidPrice" Grid.Column="0" Height="143.25" FontSize="20" Foreground="DarkRed" Margin="26.5,19,0,0"/></TextBlock>
                <TextBlock Text="{Binding BybitBidQuantity}" Name="BybitBidQuantity" Grid.Column="1" Height="143.25" FontSize="20" Foreground="DarkRed" Margin="26.5,19,0,0"/></TextBlock>
            </Grid>
        </StackPanel>
    </Grid>
</StackPanel>

<StackPanel Grid.Row="0" Grid.RowSpan="2" Grid.Column="2" Background="#121528">
    <Grid>
        <Rectangle HorizontalAlignment="Center" VerticalAlignment="Top"
            RadiusX="16.5" RadiusY="16.5" Height="210" Width="316.5"
            Fill="#0E0E22" Margin="0,0,34,0"/>
        <Rectangle HorizontalAlignment="Center" VerticalAlignment="Bottom"
            RadiusX="16.5" RadiusY="16.5" Height="377" Width="316.5"
            Fill="#0E0E22" Margin="0,221.5,34,0"/>
        <Border Name="LockBox" HorizontalAlignment="Center" VerticalAlignment="Bottom"
            CornerRadius="16.5" Height="377" Width="316.5"
            Background="#05051A" Margin="0,221.5,34,0" ZIndex="1" Opacity="15">
            <StackPanel HorizontalAlignment="Center">
                <Image Name="Lock" Source="/Assets/Lock.png"
                    Height="120" ZIndex="3" Margin="0,90,0,0"/></Image>
                <TextBlock
                    Name="LockText"
                    Text="Please enter your API Keys and Secrets in the Profile page to start trading."
                    Width="280"
                    Height="200"
                    Foreground="White"

```

```

        <TextBlock
            Name="LockText"
            Text="Please enter your API Keys and Secrets in the Profile page to start trading."
            Width="280"
            Height="200"
            Foreground="White"
            FontWeight="Regular"
            FontSize="18"
            TextAlignment="Center"
            TextWrapping="Wrap"
            Margin="0,15,0,0"
            ZIndex="1"
            HorizontalAlignment="Center"/>
    </StackPanel>

</Border>

<StackPanel>

    <TextBlock
        Text="Funding Rates"
        Foreground="White"
        FontWeight="SemiBold"
        FontSize="26"
        Margin="10,21.375,0,0">
    </TextBlock>

    <TextBlock
        Text="{Binding Countdown}"
        Name="Countdown"
        Foreground="Orange"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Light"
        Margin="10,8,0,0">
    </TextBlock>

    <StackPanel Orientation="Horizontal">
        <TextBlock Margin="10,10,0,0" FontSize="20" FontWeight="Regular" Foreground="White">Binance: </TextBlock>
        <TextBlock Text="{Binding BinanceFundingRate}" Margin="10,10,0,0" Name="BinanceFundingRate" FontSize="20" FontWeight="Light" Foreground="White"></TextBlock>
    </StackPanel>

    <StackPanel Orientation="Horizontal">
        <TextBlock
            Text="{Binding BPaymentFrom}"
            Name="BPaymentFrom"
            Foreground="{Binding BPaymentFromColor}"
            VerticalAlignment="Center"
            FontSize="18"
            FontWeight="Bold"
            Margin="10,2,0,0">
        </TextBlock>
        <TextBlock
            Text="{Binding BArrow}"
            Name="BArrow"
            Foreground="White"
            VerticalAlignment="Center"
            FontSize="18"
            FontWeight="Bold"
            Margin="10,2,0,0">
        </TextBlock>
        <TextBlock
            Text="{Binding BPaymentTo}"
            Name="BPaymentTo"
            Foreground="{Binding BPaymentToColor}"
            VerticalAlignment="Center"
            FontSize="18"
            FontWeight="Bold"
            Margin="10,2,0,0">
        </TextBlock>
    </StackPanel>

    <StackPanel Orientation="Horizontal">
        <TextBlock Margin="10,6,0,0" FontSize="20" FontWeight="Regular" Foreground="White">Bybit: </TextBlock>
        <TextBlock Text="{Binding BybitFundingRate}" Margin="10,6,0,0" Name="BybitFundingRate" FontSize="20" FontWeight="Light" Foreground="White"></TextBlock>
    </StackPanel>

```

```

        Name="BArrow"
        Foreground="White"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
    <TextBlock
        Text="{Binding BPaymentTo}"
        Name="BPaymentTo"
        Foreground="{Binding BPaymentToColor}"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
</StackPanel>

<StackPanel Orientation="Horizontal">
    <TextBlock Margin="10,6,0,0" FontSize="20" FontWeight="Regular" Foreground="White">ByBit: </TextBlock>
    <TextBlock Text="{Binding ByBitFundingRate}" Margin="10,6,0,0" Name="ByBitFundingRate" FontSize="20" FontWeight="Light" Foreground="White"></TextBlock>
</StackPanel>

<StackPanel Orientation="Horizontal">
    <TextBlock
        Text="{Binding ByPaymentFrom}"
        Foreground="{Binding ByPaymentFromColor}"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
    <TextBlock
        Text="{Binding ByArrow}"
        Foreground="White"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
    <TextBlock
        Text="{Binding ByPaymentTo}"
        Foreground="{Binding ByPaymentToColor}"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
</StackPanel>

<TextBlock
    Text="Trade"
    Foreground="White"
    FontWeight="SemiBold"
    FontSize="26"
    Margin="10,49,5,0,0">
</TextBlock>

<Label Margin="0,10,0,0" FontSize="12" Foreground="White">Threshold:</Label>
<TextBox Name="Threshold" GotFocus="Threshold_OnGotFocus" LostFocus="Threshold_OnLostFocus" Text="{Binding Threshold, Mode=TwoWay}" Margin="0,0,34,0" Background="White" Height="27.75" Width="275.25"></TextBox>

<Label Margin="0,10,0,0" FontSize="12" Foreground="White">Size:</Label>
<TextBox Name="Size" GotFocus="Size_OnGotFocus" LostFocus="Size_OnLostFocus" Text="{Binding Quantity}" Margin="0,0,34,0" Background="White" Height="27.75" Width="275.25"></TextBox>

<Label Margin="0,10,0,0" FontSize="12" Foreground="White">Frequency (ms):</Label>
<TextBox Name="Frequency" GotFocus="Frequency_OnGotFocus" LostFocus="Frequency_OnLostFocus" Text="{Binding Frequency}" Margin="0,0,34,0" Background="White" Height="27.75" Width="275.25"></TextBox>

<Button HorizontalAlignment="Center" Margin="0,28,34,0" Height="39.75" Width="173.25" Background="Orange" Foreground="#000000"
    HorizontalContentAlignment="Center" VerticalContentAlignment="Center" FontSize="24" FontWeight="Bold" Command="{Binding StartDiscrepancyCheckerAsync}">TRADE
</Button>

<Button HorizontalAlignment="Center" Margin="0,12,34,0" Height="39.75" Width="173.25" Background="DarkRed" Foreground="#000000"
    HorizontalContentAlignment="Center" VerticalContentAlignment="Center" FontSize="24" FontWeight="Bold" Command="{Binding StopDiscrepancyChecker}">STOP
</Button>
</StackPanel>
</Grid>
</StackPanel>

```



```

        Foreground="{Binding BPaymentToColor}"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
</StackPanel>

<StackPanel Orientation="Horizontal">
    <TextBlock Margin="10,6,0,0" FontSize="20" FontWeight="Regular" Foreground="White">ByBit: </TextBlock>
    <TextBlock Text="{Binding ByBitFundingRate}" Margin="10,6,0,0" Name="ByBitFundingRate" FontSize="20" FontWeight="Light" Foreground="White"></TextBlock>
</StackPanel>

<StackPanel Orientation="Horizontal">
    <TextBlock
        Text="{Binding ByPaymentFrom}"
        Foreground="{Binding ByPaymentFromColor}"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
    <TextBlock
        Text="{Binding ByArrow}"
        Foreground="White"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
    <TextBlock
        Text="{Binding ByPaymentTo}"
        Foreground="{Binding ByPaymentToColor}"
        VerticalAlignment="Center"
        FontSize="18"
        FontWeight="Bold"
        Margin="10,2,0,0">
    </TextBlock>
</StackPanel>

<TextBlock
    Text="Trade"
    Foreground="White"
    FontWeight="SemiBold"
    FontSize="26"
    Margin="10,49,5,0,0">
</TextBlock>

<Label Margin="8,10,0,0" FontSize="12" Foreground="White">Threshold</Label>
<TextBox Name="Threshold" GotFocus="Threshold_OnGotFocus" LostFocus="Threshold_OnLostFocus" Text="{Binding Threshold, Mode=TwoWay}" Margin="0,0,34,0" Background="White" Height="27.75" Width="275.25"></TextBox>

<Label Margin="8,10,0,0" FontSize="12" Foreground="White">Size</Label>
<TextBox Name="Size" GotFocus="Size_OnGotFocus" LostFocus="Size_OnLostFocus" Text="{Binding Quantity}" Margin="0,0,34,0" Background="White" Height="27.75" Width="275.25"></TextBox>

<Label Margin="8,10,0,0" FontSize="12" Foreground="White">Frequency (ms)</Label>
<TextBox Name="Frequency" GotFocus="Frequency_OnGotFocus" LostFocus="Frequency_OnLostFocus" Text="{Binding Frequency}" Margin="0,0,34,0" Background="White" Height="27.75" Width="275.25"></TextBox>

<Button HorizontalAlignment="Center" Margin="0,28,34,0" Height="39.75" Width="173.25" Background="Orange" Foreground="#808022"
    HorizontalContentAlignment="Center" VerticalContentAlignment="Center" FontSize="24" FontWeight="Bold" Command="{Binding StartDiscrepancyCheckerAsync}">TRADE
</Button>

<Button HorizontalAlignment="Center" Margin="0,12,34,0" Height="39.75" Width="173.25" Background="DarkRed" Foreground="#808022"
    HorizontalContentAlignment="Center" VerticalContentAlignment="Center" FontSize="24" FontWeight="Bold" Command="{Binding StopDiscrepancyChecker}">STOP
</Button>
</StackPanel>
</Grid>
</StackPanel>
</Grid>
</UserControl>

```

Trade.axaml.cs:

```

using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Threading.Tasks;
using AutoArbitrage_MVVM.Services;
using AutoArbitrage_MVVM.ViewModels;
using Avalonia;
using Avalonia.Controls;
using Avalonia.Input;
using Avalonia.Interactivity;
using Avalonia.Markup.Xaml;
using Avalonia.Media;
using MySql.Data.MySqlClient;

namespace AutoArbitrage_MVVM.Views;

2 references
public partial class Trade : UserControl, INotifyPropertyChanged
{
    private string connectionString = "Server=autoarbitrage.cri2yu04sa9j.ne-central-1.rds.amazonaws.com;Database=userdb;User ID=admin;Password=autoarbitrage12;";

    private string? _email;
    6 references
    public string? Email
    {
        get => _email;
        set => SetProperty(ref _email, value);
    }

    public event PropertyChangedEventHandler? PropertyChanged;

    1 reference
    protected bool SetProperty<T>(ref T backingField, T value, [CallerMemberName] string? propertyName = null)
    {
        if (Equals(backingField, value)) return false;
        backingField = value;
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
        return true;
    }

    0 references
    public Trade()
    {
        InitializeComponent();
        DataContext = new TradeViewModel();

        UserService.Instance.OnEmailChanged += (sender, args) =>
        {
            Email = UserService.Instance.Email;
            Console.WriteLine($"TradeViewModel loaded with email: {Email}");
        };

        Email = UserService.Instance.Email;

        Unlock();
    }

    1 reference
    private async Task Unlock()
    {
        Console.WriteLine("Start Unlock");

        string query = "SELECT binance_key, bybit_key, binance_secret, bybit_secret FROM users WHERE email = @Email LIMIT 1";

        string? bin_key = null;
        string? byb_key = null;
        string? bin_sec = null;
        string? byb_sec = null;

        using (var connection = new MySqlConnection(connectionString))
        {
            await connection.OpenAsync();
            using (var command = new MySqlCommand(query, connection))
            {
                command.Parameters.AddWithValue("@Email", Email);

                try
                {
                    using (var reader = await command.ExecuteReaderAsync())
                    {

```

```

        using (var reader = await command.ExecuteReaderAsync())
        {
            if (await reader.ReadAsync())
            {
                bin_key = reader["binance_key"] as string;
                byb_key = reader["bybit_key"] as string;
                bin_sec = reader["binance_secret"] as string;
                byb_sec = reader["bybit_secret"] as string;

                Console.WriteLine($"Retrieved keys and secrets for email {Email}: Binance Key - {bin_key}, Bybit Key - {byb_key}");
            }
            else
            {
                Console.WriteLine($"No keys found for email {Email}.");
            }
        }
    }
}

catch (Exception ex)
{
    Console.WriteLine($"Error retrieving data: {ex.Message}");
}
}

}

if (string.IsNullOrEmpty(byb_key) || string.IsNullOrEmpty(bin_key) ||
    string.IsNullOrEmpty(bin_sec) || string.IsNullOrEmpty(byb_sec))
{
    LockBox.Opacity = 15;
    Lock.ZIndex = 3;
    LockBox.ZIndex = 3;
    LockText.ZIndex = 3;
}

else
{
    Lock.Opacity = 0;
    LockBox.Opacity = 0;
    LockText.Opacity = 0;
    Lock.ZIndex = -1;
    LockBox.ZIndex = -1;
    LockText.ZIndex = -1;
}

}

}

0 references
private void Threshold_OnGotFocus(object? sender, GotFocusEventArgs e)
{
    Threshold.Foreground = Brushes.White;
}

}

0 references
private void Threshold_OnLostFocus(object? sender, RoutedEventArgs e)
{
    Threshold.Foreground = Brushes.Black;
}

}

0 references
private void Size_OnGotFocus(object? sender, GotFocusEventArgs e)
{
    Size.Foreground = Brushes.White;
}

}

0 references
private void Size_OnLostFocus(object? sender, RoutedEventArgs e)
{
    Size.Foreground = Brushes.Black;
}

}

0 references
private void Frequency_OnGotFocus(object? sender, GotFocusEventArgs e)
{
    Frequency.Foreground = Brushes.White;
}

}

0 references
private void Frequency_OnLostFocus(object? sender, RoutedEventArgs e)
{
    Frequency.Foreground = Brushes.Black;
}

}
}

```

Transaction:

Transaction.axaml

```
<UserControl xmlns="https://github.com/avaloniaui/"
              xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
              xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
              xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
              xmlns:viewModels="clr-namespace:AutoArbitrage_WVM.ViewModels"
              mc:Ignorable="d" d:DesignWidth="1088" d:DesignHeight="621"
              x:DataType="viewModels:TransactionsViewModel"
              x:Class="AutoArbitrage_WVM.Views.Transactions">

    <Grid Background="#121212">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>

        <!-- Header Section -->
        <TextBlock Grid.Row="0" Text="Current Transactions" FontSize="24" FontWeight="Bold"
                  Foreground="White" HorizontalAlignment="Left" Margin="24,0,0,30"/>

        <!-- Headers Row -->
        <Grid Grid.Row="1" Background="#000000" VerticalAlignment="Center">
            <Grid.RowDefinitions>
                <RowDefinition Height="40" />
            </Grid.RowDefinitions>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="*" />
            </Grid.ColumnDefinitions>

            <TextBlock Text="Price" FontWeight="SemiBold" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center" />
            <TextBlock Grid.Column="1" Text="Quantity" FontWeight="SemiBold" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center" />
            <TextBlock Grid.Column="2" Text="Exchange" FontWeight="SemiBold" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center" />
            <TextBlock Grid.Column="3" Text="Currency" FontWeight="SemiBold" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center" />
            <TextBlock Grid.Column="4" Text="Action" FontWeight="SemiBold" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center" />
            <TextBlock Grid.Column="5" Text="Profit" FontWeight="SemiBold" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center" />
            <TextBlock Grid.Column="6" Text="Date" FontWeight="SemiBold" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center" />
            <TextBlock Grid.Column="7" Text="Time" FontWeight="SemiBold" Foreground="White" HorizontalAlignment="Center" VerticalAlignment="Center" />
        </Grid>

        <!-- Orders List -->
        <Grid Grid.Row="2">
            <Grid.RowDefinitions>
                <RowDefinition Height="1000" />
            </Grid.RowDefinitions>
            <ItemsControl ItemsSource="{Binding Orders}">
                <ItemsControl.ItemTemplate>
                    <DataTemplate>
                        <Grid Margin="0, 5, 0, 5">
                            <Grid.ColumnDefinitions>
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                            </Grid.ColumnDefinitions>

                            <TextBlock Text="{Binding Price}" HorizontalAlignment="Center" />
                            <TextBlock Grid.Column="1" Text="{Binding Quantity}" HorizontalAlignment="Center" />
                            <TextBlock Grid.Column="2" Text="{Binding Exchange}" HorizontalAlignment="Center" />
                            <TextBlock Grid.Column="3" Text="{Binding Currency}" HorizontalAlignment="Center" />
                            <TextBlock Grid.Column="4" Text="{Binding Action}" HorizontalAlignment="Center" />
                            <TextBlock Grid.Column="5" Text="{Binding Profit}" HorizontalAlignment="Center" />
                            <TextBlock Grid.Column="6" Text="{Binding Date}" HorizontalAlignment="Center" />
                            <TextBlock Grid.Column="7" Text="{Binding Time}" HorizontalAlignment="Center" />
                        </Grid>
                    </DataTemplate>
                </ItemsControl.ItemTemplate>
            </ItemsControl>
        </Grid>
    </Grid>
</UserControl>
```